การแข่งขันพัฒนาโปรแกรมคอมพิวเตอร์แห่งประเทศไทย ครั้งที่ 28

The 28th National Software Contest: NSC 2026

NSC 2026

ระบบเพิ่มประสิทธิภาพเส้นทางรถเก็บขยะหลายคันบนโครงข่ายถนนจริงด้วยอัลกอริทึมไฮบริดเมตาฮิวริสติก

A Hybrid Metaheuristic Optimization System for Multi-Truck Waste Collection Routing on Real Road Networks

หมวดการแข่งขัน	หมวด 23 โปรแกรมเพื่อการประยุกต์ใช้งาน
ระดับการแข่งขัน	ระดับนักเรียน

ผู้พัฒนาโครงงาน	นาย รัชชานนท์ ปรีชา
สถานศึกษา	โรงเรียนแสงทองวิทยา
ครูที่ปรึกษา	นางสาว ปัณทิตา ก้องกิตไพศาล
หัวหน้าสถาบัน	ดร. ถนัด อนันต์

สิงหาคม 2569 / August 2026

กิตติกรรมประกาศ

ACKNOWLEDGEMENTS

ผู้พัฒนาขอกราบขอบพระคุณการแข่งขันพัฒนาโปรแกรมคอมพิวเตอร์แห่งประเทศไทย ครั้งที่ 28 (NSC 2026) ซึ่งดำเนินงานโดยสำนักงานพัฒนาวิทยาศาสตร์และเทคโนโลยีแห่งชาติ (สวทช.) ร่วมกับศูนย์ประสานงานภูมิภาค ที่เปิดโอกาสให้เยาวชนพัฒนาซอฟต์แวร์จากแนวคิดไปสู่ต้นแบบที่ต้องผ่านการออกแบบ การทดลอง การตรวจสอบ และการนำเสนออย่างเป็นระบบ โครงการนี้ผ่านเข้ารอบชิงชนะเลิศและได้รับทุนสนับสนุนการพัฒนาผลงาน โดยศูนย์ประสานงานภูมิภาคดำเนินการด้านการสนับสนุนทุน เงินสนับสนุนดังกล่าวมีส่วนช่วยในการจัดเตรียมข้อมูลถนนจริง การทดลองซ้ำ การเก็บหลักฐาน และการจัดทำเอกสารให้ผลของโครงการสามารถตรวจสอบย้อนกลับได้อย่างเป็นระบบ

ขอขอบพระคุณศูนย์ประสานงานภาคใต้ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตหาดใหญ่ สำหรับการประสานงานและการสนับสนุนตลอดกระบวนการแข่งขัน รวมถึงสำนักงานการวิจัยแห่งชาติ (วช.) ซึ่งสนับสนุนงบประมาณดำเนินงานของการแข่งขันตามข้อมูลในคู่มือ NSC 2026 ขอกราบขอบพระคุณ นางสาวปัณทิตา ก้องกิตไพศาล ครูที่ปรึกษา และโรงเรียนแสงทองวิทยา สำหรับคำปรึกษา การตรวจทาน และการสนับสนุนให้การพัฒนาดำเนินไปอย่างมีวินัยและรับผิดชอบต่อหลักฐาน ท้ายที่สุด ขอขอบคุณชุมชน OpenStreetMap ตลอดจนผู้พัฒนาไลบรารีและเครื่องมือโอเพนซอร์สที่เป็นฐานสำคัญของระบบ ซึ่งทำให้การสร้างข้อมูล การตรวจสอบ และการทดลองสามารถดำเนินการได้อย่างโปร่งใสและทำซ้ำได้

การสนับสนุนโครงการ

ทุนพัฒนาผลงานรอบข้อเสนอโครงการ • NSC 2026 / สวทช.
• ศูนย์ประสานงานภาคใต้

บทคัดย่อ

THAI ABSTRACT

โครงการนี้พัฒนา FleetRoute เป็นระบบช่วยวางแผนและส่งต่อแผนเส้นทางรถหลายคันสำหรับปัญหา ACVRPTW บนโครงข่ายถนนจริงแบบมีทิศทาง เชื่อมข้อมูล OpenStreetMap ตัวแก้เส้นทาง C++17 ตัวตรวจข้อจำกัดอิสระ และส่วนใช้งานของผู้วางแผน/ผู้ขับ โดยยืนยันแผนเฉพาะเมื่อผ่านการตรวจลูกค้าครบ ความจุ กรอบเวลา และจำนวนรถ

แกนคำนวณเริ่มจาก “ตัวแก้เส้นทางรุ่นรอบภูมิภาค” หรือ backend ที่ใช้ในการนำเสนอรอบภูมิภาค ซึ่งเสมอกับ Google OR-Tools ที่ชุดถนนจริง 50 จุด และให้ระยะทางสั้นกว่าที่ 100, 300 และ 450 จุดตามโปรโตคอลในขณะนั้น หลังรอบภูมิภาคได้ปรับนิยามต้นทุนถนนให้ตรงกับ objective ระยะทาง ยกเลิกตัวคูณช่วงเร่งด่วนที่ไม่ได้อยู่ใน input เพิ่มนโยบายค้นหาตามขนาด และเสริมการค้นหาแบบรื้อ–ซ่อม/การใช้ incumbent พร้อม evidence gate โดยไม่อ้างว่ากลไกใดกลไกหนึ่งเป็นสาเหตุของผลทั้งหมด บนอินสแตนซ์เดิม รุ่นปัจจุบันมีค่าเฉลี่ยสั้นกว่ารุ่นรอบภูมิภาคที่ 50, 300 และ 450 จุด 0.028%, 3.228% และ 1.959% ตามลำดับ แต่ยาวขึ้น 4.734% ที่ 100 จุด จึงเห็นทั้งพัฒนาการในขนาดกลาง–ใหญ่และ regression เฉพาะสเกล รุ่นปัจจุบันผ่าน feasibility 12/12 รัน และในอินสแตนซ์ 10, 15, 20 จุดที่ CP-SAT พิสูจน์ OPTIMAL ได้ มี gap 0.3933% ทุกกรณี สำหรับการเทียบภายนอก OR-Tools ถูกทดสอบ 12 configurations ต่อขนาดและ route ทุกฝ่ายถูกประเมินซ้ำด้วยนิยาม directed-road distance เดียวกัน รุ่นปัจจุบันเสมอที่ 50 จุด แพ้ที่ 100 และชนะที่ 300/450 โดยค่าเฉลี่ยสั้นกว่า OR-Tools best-of-12 3.507%/1.970%; แม้รันที่แย่ที่สุดยังสั้นกว่า 3.204%/1.267% การทดลองวิเคราะห์ 5 คู่ seed ที่ 100 จุดพบว่ารุ่นรอบภูมิภาคชนะการตั้งค่าทดลองของรุ่นปัจจุบัน 5/5 คู่ ชี้ว่าความทนทานในการค้นพบโครงสร้างเส้นทางเฉพาะสเกลเป็นประเด็นศึกษาต่อ

คำสำคัญ: ACVRPTW; โครงข่ายถนนมีทิศทาง; เมตาฮิวริสติก; การจัดเส้นทาง

Abstract

ENGLISH ABSTRACT

FleetRoute is a decision-support application for ACVRPTW on directed real-road networks, integrating OpenStreetMap data, a C++17 routing backend, independent feasibility checks, and planner/driver interfaces.

The computational story begins with the regional-round backend used during the project's regional presentation. Under that stage's protocol it tied Google OR-Tools at 50 customers and produced shorter routes at 100, 300, and 450. Afterward, road-cost evaluation was aligned with the distance objective, an input-absent peak-hour multiplier was removed, and scale-aware destroy-and-repair/incumbent search and evidence validation were strengthened. On the same instances, the current backend's native mean is 0.028%, 3.228%, and 1.959% lower at 50, 300, and 450 customers, but 4.734% higher at 100; no single change is claimed as the sole cause. It passes feasibility in all 12/12 road runs. CP-SAT proves OPTIMAL solutions at 10, 15, and 20 customers, with a 0.3933% gap in each case. OR-Tools is tested with 12 configurations per size, with all routes re-evaluated under one common directed-road distance definition. The current backend ties at 50, loses at 100, and wins at 300/450; mean advantages are 3.507%/1.970%, and even its worst runs remain 3.204%/1.267% shorter. A separate five-seed 100-customer diagnostic gives the regional-round backend 5/5 paired wins over the current analysis setting, identifying scale-specific search robustness as the next study target.

Keywords: ACVRPTW; directed road network; hybrid metaheuristic; fleet routing

สารบัญ

TABLE OF CONTENTS

กิตติกรรมประกาศ	1
บทคัดย่อภาษาไทย	2
Abstract	3
1 บทนำ	6
2 วัตถุประสงค์และเป้าหมาย	12
3 รายละเอียดการพัฒนา	16
3.1 การออกแบบแนวคิด ผังการทำงาน และแบบจำลอง	16
3.2 ทฤษฎี หลักการ เทคนิค และเทคโนโลยีที่ใช้	18
3.3 เครื่องมือที่ใช้ในการพัฒนา	34
3.4 รายละเอียดโปรแกรมที่พัฒนาในเชิงเทคนิค	35
3.5 ขอบเขตและข้อจำกัดของโปรแกรม	43
3.6 คุณลักษณะอุปกรณ์และสภาพแวดล้อม	44
4 กลุ่มผู้ใช้โปรแกรม	46
5 การทดสอบและประเมินผล	50
5.1 คำถามการประเมินและหลักการอ่านผล	51
5.2 ชุดทดสอบและเงื่อนไขที่ควบคุมร่วมกัน	52
5.3 ระเบียบวิธีทดลองของแกนคำนวณ	53
5.4 ความสมบูรณ์ของหลักฐานและความเป็นไปได้ของคำตอบ	54

5.5 ผลการทดลองซ้ำของแกนคำนวณ 54
5.6 การเปรียบเทียบกับ Google OR-Tools 58
5.7 วิเคราะห์กรณี 100 จุด: เหตุใดรุ่นปัจจุบันจึงถดถอย 63
5.8 ความหมายเชิงประยุกต์ 66
5.9 ข้อจำกัดและภัยคุกคามต่อความเที่ยงตรง 67
5.10 สรุปผลการทดสอบ 68
5.11 ทะเบียนหลักฐานและการตรวจสอบย้อนกลับ 69
6 ปัญหาและอุปสรรค 70
7 แนวทางการพัฒนาและประยุกต์ใช้ในขั้นต่อไป 74
8 ข้อสรุปและข้อเสนอแนะ 78
9 เอกสารอ้างอิง 81
10 สถานที่ติดต่อผู้พัฒนาและอาจารย์ที่ปรึกษา 84
ภาคผนวก 86

1 บทนำ
INTRODUCTION

1.1 ที่มาและความสำคัญของปัญหา
BACKGROUND AND MOTIVATION

ปัญหาการจัดเส้นทางรถมีรากฐานจาก Truck Dispatching Problem และพัฒนาเป็นตระกูล Vehicle Routing Problem (VRP) ที่ใช้กว้างขวางในโลจิสติกส์และบริการภาคสนาม [1], [3], [4] ในงานจริง ความผิดพลาดไม่ได้เกิดจากระยะทางเพียงอย่างเดียว แต่เกิดจากแผนที่ไม่สอดคล้องกับทิศทางถนน ความจุรถ เวลาบริการ และช่วงเวลาที่เข้าถึงจุดงาน ส่งผลให้เกิดระยะทางส่วนเกิน ความล่าช้า การใช้รถเกินความจำเป็น หรือแม้แต่แผนที่นำไปปฏิบัติไม่ได้ ผลต่างเพียงเล็กน้อยต่อหนึ่งเที่ยวจึงสามารถสะสมเป็นต้นทุนระดับกองรถเมื่อเกิดซ้ำตลอดหลายคันและหลายวันทำงาน

ตัวอย่างผลกระทบแบบสมมติ

หากกองรถ 100 คันวิ่งเฉลี่ย 120 กิโลเมตรต่อวัน เป็นเวลา 300 วันต่อปี ระยะทางฐานเท่ากับ 3,600,000 กิโลเมตรต่อปี การลดเพียง 2% เท่ากับลดระยะทาง 72,000 กิโลเมตร ต่อปี ตัวเลขนี้เป็นสถานการณ์ตัวอย่างเพื่อแสดงการสะสมของผลประโยชน์และ potential ที่ส่งผลต่อภาคเศรษฐกิจอย่างมหาศาล ไม่ใช่ผลประหยัดที่วัดจากหน่วยงานจริง

$$\text{Annual distance avoided} = \text{fleet} \times \text{km/day} \times \text{operating days} \times \text{improvement rate} \quad (1)$$

ความยากเชิงปฏิบัติไม่ได้อยู่ที่การเรียงลำดับจุดเพียงอย่างเดียว ถนนจริงเป็นกราฟแบบมีทิศทาง ต้นทุนจากจุด i ไป j อาจไม่เท่ากับจาก j ไป i และบางคู่ต้องอ้อมจากข้อจำกัดของถนนทางเดียว จุดกลับรถ สะพาน หรือการเข้าถึงที่ขาดหาย การใช้ระยะเส้นตรงแบบ Euclidean จึงอาจให้คำตอบที่ดูสั้นบนแผนภาพแต่ไม่สอดคล้องกับเส้นทางที่รถต้องวิ่งจริง

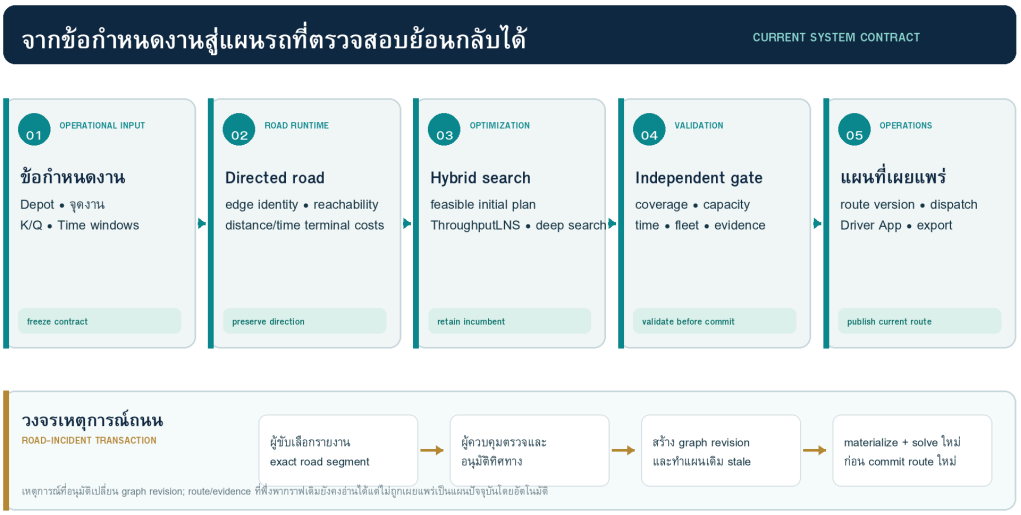
โครงการนี้จึงทำให้กราฟถนนเป็นส่วนหนึ่งของแบบจำลองและสัญญาข้อมูลตั้งแต่ต้น ไม่ใช่เพียงชั้นแสดงผลหลังการคำนวณ

1.2 โครงสร้างระบบจากการคำนวณสู่การปฏิบัติงาน

FROM OPTIMIZATION CORE TO OPERATIONAL DECISION SUPPORT

ระบบแบ่งเป็นแกนคำนวณและชั้นปฏิบัติการที่เชื่อมกันด้วยสัญญาข้อมูลเดียวกัน แกนคำนวณรับผิดชอบการแทนปัญหา การสร้างคำตอบ การค้นหา และการตรวจข้อจำกัด ส่วนชั้นปฏิบัติการรับผิดชอบการกำหนดจุดงาน สถานะโครงข่าย การส่งแผนให้ผู้ขับ และการจัดการเหตุการณ์ถนน การแบ่งขอบเขตนี้ทำให้ความหมายของคำตอบไม่ขึ้นกับหน้าจอ และทำให้ทุกแผนที่เผยแพร่สามารถย้อนกลับไปยัง input, graph revision, คำสั่งที่ใช้ และหลักฐานของรอบคำนวณได้

ผู้วางแผนทำงานผ่านพื้นที่ map-first เพื่อกำหนด depot จุดงาน fleet ceiling ความจุ และเหตุการณ์ถนน ผู้ขับได้รับใบงานเฉพาะรถที่ถูกใช้งานจริงและสามารถรายงาน road segment ที่มีปัญหา ขณะที่ backend ดูแล graph revision, data contract, job lifecycle, route version และ evidence chain ความซับซ้อนของคำสั่งคอมไพล์และพารามิเตอร์ภายในจึงถูกซ่อนไว้จากผู้ใช้ทั่วไป แต่ยังเปิดให้ผู้ตรวจสอบเข้าถึง invocation, log และหลักฐานที่จำเป็นต่อการตรวจย้อนกลับ



รูปที่ 1-1 สถาปัตยกรรมเชิงปฏิบัติการจากข้อมูลถนนและข้อกำหนดงานสู่แผนที่ผ่านการตรวจ

ทุกชั้นอ้างอิง data identity และ routing contract ชุดเดียวกัน เมื่อเหตุการณ์ถนนเปลี่ยน graph revision ผลลัพธ์ที่พึ่งพากราฟเดิมจะถูกทำเครื่องหมายว่าไม่เป็นปัจจุบันและต้องผ่านการประเมินใหม่ก่อนเผยแพร่เป็นแผนฉบับถัดไป

1.3 นิยามปัญหาในโครงการ

PROBLEM DEFINITION

โครงการศึกษาปัญหา Asymmetric Capacitated Vehicle Routing Problem with Time Windows (ACVRPTW) บนกราฟถนนจริงแบบมีทิศทาง โดยมีศูนย์ต้นทาง (depot) เดียวและกองรถ homogeneous แบบหลายคัน แต่ละจุดงานมี demand, service time และ time window ส่วนรถมีความจุต่อเที่ยวจำกัดและสามารถกลับ depot เพื่อเริ่มเที่ยวถัดไปได้เมื่อเปิดใช้ multiple trips (MTCVRPTW) จำนวนรถ K หมายถึงจำนวนรถที่พร้อมใช้สูงสุด ไม่ใช่จำนวนเส้นทางที่ต้องสร้าง ดังนั้นคำตอบที่ใช้รถน้อยกว่า K ยังคงถูกต้องเมื่อครอบคลุมจุดงานครบและผ่านข้อจำกัดทั้งหมด

- Asymmetric: ต้นทุนไปและกลับอาจไม่เท่ากันเพราะทิศทางถนนและโครงสร้างเครือข่าย
- Capacitated: ปริมาณงานรวมภายในแต่ละเที่ยวต้องไม่เกินความจุ Q

- Time Windows: เวลาเริ่มบริการต้องอยู่ในช่วง $[a_i, b_i]$ เมื่อเปิดการบังคับใช้
- Multiple trips: รถคันเดิมอาจกลับ depot เพื่อรีเซ็ตความจุ แต่เวลาในวันทำงานยังเดินต่อเนื่อง
- Maximum fleet: จำนวนรถที่ใช้งานจริง m ต้องเป็น $1 \le m \le K$ และไม่แสดงรถว่างใน Driver App

1.4 คำถามวิจัย

RESEARCH QUESTIONS

ตารางที่ 1-1 คำถามวิจัยและสัญญาหลักฐาน

รหัส	คำถาม	หลักฐานที่ใช้ตอบ
RQ-1	ระบบแทนปัญหา ACVRPTW บนถนนจริงแบบมีทิศทางอย่างสอดคล้องกันตั้งแต่ข้อมูลถึงผลลัพธ์หรือไม่	data contract, terminal matrix, independent validator
RQ-2	แกน C++ และ backend ส่งต่อ candidate ตรวจข้อจำกัด และบันทึก route version ได้ถูกต้องหรือไม่	native solver run, handoff, feasibility gate, atomic commit
RQ-3	หน้าจอสื่อความแตกต่างระหว่างโครงข่าย แผนเบื้องต้น และผล Optimizer ตามสถานะจริงหรือไม่	route lifecycle, UI semantics, persisted state transitions
RQ-4	เหตุการณ์ถนนเปลี่ยนกราฟและกระตุ้นการจัดเส้นทางใหม่โดยไม่ทำลายแผนเดิมได้หรือไม่	edge event, stale state, transactional rerun
RQ-5	ตัวแก้ของโครงการให้คุณภาพคำตอบระดับใดเมื่อเทียบกับ optimum ที่พิสูจน์ได้บนชุดเล็กและ Google OR-Tools baseline บนชุดถนนจริงภายใต้หลักฐานที่ตรวจย้อนกลับได้	CP-SAT optimum, repeated OSM runs, convergence, OR-Tools best-of-12 และ evidence boundary

1.5 ส่วนสมทบของโครงการ

CONTRIBUTIONS

1. แบบจำลองและ data contract สำหรับ ACVRPTW บน directed road graph ที่รักษาระยะทาง เวลา การเข้าถึง และความหมายของ maximum available fleet อย่างสอดคล้องกัน
2. แกนตัวแก้ C++17 แบบไฮบริดที่แยกบทบาทการสร้างคำตอบ การกระจายพื้นที่ค้นหา การรื้อ–ซ่อมโครงสร้างเส้นทาง การเร่งค้นหารอบ incumbent และการขัดเกลาผลสุดท้าย โดยทุกเฟสใช้แบบจำลองต้นทุนและเงื่อนไข feasibility ชุดเดียวกัน
3. feasibility and evidence gate ที่ตรวจคำตอบอย่างอิสระ เก็บ command, stdin hash, stdout, stderr, convergence และ route version ก่อนยืนยันแผนฉบับใหม่
4. แอปพลิเคชันสำหรับผู้วางแผน ผู้ขับ และผู้ตรวจสอบ พร้อม road-incident workflow และ event-driven rerouting; ขอบเขตปัจจุบันใช้เหตุการณ์ถนนที่มีผู้รายงานการปิดถนนและอนุมัติการจัดเส้นทางใหม่
5. กรอบหลักฐานที่แยก implementation inspection, contract/state verification, exact small-case validation และ repeated solver benchmark ออกจากกัน พร้อมทะเบียนไฟล์ แฮช คำสั่ง นโยบายไดนามิก และผลรันที่ตรวจย้อนกลับได้

1.6 หลักการรายงานผล

REPORTING CONVENTIONS

ระดับของผลลัพธ์

ผลจากการตรวจ implementation และ workflow ใช้ยืนยันความสอดคล้องของสัญญาข้อมูลและสถานะระบบ ส่วนคุณภาพเส้นทางสรุปจาก exact small-case, repeated road runs และ external baselines แยกต่างหาก โดยไม่ใช้ผลประเภทหนึ่งแทนอีกประเภท และไม่อ้าง BKS หรือผลประหยัดภาคสนามเมื่อหลักฐานไม่รองรับ

2 วัตถุประสงค์และเป้าหมาย
OBJECTIVES AND SUCCESS CRITERIA

2.1 วัตถุประสงค์ของโครงการ
PROJECT OBJECTIVES

ตารางที่ 2-1 วัตถุประสงค์และผลส่งมอบ

ลำดับ	วัตถุประสงค์	ผลส่งมอบที่ตรวจสอบได้
O1	สร้างปัญหาเส้นทางบนถนนจริงแบบมีทิศทาง	OpenStreetMap(OSM) graph, reachability checks, terminal matrix
O2	รองรับ capacity, service, time windows, multiple trips และ fleet ceiling	shared data contract และ independent validator
O3	พัฒนาตัวแก้ hybrid metaheuristic ที่ให้ผลได้ตลอดเวลา	C++17 solver, convergence trace, configurable budget
O4	ตรวจ candidate อย่างอิสระก่อนยืนยันแผน	feasibility gate, fleet cross-check, atomic route commit
O5	ทำให้ระบบใช้งานได้โดยผู้วางแผนและคนขับ	map-first workspace, Driver App, road reports, simulation
O6	รองรับการเปลี่ยนแปลงของถนนอย่างควบคุมได้	incident approval, graph revision, stale state, rerun
O7	ทำให้ผลย้อนกลับไปยังหลักฐานได้	route version, invocation manifest, logs, hashes, exports
O8	ตรวจระบบและประเมินสมรรถนะของ solver	contract/state checks, exact small cases, repeated OSM benchmark และ baseline comparison

2.2 เกณฑ์ความสำเร็จ

MEASURABLE SUCCESS CRITERIA

$$\text{Success} = \text{Feasibility} \wedge \text{Evidence} \wedge \text{Reproducibility} \wedge \text{Practical utility} \quad (2)$$

เกณฑ์ดังกล่าวตั้งใจป้องกันการสรุปความสำเร็จจาก objective เพียงค่าเดียว: คำตอบที่สั้นแต่ละเมิดข้อจำกัดไม่ใช่คำตอบที่ใช้ได้ และคำตอบที่ถูกต้องแต่ไม่สามารถระบุ input, configuration และหลักฐานต้นทางก็ยังไม่เพียงพอสำหรับงานที่ต้องตรวจสอบและทำซ้ำได้ ดังนั้นความสำเร็จของระบบต้องเกิดพร้อมกันทั้ง feasibility, evidence, reproducibility และ practical utility

ตารางที่ 2-2 เกณฑ์รับรองระบบ

มิติ	เกณฑ์รับรอง	หลักฐาน
Feasibility	ลูกค้าครบหนึ่งครั้ง; overload=0; lateness=0; โครงสร้างเที่ยวถูกต้อง	validator และ fleet cross-check
Correctness	objective, route และสถานะสอดคล้องแบบจำลองเดียวกัน	shared matrix, parser และ independent validator
Integration	backend เรียก native C++ และรับผลตามสัญญา	actual solver run, handoff และ process evidence
Reproducibility	ระบุ data identity, seed, command, build และไฟล์ผล	JSON/CSV/log/checksum
Application	ผู้วางแผนทำ workflow หลักได้โดยไม่แก้โค้ด	guided UI, two-cell notebook และ persisted workflow state
Truthful status	ไม่ประกาศ routing complete ก่อน optimizer+validator	route-state transitions และ transaction receipts

มิติ	เกณฑ์รับรอง	หลักฐาน
Performance evidence	สรุปคุณภาพ/เวลาเมื่อมี benchmark แบบควบคุมเท่านั้น	บทที่ 5: ผลรันซ้ำ อินพุตตรงกันทุกไบต์ และ baseline ที่มีหลักฐานครบ

2.3 ตัวชี้วัดสำหรับการประเมินแกนคำนวณ

PLANNED EVALUATION METRICS

$$\text{Gap}(\%) = 100 \times (z_H - z^*) / z^* \quad (3)$$

$$\text{Improvement}(\%) = 100 \times (z_base - z_final) / z_base \quad (4)$$

การประเมินใช้ gap เฉพาะชุดเล็กที่ CP-SAT รายงาน OPTIMAL และ objective เท่ากับ best bound เท่านั้น สำหรับชุดถนนจริงที่ไม่มีค่าที่เหมาะที่สุดซึ่งพิสูจน์ได้ รายงานแยกใช้ feasible rate, final distance, vehicles used, wall-clock runtime, การกระจายจากสามรัน และ external baseline ที่ใช้สัญญาต้นทุนเดียวกัน ทุกกรณีเก็บ raw route, validator result, command, resolved policy, checksum และ convergence trace เพื่อให้ตัวเลขในบทผลทดลองย้อนกลับไปยังหลักฐานต้นทางได้

2.4 การเชื่อมวัตถุประสงค์กับบทผลทดลอง

OBJECTIVE–EVIDENCE TRACEABILITY

ตารางที่ 2-3 เส้นทางจากวัตถุประสงค์สู่หลักฐาน

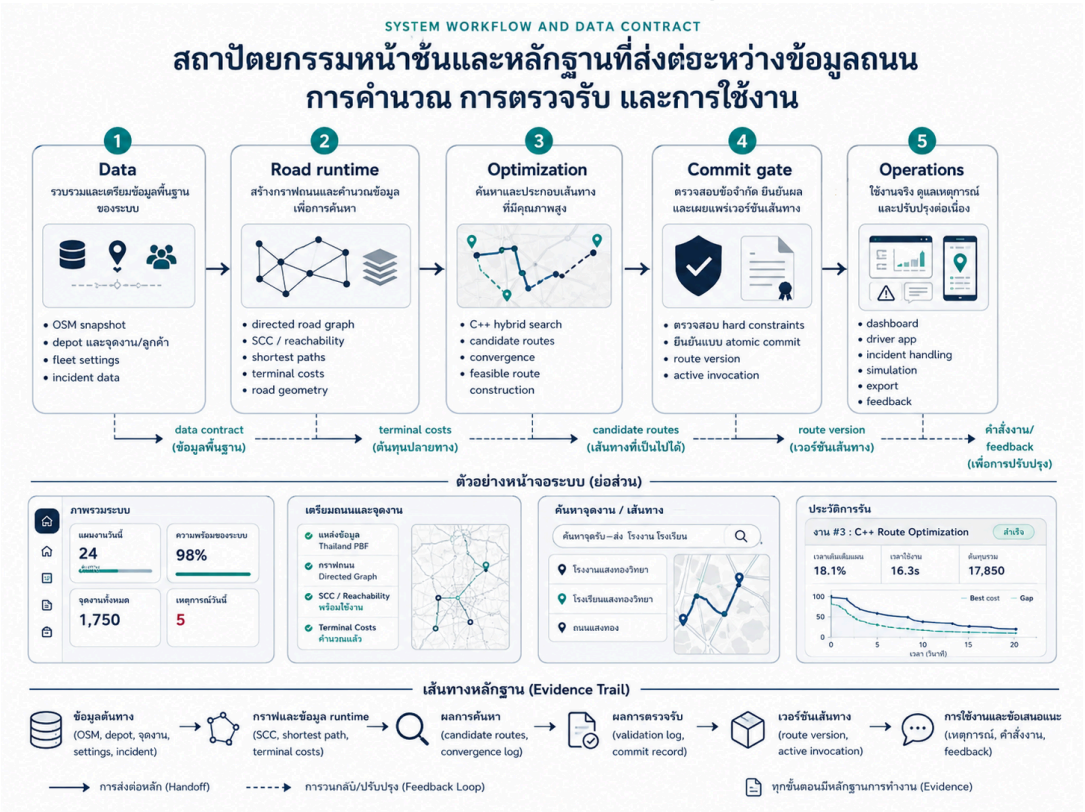
วัตถุประสงค์	หัวข้อผล	คำตัดสิน
O1–O4	5.2–5.4	สัญญาข้อมูล native solver และ route-commit contract ผ่านการตรวจระบบ

วัตถุประสงค์	หัวข้อผล	คำตัดสิน
O5–O7	5.3–5.4	workflow, road incident, route state และ evidence contract เชื่อมกับสถานะที่บันทึกได้
O8	5.1–5.6	ผลรันซ้ำของแกนคำนวณ, exact small cases และ OR-Tools baseline เชื่อมกลับไปยังหลักฐานครบ
O8	5.6.2, 5.7 และ 5.9	ตรวจ exact small case วิเคราะห์ความแปรผันกรณี 100 จุด และกำหนดขอบเขตการสรุปอย่างตรงไปตรงมา

3 รายละเอียดการพัฒนา
SYSTEM AND METHOD DEVELOPMENT

3.1 การออกแบบแนวคิด ผังการทำงาน และแบบจำลอง
DESIGN AND STORYBOARD

ระบบใช้สถาปัตยกรรมแบบแยกชั้นระหว่างข้อมูลถนน การเตรียมอินสแตนซ์ แกนคำนวณ การตรวจรับ และการใช้งาน แต่ละชั้นส่งต่อวัตถุที่มีความหมายชัดเจนภายใต้ data contract เดียวกัน จึงสามารถเปลี่ยนหน้าจอหรือวิธีนำเสนอได้โดยไม่เปลี่ยนความหมายของ input.txt หรือคำตอบของ solver และทำให้ C++ core ใช้งานได้ทั้งจากแอปพลิเคชันและจาก notebook สำหรับการทดลองที่ควบคุมได้



รูปที่ 3-1 สถาปัตยกรรมห้าชั้นและหลักฐานที่ส่งต่อระหว่างข้อมูลถนน การคำนวณ การตรวจรับ และการใช้งาน

วงจรเหตุถนนเริ่มจากชั้นใช้งาน สร้าง graph revision ใหม่ แล้วเดินผ่านทุกชั้นก่อนเผยแพร่ route version ใหม่; หากตรวจไม่ผ่านให้คงแผนล่าสุดที่ผ่านการตรวจ

ตารางที่ 3-1 ความรับผิดชอบของแต่ละชั้น

ชั้น	หน้าที่	สิ่งที่ส่งต่อ
Data	เก็บ OSM snapshot, depot, จุดงาน, fleet settings และเหตุการณ์ถนน	ข้อมูลที่มี identity และ revision
Road runtime	สร้างกราฟแบบมีทิศทาง ตรวจ SCC/reachability และ shortest paths	terminal costs และ road geometry
Optimization	สร้างแผนเบื้องต้นและปรับปรุงด้วย C++17 hybrid search	candidate routes + convergence
Commit gate	ตรวจ hard constraints และเชื่อมผลกับ exact invocation	route version ที่ยืนยันแล้ว
Operations	แสดงแผน ส่งใบงาน รับ incident จำลอง และ export	คำสั่งงานและ feedback

3.1.1 เนื้อเรื่องย่อการใช้งาน (Storyboard)

OPERATIONAL STORYBOARD

1. ผู้วางแผนกำหนดศูนย์ต้นทางและเพิ่มจุดงาน ระบบตรวจว่าทุกจุดไป–กลับบน directed road graph ได้
2. ระบบเตรียมกราฟและสัญญาข้อมูล แล้วแสดงว่าโครงข่ายพร้อมโดยยังไม่เรียกว่าการจัดเส้นทางเสร็จ
3. ระบบสร้างแผนเบื้องต้นที่ feasible และแสดงเป็นเส้นประ เพื่อให้ตรวจโครงสร้างก่อนเริ่ม optimizer
4. ผู้วางแผนเลือก objective, K, Q และงบเวลา จากนั้น ThroughputLNS กับ deep search ทำงานตาม phase
5. เมื่อ C++ จบ ตัวตรวจอิสระคำนวณข้อจำกัดซ้ำและ commit route version เฉพาะผลที่ผ่าน
6. ผู้วางแผนส่งใบงานตามรถ คนขับเห็นลำดับจุดและเปิด navigation handoff ได้
7. เมื่อพบถนนใช้การไม่ได้ คนขับเลือกรายงาน exact segment ผู้ควบคุมอนุมัติ แล้วระบบทำเครื่องหมายแผนเดิมว่า stale ก่อนคำนวณใหม่

3.2 ทฤษฎี หลักการ เทคนิค และเทคโนโลยีที่ใช้

> หมายเหตุเกี่ยวกับพัฒนาการของตัวแก้เส้นทาง

หัวข้อนี้อธิบายหลักการและองค์ประกอบของตัวแก้เส้นทาง **รุ่นปัจจุบัน** ซึ่งเป็นรุ่นที่ใช้สำหรับผลการทดลองหลักในรายงานฉบับนี้ สำหรับรายละเอียดเชิงทฤษฎีของกลไกค้นหาและคำอธิบาย operator และ Sparse Spectral Clustering ใน

ตัวแก้เส้นทางรุ่นที่ใช้ในการนำเสนอรอบภูมิภาค

สามารถศึกษาเพิ่มเติมได้จาก ไฟล์*รายงานความก้าวหน้าโครงงานที่ถูกอัปโหลดบน* https://www.nstda.or.th/sims *หัวข้อ 3.2 “ทฤษฎี หลักการ เทคนิค และเทคโนโลยีที่ใช้” หน้า 39* ตามรายงาน ทั้งนี้ รายงานความก้าวหน้าดังกล่าวใช้เพื่ออธิบายที่มาและพัฒนาการของแนวคิดเท่านั้น รายละเอียดใดที่แตกต่างกันเช่น peak-hour multipler และ ThroughputLNS ให้ยึดสถาปัตยกรรม การตั้งค่า และผลการทดลองในรายงานฉบับสมบูรณ์นี้เป็นหลัก

3.2.1 แบบจำลองปัญหา ACVRPTW

PROBLEM FORMULATION

ให้ $G=(V,E)$ เป็นกราฟถนนแบบมีทิศทาง โดยมี depot 0 และเซตจุดงาน $C \subset V$ สำหรับแต่ละจุด $i \in C$ กำหนด demand q_i, service time s_i และ time window $[a_i,b_i]$ กองรถมีรถพร้อมใช้สูงสุด K คันและความจุต่อเที่ยว Q เส้นทางของรถคันหนึ่งเริ่มและจบที่ depot และอาจมี depot markers ภายในเพื่อแบ่งเป็นหลายเที่ยว

$$R = \{R_1, \ldots, R_m\},\ 1 \le m \le K \quad (5)$$

$$\forall\, i \in C : \sum_r 1[i \in R_r] = 1 \quad (6)$$

$$\forall \text{ trip } \tau : \sum_{i \in \tau} q_i \le Q \quad (7)$$

$$B_j \ge \max(a_j, B_i + s_i + t_{ij}),\ B_j \le b_j \quad (8)$$

สมการ (6) บังคับให้ทุกจุดงานถูกบริการหนึ่งครั้ง สมการ (7) จำกัดโหลดในแต่ละเที่ยว และสมการ (8) อัปเดตเวลาเริ่มบริการเมื่อเปิดใช้ time windows รถที่กลับ depot จะรีเซ็ตความจุ แต่ clock ของรถคันนั้นยังต่อเนื่อง เพื่อไม่ให้หลายเที่ยวซ้อนเวลากันโดยไม่ตั้งใจ

3.2.2 ต้นทุนจากกราฟถนน

DIRECTED TERMINAL COSTS

สำหรับ terminal ทุกคู่ ระบบคำนวณ shortest path บนกราฟแบบมีทิศทาง ไม่สมมติว่า $c_{ij}=c_{ji}$ และเก็บทั้งระยะทาง d_{ij} กับระยะเวลา t_{ij} การได้มาและวิเคราะห์กราฟถนนอาศัยแนวทางจาก OSMnx และ NetworkX [10], [11] การรายงานเส้นทางจึงเชื่อมลำดับ terminal กลับไปยัง road geometry ที่ใช้จริงได้

$$d_{ij} = \text{shortest-distance}(i,j \mid G),\ t_{ij} = \text{shortest-time}(i,j \mid G) \quad (9)$$

3.2.3 ฟังก์ชันวัตถุประสงค์

OBJECTIVE MODES

ตารางที่ 3-2 โหมดวัตถุประสงค์ที่แอปพลิเคชันและตัวตรวจใช้ร่วมกัน

โหมด	ต้นทุนที่ลด	การตีความ
Distance	$\Sigma\, d_{ij}$	ลดระยะทางบนถนน หน่วยเมตร
Time	Σ (travel + waiting + service)	ลดเวลาแผนรวมตาม time windows
Hybrid	$\Sigma\, d_{ij} + 0.25\, \Sigma\, \text{duration}_{ij}$	รวมระยะทางและเวลาในสเกลที่กำหนด

Feasibility มาก่อน objective

ระหว่าง search อาจใช้ penalty เพื่อจัดลำดับ candidate ที่ยังไม่สมบูรณ์ แต่ route version ที่ส่งให้ผู้ใช้ต้องผ่าน hard constraints และมี violation เท่ากับศูนย์ตามการตั้งค่าที่เปิดใช้

3.2.4 การเตรียมข้อมูลถนนและอินสแตนซ์

ROAD AND INSTANCE PREPARATION

1. เตรียม Thailand OSM snapshot และสร้างกราฟย่อยรอบพื้นที่ปฏิบัติงาน
2. ตรวจ depot และจุดงานทุกจุดว่าสามารถไป–กลับบนกราฟแบบมีทิศทางได้
3. จัดการเหตุการณ์ถนนที่อนุมัติโดยลบ exact directed edge และอาจลบ reverse edge ที่จับคู่ geometry ได้เพียงหนึ่งเส้นเมื่อรายงานปิดสองทิศทาง
4. คำนวณ terminal-cost matrix และเก็บ node/geometry mapping
5. สร้าง input.txt, dataset.json และ dataset_summary.json แบบ atomic พร้อม SHA-256 และ contract checks
6. สร้างแผนเบื้องต้นที่ feasible เพื่อให้ผู้ใช้เห็นโครงสร้างงานก่อนรัน optimizer

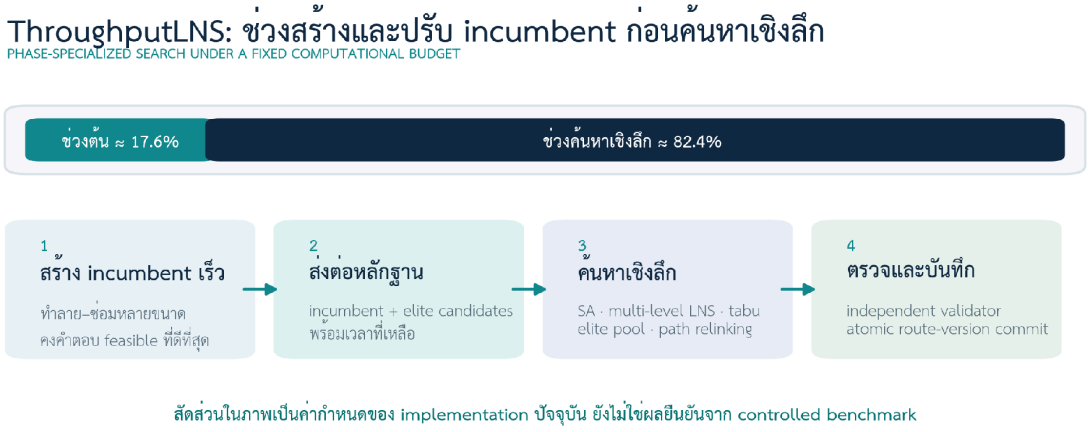
การลบถนนไม่ได้ลบถนนทุกเส้นที่มีชื่อเดียวกัน และไม่ตีความถนนสองทิศทางจากชื่อเพียงอย่างเดียว ระบบใช้ edge identity และ geometry เพื่อหลีกเลี่ยงการปิด parallel roads ที่ไม่เกี่ยวข้อง เมื่อเปิดถนนอีกครั้งหรือเหตุการณ์หมดอายุ ระบบกู้สัญญากราฟต้นทาง ทำให้ route เดิมล้าสมัย และรอการคำนวณใหม่

3.2.5 ไฮบริดเมตาฮิวริสติกและ ThroughputLNS

HYBRID METAHEURISTIC

แกนคำนวณใช้เมตาฮิวริสติกแบบหลายระยะเพื่อแยก diversification และ intensification ออกจากกันอย่างชัดเจน จุดเริ่มต้นต้องเป็นคำตอบที่ผ่านข้อจำกัด โดยนโยบายตามขนาดปัญหาเลือกได้ระหว่าง Clarke-Wright Savings และ feasible greedy certificate ดังนั้น Savings ไม่ใช่ขั้นบังคับของทุกอินสแตนซ์ จากนั้น ThroughputLNS ใช้ destroy-repair ต้นทุนต่ำเพื่อสร้าง improvement อย่างรวดเร็วและรักษา feasible incumbent ก่อนส่งต่อไปยังการค้นหาเชิงลึกที่ประกอบด้วย parallel simulated annealing, large-neighborhood reconstruction, granular tabu intensification, elite reuse/path

relinking และ final refinement ชื่อกลไกเหล่านี้อธิบายบทบาทของแต่ละช่วง และไม่ได้ใช้เพื่ออ้างว่าเป็น implementation ของ HGS แบบเต็มรูปแบบ



รูปที่ 3-2 การแบ่งงบเวลาของ ThroughputLNS และการส่งต่อคำตอบเข้าสู่ช่วงค้นหาเชิงลึก

ตารางที่ 3-3 องค์ประกอบของกระบวนการค้นหา

ระยะ	กลไก	หน้าที่
Initialization	Scale-dependent warm start: Clarke–Wright Savings หรือ feasible greedy certificate	สร้างคำตอบ feasible ภายใต้ fleet ceiling; policy สามารถปิด Savings เมื่อโครงสร้างระดับขนาดนั้นไวต่อการ merge
Early throughput	ThroughputLNS destroy–repair	สร้าง improvement ให้เร็ว คง feasible incumbent และรวบรวม elite candidates
Diversification	parallel simulated annealing	สำรวจหลาย trajectory และยอมรับการเปลี่ยนแปลงบางส่วนตามอุณหภูมิ
Reconstruction	multi-level LNS	รื้อและซ่อมคำตอบทั้งระดับเส้นทาง ภูมิภาค และ boundary
Intensification	granular tabu search	ค้นหาเพื่อนบ้านที่มีศักยภาพโดยควบคุมการย้อนกลับ
Knowledge reuse	elite pool / path relinking	ใช้โครงสร้างจากคำตอบที่ดีเพื่อสร้าง candidate ใหม่

ระยะ	กลไก	หน้าที่
Finalization	final refinement + independent validation	ขัดเกลาและรับรองผลก่อน commit

```
Algorithm 1 · ThroughputLNS, deep search and route-commit contract
1 materialize directed-road instance and validate the contract
2 S_best <- build_feasible_initial_plan(max_fleet = K)
3 early_budget <- min(0.176 * T, implementation safety caps)
4 (S_best, elites) <- ThroughputLNS(S_best, early_budget)
5 launch bounded deep-search trajectories from common handoff
6 while time remains and stopping rule is not met:
7       destroy/repair, SA, tabu, elite reuse or path relinking
8       update worker best, global best, elite pool, convergence trace
9       S_final <- final_refinement(S_best)
10 if independent_validate(S_final): atomic_commit(route_version)
11 else: preserve previous valid route and evidence
```

งบเวลาหลักเป็นค่าที่ผู้ใช้กำหนด ส่วน profile, สัดส่วน early phase, destroy ratio, safety cap และจำนวน worker ถูก resolve จากนโยบายของ implementation และบันทึกไว้กับหลักฐานทุกครั้ง ค่าดังกล่าวจึงเป็น software configuration ของระบบ ไม่ใช่พารามิเตอร์ที่พิสูจน์แล้วว่าเหมาะที่สุดเชิงสากล บทที่ 5 ประเมิน configuration แบบ end-to-end นี้ภายใต้ controlled benchmark บนอินพุตเดียวกันทุกไบต์ และแยกให้ชัดจาก hyperparameter ablation ซึ่งเป็นงานในขั้นถัดไป

3.2.6 การจัดกลุ่มเพื่อเตรียมคำตอบ

CLUSTERING AS PREPROCESSING

โมดูล clustering มีหน้าที่เตรียมโครงสร้าง ไม่ได้บังคับ final route count ให้เท่ากับ K เมื่อเลือก clustering ระบบใช้จำนวนกลุ่ม active ขั้นต่ำตาม lower bound ของ capacity ซึ่งไม่เกินจำนวนรถที่มี ส่วน

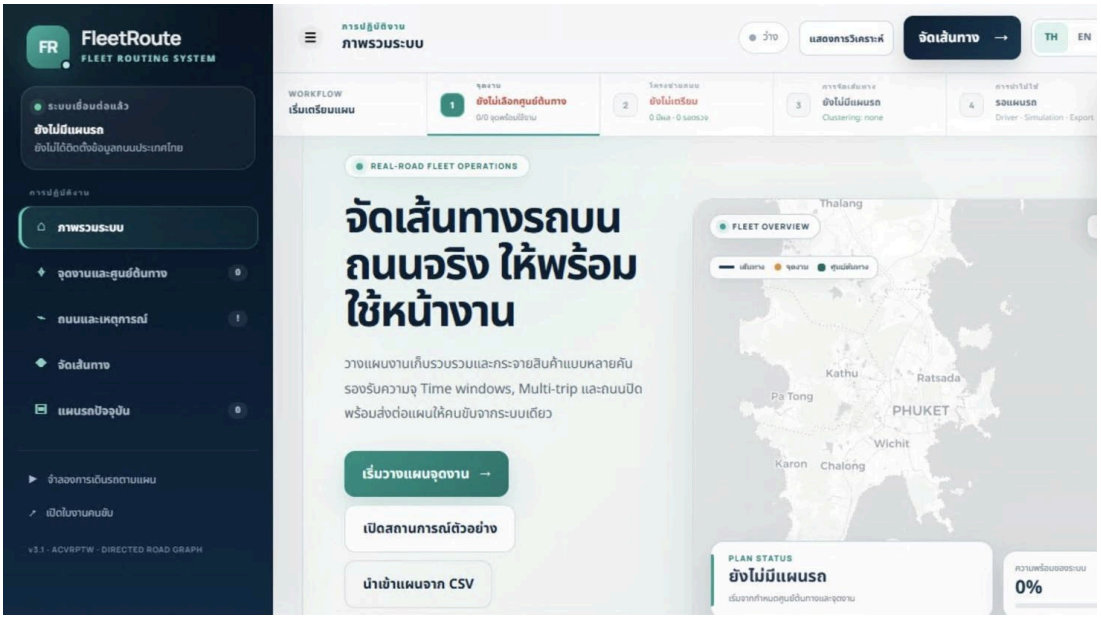
primary input และ solver ยังคงใช้ K เป็นเพดานสูงสุด คำว่า Savings ในหัวข้อนี้หมายถึงวิธีจัดกลุ่ม/สร้าง route proxy สำหรับ preprocessing และต้องแยกจากตัวเลือก Savings warm-start ภายใน native solver ซึ่ง policy สามารถเปิดหรือปิดตามขนาดปัญหาได้.

- Savings รวมกลุ่มจากค่าประโยชน์เชิงเส้นทางและใช้ route proxy เป็นสัญญาณหลัก โดยไม่บังคับจำนวนกลุ่มให้เท่ากับ K
- SparseSpectral สร้างกราฟความสัมพันธ์แบบ k-nearest-neighbour จากต้นทุนถนน หา spectral embedding ตามแนวคิด spectral clustering [8], [9] และให้น้ำหนักกับ compactness กับ balance
- ผล clustering ถูกประเมินแยกจาก downstream solver เพื่อหลีกเลี่ยงการอ้าง superiority ที่หลักฐานไม่รองรับ

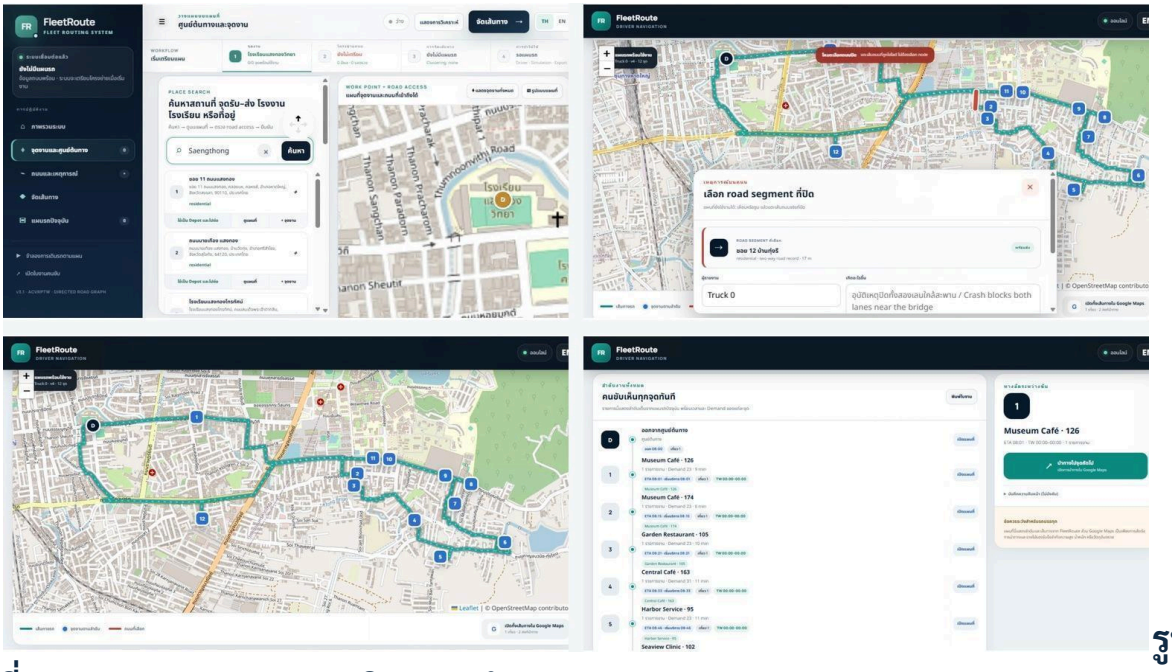
3.2.7 เทคโนโลยีและการออกแบบแอปพลิเคชัน

APPLICATION DESIGN

หน้าจอหลักจัดโครงสร้างตามงาน ไม่ใช่ตามชื่อไฟล์หรือชื่ออัลกอริทึม ผู้ใช้ทั่วไปเห็นลำดับ “จุดงานและศูนย์ต้นทาง → ถนนและเหตุการณ์ → จัดเส้นทาง → แผนรถปัจจุบัน” ขณะที่ Clustering, Convergence, exact command และ runtime inspector อยู่ในพื้นที่วิเคราะห์สำหรับผู้ตรวจสอบและนักวิจัย ภาพหน้าจอในรูปที่ 3-3 ถึง 3-5 มาจากระบบที่รันจริงและใช้เพื่อแสดง state transition, road geometry และข้อมูลที่ถูกส่งต่อระหว่างผู้วางแผนกับผู้ขับ ไม่ใช่ภาพจำลองเชิงตกแต่ง.



รูปที่ 3-3 หน้าจอภาพรวมของระบบจริง แสดงลำดับงานตั้งแต่กำหนดจุดงาน เตรียมโครงข่าย จัดเส้นทาง และส่งต่อแผนรถ



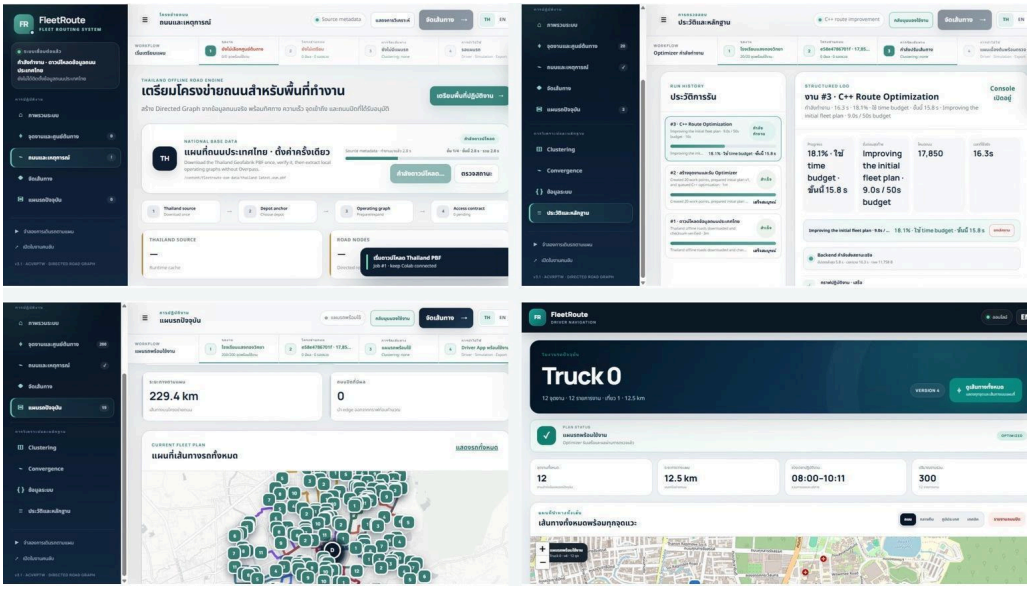
รูปที่ 3-4 มุมมองจากระบบจริง: การกำหนดจุดงาน การรายงาน exact road segment แผนที่ผู้ขับ และลำดับจุดบริการ

ตารางที่ 3-4 พื้นที่ทำงานและผู้ใช้

พื้นที่	ผู้ใช้หลัก	การตัดสินใจ
System overview	ผู้วางแผน/ผู้ตรวจสอบ	อ่านความพร้อม สถานะแผน งานล่าสุด และหลักฐาน
Plan	ผู้วางแผน	เลือก depot เพิ่ม/นำเข้าจุดงาน และตรวจ road access
Roads & incidents	ผู้วางแผน /Dispatcher	เตรียมกราฟ ตรวจรายงาน และอนุมัติการปิดถนน
Optimizer	ผู้วางแผน/นักวิจัย	กำหนด fleet ceiling, capacity, budget และ objective
Dispatch	ผู้วางแผน	ตรวจ route version ส่ง Driver App จำลอง และ export
Driver App	คนขับ	อ่านลำดับงาน เปิด navigation handoff และรายงานถนน
Evidence	ผู้ตรวจสอบ/นักวิจัย	ตรวจ invocation, logs, hashes, convergence และไฟล์ผล

3.2.7.1 สถานะเส้นทางที่สื่อสารตามข้อเท็จจริง

TRUTHFUL ROUTE STATES



รูปที่ 3-5 ตัวอย่างสถานะจริงตั้งแต่เตรียมโครงข่าย งาน Optimizer แผนรถที่ผ่านการตรวจ และ Driver App

ระบบใช้คำว่า “เตรียมโครงข่ายเสร็จ” เมื่อกราฟพร้อม ใช้คำว่า “แผนเบื้องต้นพร้อม” เมื่อมี deterministic feasible route และใช้คำว่า “แผนรถสมบูรณ์” เฉพาะเมื่อ C++ process รันสำเร็จและผ่าน independent validator หาก candidate ไม่ดีกว่าแผนเบื้องต้น ระบบยังถือว่ารอบ optimization เสร็จ แต่บันทึกชัดว่าคงแผนเดิมไว้

สำหรับงานที่วัด progress ได้ เช่น download และ checksum หน้าจอแสดง bytes, rate, elapsed time และ ETA จริง สำหรับ C++/OSMnx/osmium ซึ่งไม่มี denominator ที่เชื่อถือได้ หน้าจอแสดง indeterminate live state และ elapsed time แทนเปอร์เซ็นต์ที่คาดเดางานที่ล้มเหลวหรือถูกยกเลิกไม่แสดง 100%

3.2.7.2 กระบวนการรายงานถนนและคำนวณใหม่

ROAD-INCIDENT TRANSACTION

1. คนขับเปิดแบบรายงาน เลื่อนหรือซูมแผนที่ และเลือก road segment ที่เห็นจริง
2. รายงานเก็บ edge identity, direction, geometry evidence, ผู้รายงาน และหมายเหตุ
3. Dispatcher ตรวจรายงานก่อนอนุมัติ ไม่แก้กราฟทันทีจากข้อมูลที่ยังไม่ตรวจ
4. เมื่ออนุมัติ ระบบลบ directed edge ที่เลือกและ reverse edge ที่จับคู่ได้เมื่อปิดสองทิศทาง
5. active input และ route เดิมถูกทำเครื่องหมาย stale; evidence เดิมยังคงอยู่
6. ผู้วางแผนรัน optimizer ใหม่ และบันทึก route version ใหม่เมื่อ commit gate ผ่าน

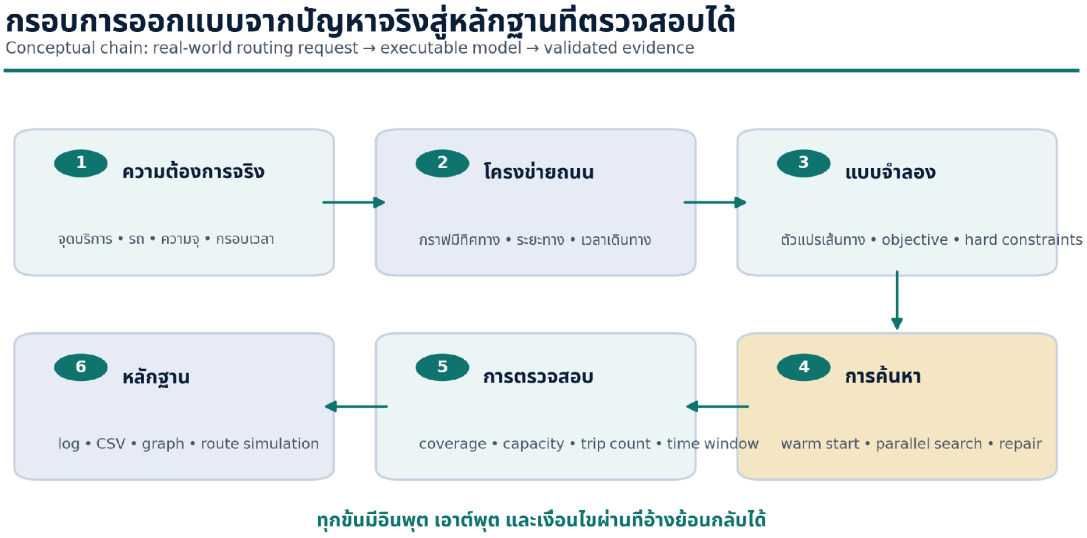
ขอบเขตของการอัปเดตเหตุการณ์

ระบบอัปเดตสถานะงานและตอบสนองต่อเหตุการณ์ถนนแบบ event-driven จากรายงานที่มีผู้อนุมัติ การเชื่อมตำแหน่ง GPS และข้อมูลจราจรสดอยู่ในแผนพัฒนาระยะถัดไป

3.2.8 จากปัญหาจริงสู่แบบจำลองที่ตรวจสอบได้

FROM OPERATIONAL REQUEST TO VERIFIABLE MODEL

ระบบไม่ส่งพิกัดจากหน้าจอเข้าสู่ตัวแก้โดยตรง แต่แปลงคำขอเชิงปฏิบัติการเป็นวัตถุที่ตรวจสอบได้ ได้แก่ depot, จุดบริการ, ข้อจำกัดรถ, road snapshot, terminal-cost matrix และ data contract เดียวกัน จากนั้นแกนค้นหาสร้าง candidate ซึ่ง independent validator จะคำนวณระยะ เวลา โหลด จำนวนรถ และเงื่อนไข hard constraints ซ้ำก่อนเปลี่ยนสถานะเป็นแผนที่เผยแพร่ได้ การแยกชั้นนี้ทำให้ความถูกต้องของแบบจำลองไม่ขึ้นกับข้อความบน UI และทำให้ทุก route version เชื่อมกลับไปยัง input และ invocation ที่ใช้จริง



รูปที่ 3-6 กรอบการออกแบบจากปัญหาจริงสู่หลักฐานที่ตรวจสอบย้อนกลับได้
ปรับจากกรอบอธิบายในรายงานต้นแบบและตรวจให้สอดคล้องกับ data contract ของแอปพลิเคชัน

ชั้น	สิ่งแทนในระบบ	หน้าที่
Operational layer	depot, รถ, ความจุ, จุดงาน, กรอบเวลา, เหตุถนน	ข้อมูลที่ผู้วางแผนและผู้ขับเข้าใจร่วมกัน
Road layer	directed graph, edge identity, distance, speed, geometry	รักษาทิศทางถนนและ revision ของข้อมูล

ชั้น	สิ่งแทนในระบบ	หน้าที่
Optimization layer	route sequence, vehicle assignment, objective, constraints	สร้างและปรับคำตอบภายใต้งบเวลา
Evidence layer	validator, hash, command, trace, route version	อนุญาตให้เผยแพร่และตรวจซ้ำ

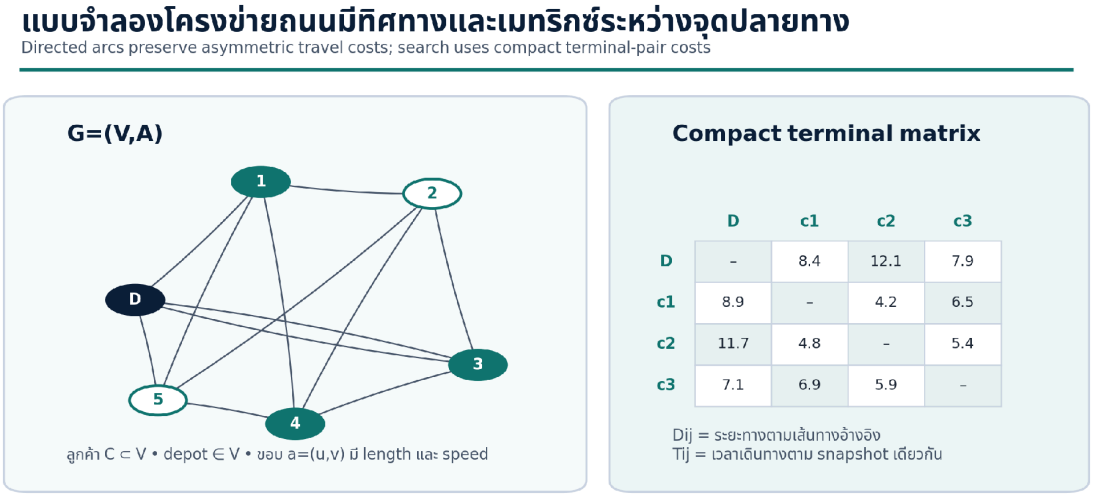
ตารางที่ 3-5 การเชื่อมความหมายจากงานปฏิบัติการถึงหลักฐาน

สรุปจากโครงสร้าง frontend, backend, engine และ validator ใน notebook หลักของโครงการ

3.2.9 โครงข่ายถนนแบบมีทิศทางและเมทริกซ์ระหว่างจุดปลายทาง

DIRECTED ROAD GRAPH AND TERMINAL-PAIR MATRICES

ให้ $T=\{0\}\cup C$ เป็นเซต terminal ซึ่งประกอบด้วย depot และจุดบริการ ระบบคำนวณ shortest path จากแต่ละต้นทางใน T บนกราฟ $G=(V,A)$ แล้วเก็บ D_{ij} และ T_{ij} เป็นเมทริกซ์ระยะทางกับเวลาเดินทางแบบหนาแน่น การค้นหาจึงประเมินการแทรกหรือสลับจุดได้ในเวลาคงที่จากค่า pair cost โดยไม่ต้องเดินกราฟถนนใหม่ทุกครั้ง ขณะที่การวาดผลบนแผนที่ยังย้อนจากคู่ terminal ไปยัง node path และ geometry จริง



รูปที่ 3-7 กราฟถนนแบบมีทิศทางและ compact terminal matrix ที่ใช้โดยตัวแก้
ตัวเลขในภาพเป็นตัวอย่างอธิบายโครงสร้าง ไม่ใช่ค่าจากชุด benchmark

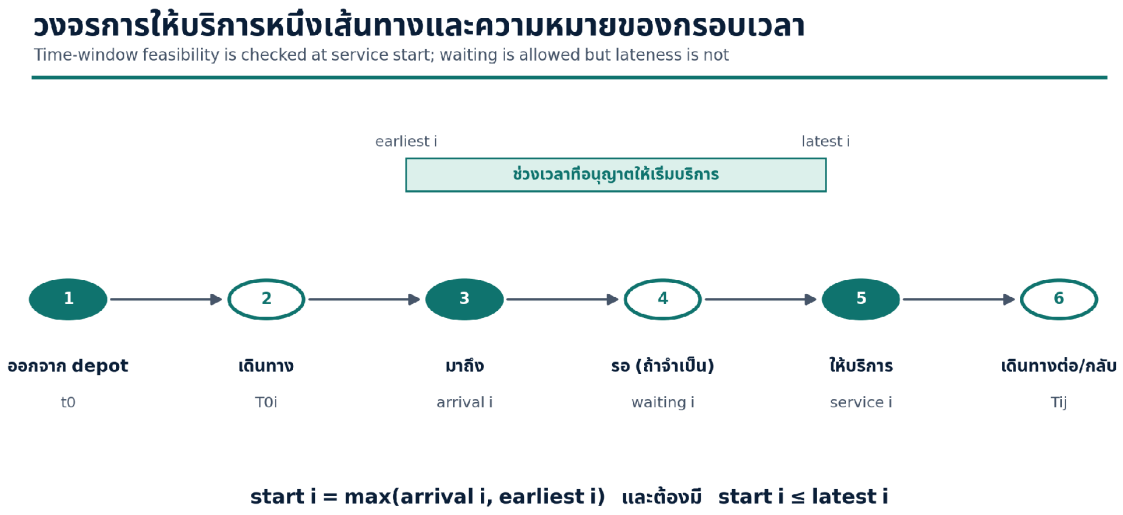
D_{ij} = dist_G(i,j), T_{ij} = time_G(i,j), $D_{ij} \neq D_{ji}$ โดยทั่วไป ต้นทุนไปและกลับคำนวณแยกกันตามทิศทางของกราฟ ไม่แทนด้วยระยะเส้นตรง

หาก |T|=n การเก็บ D และ T ใช้หน่วยความจำ $O(n^2)$ ส่วนการคำนวณ shortest path แบบหลายต้นทางมีต้นทุนโดยประมาณ O(n(|A|+|V|log|V|)) เมื่อใช้โครงสร้าง priority queue เหตุถนนที่ได้รับอนุมัติจึงเพิ่ม graph revision ทำให้ cache กับ matrix ของ revision เดิมไม่ถูกนำมาใช้กับแผนใหม่โดยไม่ตรวจสอบ

3.2.10 ความเป็นไปได้ด้านเวลา ความจุ และหลายเที่ยว

TIME, CAPACITY AND MULTI-TRIP FEASIBILITY

เมื่อรถมาถึงจุด i ก่อน earliest time ระบบอนุญาตให้รอและกำหนดเวลาเริ่มบริการเป็น $\max(\text{arrival}_i, a_i)$ แต่หากเวลาเริ่มบริการเกิน b_i คำตอบไม่ผ่าน hard feasibility เวลาบริการ s_i ถูกบวกก่อนเดินทางสู่จุดถัดไป สำหรับ multiple trips สัญลักษณ์ depot ภายใน route รีเซ็ตโหลดของเที่ยวถัดไป แต่เวลาของรถคันเดิมยังต่อเนื่อง จึงไม่สร้างเที่ยวหลายเที่ยวที่ซ้อนกันในวันเดียว



รูปที่ 3-8 วงจรการเดินทาง การรอ และการเริ่มบริการภายในกรอบเวลา

ภาพอธิบาย semantic ที่ independent validator ใช้ตรวจซ้ำ

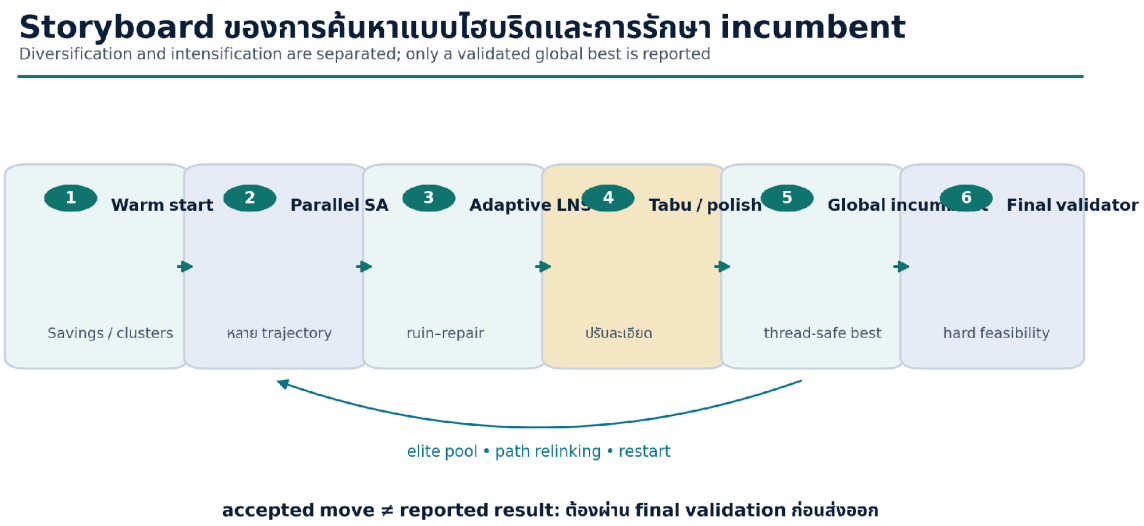
ข้อจำกัดแบบ hard ก่อน objective

coverage, capacity, time window และจำนวนรถถูกตรวจเป็นเงื่อนไขรับคำตอบ ไม่ใช่เพียงค่าปรับที่ยอมให้ละเมิดเพื่อแลกกับระยะทางสั้นกว่า ดังนั้นคำตอบที่ objective ต่ำแต่ไม่ feasible จะไม่ถูก commit เป็น route version ใหม่

3.2.11 ตรรกะของการค้นหาแบบไฮบริด

HYBRID-SEARCH LOGIC

แกนค้นหาแบ่งบทบาทเป็นการกระจายพื้นที่ค้นหา (diversification) และการปรับละเอียดรอบคำตอบที่ดี (intensification) คำตอบเริ่มต้นให้จุดอ้างอิงที่ feasible; หลาย trajectory สำรวจเส้นทางต่างกัน; ขั้นทำลาย–ซ่อมเปลี่ยนโครงสร้างกลุ่มลูกค้าในระดับที่ local move ทำไม่ได้; ขั้น tabu/polish ลดการวนกลับและเร่งปรับคำตอบ ส่วน global incumbent เป็นค่าดีที่สุดที่ผ่านเกณฑ์ภายในและถูกป้องกันการเขียนพร้อมกันระหว่าง worker



รูปที่ 3-9 Storyboard ของการค้นหาแบบไฮบริดและการรักษา incumbent

ชื่อเฟสใช้เพื่ออธิบายบทบาทเชิงแนวคิด; ลำดับและงบจริงมาจาก dynamic policy ของแอป

การกด Optimize ไม่เรียก configuration ชุดตายตัว แต่ใช้ dynamic_solver_policy() เลือก tier ที่เล็กที่สุดซึ่งครอบคลุมจำนวนลูกค้าปัจจุบันจากระดับ 10, 15, 20, 50, 100, 300 และ 450 จุด ส่วนจำนวนที่เกินช่วงใช้ tier หนักสุดเป็น fallback งบเวลาหลักยังเป็นค่าที่ผู้ใช้เลือก ขณะที่จำนวน process ขนานถูกจำกัดด้วยเพดานของ tier และ no-improvement patience ถูกผูกกับทั้งขนาดปัญหาและ runtime จากนั้น _solver_command() จึงสร้าง argv จริงโดยส่ง objective, time-window semantics, multi-trip semantics, metric=road และค่าที่ผ่าน validation ไปยัง C++ ทำให้ทุกการรันตรวจย้อนกลับได้ว่าระบบ resolve นโยบายใดและเรียก native solver ด้วยพารามิเตอร์ใด

โครงสร้าง	ข้อมูลที่เก็บ	เหตุผลที่ใช้
Directed adjacency	edge/node arrays และ reachability	เตรียมกราฟและ shortest path
Dense D/T matrices	contiguous pair-cost arrays	ประเมินลำดับ terminal จำนวนมากโดยไม่เดินกราฟซ้ำ
Route state	sequence, load, arrival/start, depot markers	คำนวณ delta และตรวจผลกระทบเฉพาะช่วง

โครงสร้าง	ข้อมูลที่เก็บ	เหตุผลที่ใช้
Candidate lists	neighbor ranks และรายการคู่ต้นทุนต่ำ	จำกัด move ที่ควรทดลองก่อน โดยไม่เปลี่ยนเกณฑ์ feasibility
Elite/incumbent	solution snapshots และ thread-safe best	รักษาคำตอบดี กระจายให้ worker และส่ง final validator
Evidence records	JSON/CSV/log/hash/version	แยกหลักฐานการรันออกจากสถานะบน UI

ตารางที่ 3-6 โครงสร้างข้อมูลหลักและบทบาทต่อประสิทธิภาพ
สรุปเชิงแนวคิดจาก implementation ของระบบ ไม่ใช่ผล ablation

การเร่ง hot path ต้องรักษาความหมายของ objective เดิม Candidate list หรือการจัดอันดับเพื่อนบ้านมีหน้าที่เปลี่ยนลำดับการทดลอง move ไม่ใช่ลดเงื่อนไข coverage, capacity หรือ time window เมื่อพบ incumbent ใหม่ ระบบคำนวณค่ารวมและตรวจความสอดคล้องก่อนเผยแพร่สู่ส่วนอื่น การออกแบบนี้แยก ‘คำตอบที่ถูกยอมรับระหว่างการค้นหา’ ออกจาก ‘ผลสุดท้ายที่รายงานได้’

3.2.12 การหยุดแบบ anytime และการตีความ convergence

ANYTIME STOPPING AND CONVERGENCE

ตัวแก้สามารถคืน incumbent ที่ดีที่สุดเมื่อครบงบเวลาหรือเมื่อไม่พบ meaningful improvement ต่อเนื่องตาม patience ที่กำหนด convergence.csv บันทึกเวลาและ objective ของเหตุการณ์ที่ดีขึ้นจริง จึงใช้ตรวจว่าคุณภาพเกิดขึ้นในเฟสใดและหยุดเพราะเหตุใด ใน benchmark ของบทที่ 5 การรันถูกตรึงให้อยู่บน Google Colab VM / Intel Xeon 2.4 GHz ตาม hardware protocol เดียวกัน จึงใช้ wall-clock time เปรียบเทียบภายใน protocol นี้ได้ อย่างไรก็ตาม runtime ที่ต่ำกว่างบยังไม่ควรถูกเรียกว่า speedup โดยอัตโนมัติ เพราะเมตาฮิวริสติกอาจหยุดก่อนกำหนดจาก no-improvement rule และอัลกอริทึมคู่เทียบมี stopping behavior ต่างกัน

3.2.13 แบบจำลองอ้างอิงและขอบเขตการตรวจแบบแม่นตรง

REFERENCE FORMULATION AND EXACT-VALIDATION BOUNDARY

แบบจำลองอ้างอิงใช้ตัวแปรทวิภาค x_{ijk} เพื่อแทนการใช้เส้นเชื่อมระหว่าง terminal โดยรถ k และตัวแปรเวลา/ลำดับเพื่อบังคับ flow, capacity, time windows และกำจัด subtour ฟังก์ชันวัตถุประสงค์ลดผลรวม $c_{ij}x_{ijk}$ แบบจำลองนี้อธิบายโครงสร้างในภาษาของ MILP ส่วนการให้ใบรับรองผลในชุดเล็กใช้ OR-Tools CP-SAT ซึ่งเป็นตัวแก้เชิงข้อจำกัดจำนวนเต็ม ไม่เรียกว่า MILP solver เพื่อหลีกเลี่ยงการสื่อชื่อเครื่องมือคลาดเคลื่อน

$$\text{gap}(\%) = 100 \times (z_\text{project} - z^*) / z^*$$

คำนวณเฉพาะเมื่อ exact solver รายงาน OPTIMAL และ objective เท่ากับ best bound; ชุดถนนจริง ที่ไม่มี z^* ไม่ใช้สมการนี้

การตรวจ exact ขนาดเล็กตอบคำถามว่ากลไกสร้างคำตอบใกล้ optimum และรักษาข้อจำกัดได้หรือไม่ แต่ไม่ยืนยัน global optimum บนชุดถนนจริงขนาด 50–450 จุด ส่วนการเทียบ OR-Tools best-of-12 ในบทที่ 5 เป็น controlled empirical baseline ไม่ใช่ lower bound หรือ BKS จึงรายงานสองชั้นหลักฐานแยกกัน

3.2.14 สัญญาหลักฐานและความซื่อสัตย์ของสถานะ

EVIDENCE CONTRACT AND TRUTHFUL STATE

หนึ่งรอบคำนวณผูก source revision, input SHA-256, dynamic policy, argv, seed/repetition, stdout, stderr, convergence, native summary, validator output และ route version เข้าด้วยกันเป็น evidence chain เดียว หาก process ล้มเหลว parser อ่านผลไม่ได้ หรือ independent validator ปฏิเสธ candidate ระบบยังเก็บหลักฐานของ attempt นั้นไว้แต่ไม่เขียนทับ previous valid plan ดังนั้นสถานะ “จัดเส้นทางเสร็จ” จึงหมายถึง native solver สิ้นสุดตามสัญญาและ candidate ผ่านการตรวจจริง ไม่ใช่เพียงเตรียมกราฟหรือวาดเส้นทางได้

3.3 เครื่องมือที่ใช้ในการพัฒนา

TOOLS AND TECHNOLOGIES

กลุ่ม	เทคโนโลยี	บทบาท
Optimization	C++17, g++, multi-threading	ตัวแก้หลักและ clustering binary
Backend	Python, FastAPI, Uvicorn, SQLite	API, jobs, state, persistence, transactional commit
Road data	OpenStreetMap, OSMnx 2.1.0, NetworkX, Shapely, pyproj	กราฟถนน การคำนวณ shortest paths และ geometry
Frontend	HTML, CSS, JavaScript, Leaflet, Chart.js	map-first workspace, Driver App, charts
Analysis	NumPy, pandas, SciPy, scikit-learn, Matplotlib	วิเคราะห์ข้อมูล clustering สถิติ และเตรียมกราฟสำหรับ benchmark
Validation	Google OR-Tools / CP-SAT	exact small-case verification และ external baseline comparison
Delivery	Google Colab, two-cell launcher	ติดตั้งและเปิดแอปจาก notebook

ข้อมูลถนนอยู่ภายใต้ OpenStreetMap contributors [16] ส่วนการวิเคราะห์ตารางและภาพใช้ NumPy, pandas และ Matplotlib [12]–[14] การออกแบบ benchmark ระยะถัดไปอ้างอิงรูปแบบและระบบนิเวศของ PyVRP/CVRPLIB [17], [18] ชั้นแอปพลิเคชันใช้ FastAPI, SQLite และ Leaflet ตามเอกสารของโครงการเหล่านั้น [22]–[24] แต่ logic การจัดเส้นทาง สัญญาข้อมูล ตัวตรวจรับ workflow และการประกอบระบบเป็นส่วนที่พัฒนาขึ้นในโครงการ

3.4 รายละเอียดโปรแกรมที่พัฒนาในเชิงเทคนิค

SOFTWARE SPECIFICATION

3.4.1 ข้อมูลนำเข้า ผลลัพธ์ และสัญญาข้อมูล

INPUT, OUTPUT AND DATA CONTRACT

input.txt เป็น solver-facing contract ที่ C++ อ่านโดยตรง ขณะที่ dataset.json เก็บความหมายเต็มของกราฟ จุดงาน mapping จาก OSM และ routing contract ส่วน dataset_summary.json เก็บ identity, counts, feasibility summary และ checksums ทั้งสามไฟล์ถูกสร้างจาก snapshot ในหน่วยความจำชุดเดียว ก่อนเผยแพร่ backend จะ parse สัญญาที่ serialize แล้วกลับมาตรวจค่าขอบถนนและจุดงานกับ dataset.json และจึงแทนที่ไฟล์ runtime แบบ atomic วิธีนี้ลดความเสี่ยงของไฟล์ครึ่งชุดและทำให้ hash ของ stdin เชื่อมกับคำสั่ง solver ได้โดยตรง

```
input.txt · solver-facing contract
N M
K # maximum available vehicles
depot_node
u v length_m speed_kph # repeated M lines
customer_count
node demand service earliest latest # repeated customer_count lines
capacity
```

ความหมายของ K ถูกตรึงเป็น maximum available vehicles ระบบอนุญาตให้ใช้รถจริง m ใด ๆ ที่ $1 \le m \le K$ และไม่สร้างรถว่างเพื่อให้ครบ K เมื่อเปิด multiple trips การกลับ depot รีเซ็ตโหลดของเที่ยวถัดไป แต่เวลาของรถคันเดิมยังต่อเนื่องตาม route_start_time และ service/travel/waiting time
ในระดับกราฟ ระบบรักษา parallel OSM edges ไว้ เพราะ edge ที่สั้นที่สุดระหว่างคู่โหนดอาจไม่ใช่ edge ที่เร็วที่สุด การ collapse ทุก (u,v) ให้เหลือเพียง edge เดียวจึงอาจทำให้ objective แบบ distance หรือ time ไม่สามารถเลือก road path ที่ถูกต้องได้ ข้อมูล length และ speed

ถูกปัดค่าก่อนคำนวณ travel time เพื่อให้ JSON กับ stdin แทน snapshot เดียวกัน

ตารางที่ 3-8 ไฟล์หลักและหลักฐาน

ไฟล์/วัตถุ	หน้าที่	หลักฐานสำคัญ
input.txt	stdin ของ C++	node/edge/customer/K/Q ตรง dataset
dataset.json	ข้อมูลเต็มและ mapping	vehicle_count_semantics, geometry, windows
dataset_summary.json	สรุป identity	SHA-256, counts, contract checks
solver_invocation.json	ผูก process กับ input	argv, cwd, stdin path/size/hash, output paths
solver_output.txt	stdout ที่ใช้ parse route	final objective, routes, fleet totals
solver_stderr.txt	stderr/cerr แยกช่อง	heartbeat, warning, exit/timeout/cancel
convergence.csv	พฤติกรรม search	worker stream, current/best/global best, timestamps
routes.json / route version	แผนที่ส่งต่อ	solution source, objective, contract revision

3.4.2 ข้อกำหนดเชิงหน้าที่ของระบบ

FUNCTIONAL SPECIFICATION

ตารางที่ 3-9 ข้อกำหนดเชิงหน้าที่ของระบบ

รหัส	ความสามารถ	เงื่อนไขสำคัญ
F-01	กำหนด depot จุดงาน กองรถ และข้อจำกัด	ตรวจ identity, หน่วย และ data revision
F-02	สร้างกราฟถนนแบบมีทิศทางและ terminal costs	ทุกจุดต้องไป–กลับ depot ได้

รหัส	ความสามารถ	เงื่อนไขสำคัญ
F-03	สร้างแผนเบื้องต้นและรัน C++ Optimizer	แยก prepared plan จาก optimized route
F-04	ตรวจคำตอบและบันทึก route version	commit เมื่อ parser และ independent validator ผ่าน
F-05	ส่งใบงานให้คนขับและ export หลักฐาน	แสดงเฉพาะรถที่ใช้จริงและผูก route version
F-06	รับ–อนุมัติเหตุถนนและคำนวณใหม่	แก้ exact directed edge; เก็บ previous valid plan

3.4.3 สถาปัตยกรรมซอฟต์แวร์และขอบเขตความรับผิดชอบ

SOFTWARE ARCHITECTURE AND RESPONSIBILITY

สถาปัตยกรรมแบ่งเป็น frontend, FastAPI backend, SQLite state, road-graph runtime, C++ optimizer และ validation/evidence pipeline โดยมี data contract และ persisted workflow state เป็นจุดเชื่อม ไม่ใช่การเรียกไฟล์ต่อไฟล์แบบหลวม ๆ Road runtime เก็บกราฟที่เตรียมแล้ว โครงสร้างค้นหาเชิงพื้นที่ และ cache ที่สัมพันธ์กับ graph revision; backend ทำหน้าที่ materialize อินพุต จัดคิวงาน คุม process และจัดการ state transition; C++ รับเฉพาะสัญญาที่ freeze แล้วและส่งผลผ่าน stdout/convergence; validator คำนวณ coverage, capacity, timing, fleet totals และ route geometry ซ้ำก่อนอนุญาตให้เผยแพร่

3.4.4 โครงสร้างโมดูลและที่มาของส่วนประกอบ

MODULE BOUNDARIES AND PROVENANCE

โมดูล	หน้าที่	ที่มา/ความรับผิดชอบ
Frontend / planning workspace	หน้าจอ map-first, route truth, driver manifest, incident flow	พัฒนาตรรกะหน้าจอและ workflow ในโครงการ; ใช้ Leaflet เป็นไลบรารีแผนที่
FastAPI backend	API, state transition, job lifecycle, atomic commit	พัฒนา API, state transition, job lifecycle และ commit logic ในโครงการ; ใช้ FastAPI/Uvicorn เป็น framework/runtime
Road runtime	OSM ingestion, directed edge identity, matrix, revision	พัฒนา integration, edge identity, revision และ terminal-cost workflow ในโครงการ; ใช้ข้อมูล OSM และไลบรารี OSMnx/NetworkX
C++ optimizer	ThroughputLNS, parallel deep search, incumbent, refinement	ตรรกะค้นหาและ orchestration อยู่ใน solver.cpp ของโครงการ
Validation/evidence	parser, independent checks, hash, log, export	พัฒนา parser, independent checks, transaction receipts, hash/log/export pipeline ในโครงการ
External baselines	OR-Tools routing และ CP-SAT exact checks	เครื่องมืออ้างอิงภายนอก ไม่อ้างเป็นส่วนสมทบของโครงการ

ตารางที่ 3-10 ขอบเขตส่วนที่พัฒนาขึ้นเองและซอฟต์แวร์ภายนอก

รายละเอียดใบอนุญาตและเอกสารอ้างอิงระบุในบทที่ 9 และภาคผนวก

3.4.5 สัญญาการรันและหลักฐานที่ตรวจสอบย้อนกลับได้

EXECUTION AND EVIDENCE CONTRACT

ช่วง	ไฟล์/วัตถุ	เงื่อนไขผ่าน
Materialize	input.txt, dataset.json, dataset_summary.json	ข้อมูล โครงสร้าง และ checksum สอดคล้องกัน
Invoke	solver_invocation.json และ dynamic_policy.json	ทราบ binary, argv, cwd, seed, threads และงบจริง
Observe	stdout, stderr, convergence.csv	ตรวจ phase, exit state และลำดับการปรับปรุง
Validate	routes/summary และ independent validator	coverage, load, time, fleet และ totals ผ่าน
Commit	route version, graph revision, previous-plan link	เผยแพร่แบบ atomic และย้อนประวัติได้

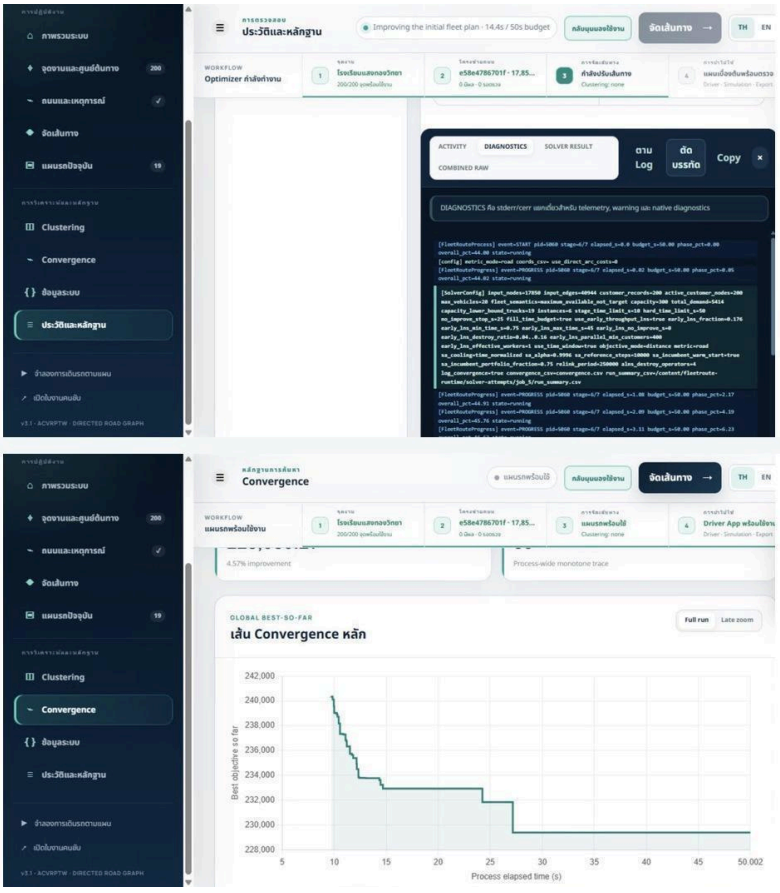
ตารางที่ 3-11 สัญญาหลักฐานตั้งแต่สร้างอินพุตถึงยืนยันแผน

ชื่อไฟล์บางรายการมี suffix ตาม dataset และ repetition ในชุด benchmark

หนึ่ง solve attempt ผูก source signature, materialization identity, stdin SHA-256, argv, seed, policy, stdout, stderr/cerr, convergence, run summary และ candidate validation ไว้ใน evidence chain เดียว Solver stdout เป็น stream เดียวที่ parser ใช้อ่าน route ส่วน stderr/cerr เก็บแยกเพื่อไม่ให้ข้อความ diagnostic ปะปนกับผลลัพธ์

การ commit ใช้เงื่อนไขพร้อมกันสี่ด้าน: native process ต้องจบปกติ, candidate ต้องผ่าน independent validation, evidence bundle ต้องครบ และ source contract ต้องยังตรงกับสถานะปัจจุบัน หาก candidate ผ่านทุกข้อแต่ objective ไม่ดีกว่า reference plan ระบบบันทึกผลการรันไว้แต่คงแผนเดิม หาก process ล้มเหลว ถูกยกเลิก หรือ input เปลี่ยนระหว่างรัน ระบบไม่ให้ attempt นั้นเขียนทับแผนที่ผ่านการตรวจล่าสุด

Atomic route commit
การบันทึก candidate และการเปลี่ยน current route เป็น transaction เดียวกัน หาก validation หรือการเขียนข้อมูลส่วนใดล้มเหลว current route ยังคงชี้ไปยังแผนที่ผ่านการตรวจล่าสุด ขณะที่รอบที่ล้มเหลวยังคงมี log สำหรับวิเคราะห์



รูปที่ 3-10 Runtime/Evidence Inspector และ Convergence จากการรันจริง แสดง diagnostics ของ native process และ global best-so-far บน process clock เดียว

หน้าจอหลักฐานไม่ได้สร้างค่าความก้าวหน้าแบบสมมติ: diagnostics อ่านจาก stderr/cerr และ structured process events ส่วนกราฟ Convergence สร้างจาก incumbent ที่ดีขึ้นจริงบน process clock เดียวกัน ทำให้ผู้ตรวจสามารถย้อนจากเส้นกราฟไปยัง attempt, command และไฟล์ผลของรอบเดียวกันได้.

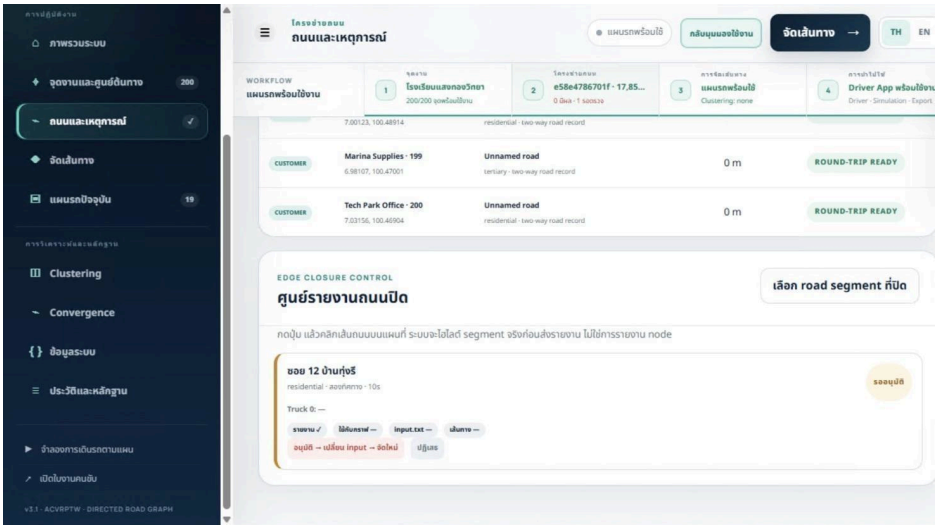
3.4.6 ข้อกำหนดเหตุถนนและการคำนวณเส้นทางใหม่

ROAD-EVENT AND REROUTING CONTRACT

เหตุการณ์	การเปลี่ยนแปลง	ผลที่อนุญาต
Report	เก็บ directed edge, geometry, direction, note และผู้รายงาน	ยังไม่แก้ active graph
Approve	ผู้ควบคุมยืนยันผลกระทบและทิศทาง	สร้าง graph revision ใหม่
Invalidate	matrix/input/route เดิมถูกระบุ stale	ยังอ่าน previous valid plan ได้
Recompute	สร้าง matrix และ input จาก revision ใหม่	รัน solver ด้วย policy ที่บันทึกได้
Commit	validator ผ่านและ totals ตรง	เผยแพร่ route version ใหม่; ถ้าไม่ผ่านให้คงแผนเดิม

ตารางที่ 3-12 สถานะของ road incident และ route version

การปิดถนนอ้าง exact directed edge ด้วย OSM u/v/key และ geometry evidence ไม่ได้ลบถนนทุกเส้นที่มีชื่อเดียวกัน หากผู้ควบคุมยืนยันว่าปิดสองทิศทาง ระบบจะลบ reverse edge ได้ไม่เกินหนึ่งเส้นที่จับคู่ geometry ได้อย่างชัดเจน การอนุมัติหรือเปิดถนนคืนเพิ่ม graph revision ทำให้ matrix/input/route ที่พึ่งพากราฟเดิมถูกระบุ stale และต้องคำนวณใหม่ก่อนประกาศเป็นแผนปัจจุบัน



รูปที่ 3-11 ศูนย์ควบคุมเหตุถนนจากระบบจริง: รายงาน exact road segment จะรอการอนุมัติก่อนเพิ่ม graph revision และคำนวณเส้นทางใหม่

3.4.7 นโยบายแบบไดนามิกและการส่งต่อสู่ Native Solver

DYNAMIC POLICY AND NATIVE SOLVER HANDOFF

ผู้ใช้กำหนด objective, runtime, fleet ceiling และ capacity ในระดับงาน ส่วนรายละเอียดการค้นหาถูก resolve เป็น execution policy ตามจำนวนลูกค้า การแยกสองระดับนี้ช่วยให้หน้าจอไม่บังคับให้ผู้ใช้รู้พารามิเตอร์ภายในทั้งหมด แต่ยังทำให้การทดลอง reproducible เพราะ policy ที่ resolve แล้วถูกบันทึกพร้อมคำสั่งจริง นโยบายสามารถเปลี่ยน warm-start, Early-LNS budget, patience และจำนวน worker ตาม scale; ตัวอย่างสำคัญคือ tier 100 จุดปิด Savings warm-start แต่ยังเปิด Early ThroughputLNS เพื่อหลีกเลี่ยงการยึด route partition จากการ merge ระยะแรกโดยอัตโนมัติ.

องค์ประกอบ	กฎที่ใช้	เหตุผลเชิงระบบ
Customer tier	เลือก tier เล็กที่สุดที่ครอบคลุมจำนวนลูกค้า; มากกว่า 450 ใช้ระดับหนักสุด	หลีกเลี่ยง configuration เดียวสำหรับทุก scale
Runtime	ใช้ค่าที่ผู้ใช้เลือกภายในขอบเขตที่ backend ยอมรับ	งบหลักยังเป็น decision ของผู้ใช้

องค์ประกอบ	กฎที่ใช้	เหตุผลเชิงระบบ
Parallel instances	effective = min(requested, tier cap)	ควบคุม oversubscription และรักษานโยบายที่ตรวจได้
No-improvement stop	min(tier patience, max(5 s, ~50% runtime))	ไม่ให้ patience ยาวกว่างบที่มีนัยสำคัญ
Early LNS	งบเป็นสัดส่วนที่ถูก cap; ใช้หลาย worker เฉพาะเมื่อถึง threshold	เร่งช่วงต้นโดยไม่แทน deep-search pipeline
Advanced settings	typed validation เทียบกับ flag ที่ native parser รองรับ	ป้องกัน flag ที่ไม่รู้จักและ argv ซ้ำซ้อน

ตารางที่ 3-13 กฎหลักของ execution policy และ native handoff

ค่าดังกล่าวเป็นนโยบายการทำงานของซอฟต์แวร์และจุดตั้งต้นจากการทดลอง ไม่ใช่ค่าที่พิสูจน์แล้วว่าเหมาะที่สุดสำหรับทุกอินสแตนซ์ การสรุปผลจึงรายงาน policy และ runtime คู่กับ objective เสมอ

3.5 ขอบเขตและข้อจำกัดของโปรแกรม

SCOPE AND LIMITATIONS

ตารางที่ 3-14 ขอบเขตการรองรับ

ขอบเขตที่รองรับ	ขอบเขตสำหรับการพัฒนาต่อ
ศูนย์เดียว รถหลายคัน ความจุเดียวกัน	heterogeneous fleet และ vehicle-specific skill/cost
directed road snapshot และเหตุปิดถนนที่อนุมัติ	live traffic และ dynamic travel times
capacity, service, optional time windows, multiple trips	multi-depot, pickup-delivery precedence, split delivery
daily planning, driver manifest, planned-route simulation	live GPS dispatch และการควบคุมรถอัตโนมัติ

ขอบเขตที่รองรับ	ขอบเขตสำหรับการพัฒนาต่อ
distance/time/hybrid objectives	emissions, tolls, tanker compartments และกฎเฉพาะรถทุกชนิด

สำหรับงานส่งน้ำ น้ำแข็ง หรือของเหลวชนิดเดียวจากโรงงานหนึ่งแห่ง โครงสร้าง capacity ต่อเที่ยวสามารถแทนปริมาณส่งได้เมื่อข้อมูล demand และข้อจำกัดสอดคล้องกับแบบจำลอง งานขนส่งของเหลวที่มีเงื่อนไขเฉพาะต้องขยาย constraint audit ให้ครอบคลุมช่องถัง การปนเปื้อน ลำดับการโหลด น้ำหนักเพลา และข้อจำกัดถนนรายรถก่อนเริ่มใช้งาน

3.6 คุณลักษณะอุปกรณ์และสภาพแวดล้อม

HARDWARE AND RUNTIME ENVIRONMENT

สภาพแวดล้อมใช้งานถูกออกแบบให้เริ่มจาก Google Colab แบบสองเซลล์: เซลล์แรกติดตั้ง dependency และคอมไพล์ solver.cpp/cluster.cpp จาก source bundle ใน notebook ส่วนเซลล์ที่สองเปิด FastAPI application และเผยแพร่ลิงก์ HTTPS สำหรับใช้งานหน้าเว็บ การแยกขั้นติดตั้งออกจากขั้นเปิดระบบช่วยให้การสาธิตทำซ้ำได้ง่าย และทำให้การวัดแกนคำนวณสามารถแยกจากเวลาโหลดหน้า UI หรือเวลาติดตั้ง dependency

ฝั่งผู้ใช้รองรับเบราว์เซอร์สมัยใหม่บน desktop, tablet และ mobile ส่วนฝั่งคำนวณต้องมี Python 3, compiler ที่รองรับ C++17 และหน่วยความจำเพียงพอสำหรับ directed graph กับ terminal matrix โหมด offline-road ใช้ Thailand PBF และ regional graph cache เพื่อลดการดาวน์โหลดซ้ำและทำให้ข้อมูลถนนที่ใช้สร้างอินสแตนซ์มีที่มาชัดเจน การทดลองในบทที่ 5 ควบคุมสภาพแวดล้อมเป็น Google Colab VM / Intel Xeon 2.4 GHz ภายใต้ protocol เดียวกันทั้งตัวแก้เส้นทางของโครงการและ OR-Tools จึงใช้ wall-clock time เปรียบเทียบภายในงานนี้ได้โดยตรง อย่างไรก็ตาม ข้อสรุปด้านเวลาไม่ถูกขยายเป็น universal

speedup นอกสภาพแวดล้อมดังกล่าวหรือข้าม stopping condition ที่ต่างกัน

4 กลุ่มผู้ใช้โปรแกรม
PROGRAM USERS

4.1 ผู้ใช้เป้าหมาย
TARGET USERS

ระบบถูกออกแบบเป็น decision-support system แบบ human-in-the-loop ผู้ใช้แต่ละกลุ่มเห็นข้อมูลตามการตัดสินใจที่ตนรับผิดชอบ และทุกหน้าจออ้างอิง route version เดียวกันเพื่อลดความคลาดเคลื่อนระหว่างศูนย์ควบคุม ผู้ตรวจสอบ และผู้ปฏิบัติงานภาคสนาม

ตารางที่ 4-1 Persona และข้อมูลที่เหมาะกับงาน

บทบาท	เป้าหมายหลัก	ข้อมูลที่ต้องเห็น	สิ่งที่ไม่ควรถูกบังคับให้รู้
ผู้วางแผน/ผู้ดูแล	สร้างและปรับแผนรถ	จุดงาน กองรถ เหตุถนน สถานะ optimizer และแผนฉบับที่ยืนยันแล้ว	คำสั่งคอมไพล์และโครงสร้างไฟล์ภายใน
Dispatcher	รับ–ตรวจเหตุการณ์และส่งแผน	route version, incident geometry, stale state และสถานะรถ	รายละเอียด neighborhood ของเมตาฮิวริสติก
คนขับ	ปฏิบัติตามใบงานและรายงานอุปสรรค	ลำดับจุด ระยะ/เวลาโดยประมาณ navigation handoff และปุ่มรายงานถนน	กราฟ convergence และ exact bound
ผู้ตรวจสอบ/นักวิจัย	ตรวจผลและทำซ้ำ	data identity, command, hash, logs, convergence, validator และ paired protocol	การอนุมานจากภาพหน้าจอเพียงอย่างเดียว

4.2 เส้นทางการใช้งานหลัก

PRIMARY USER JOURNEY

1. ผู้วางแผนกำหนดศูนย์ต้นทางและจุดงานบนแผนที่ หรือนำเข้าชุดข้อมูลที่มีสัญญาครบ
2. ระบบเตรียมกราฟถนน ตรวจการเข้าถึง และแสดงว่า “โครงข่ายพร้อม” โดยยังไม่เรียกขั้นนี้ว่าจัดเส้นทางเสร็จ
3. ระบบสร้างแผนเบื้องต้นที่ feasible และวาดเป็นเส้นประ เพื่อช่วยตรวจข้อมูลก่อนเริ่มการค้นหา
4. ผู้วางแผนกำหนดจำนวนรถสูงสุด ความจุ งบเวลา และ objective แล้วเริ่ม optimizer
5. เมื่อ C++ process จบ ระบบตรวจผลอย่างอิสระและ commit route version ใหม่แบบ atomic เฉพาะเมื่อผ่าน
6. Dispatcher ตรวจแผน ส่งใบงานตามรถ และคนขับเปิดลิงก์แผนที่ภายนอกได้ตามต้องการ
7. เมื่อพบถนนปิด คนขับเลือก segment และส่งรายงาน ผู้ดูแลตรวจแล้วอนุมัติก่อนแก้ directed graph
8. แผนเดิมถูกระบุว่า stale และยังคงอ่านได้ จากนั้นผู้วางแผนสั่งคำนวณใหม่เพื่อสร้าง route version ถัดไป

> **การรักษาแผนที่ตรวจผ่านแล้ว**
> ระหว่างการคำนวณใหม่ หรือเมื่อรอบใหม่ล้มเหลว ระบบเก็บ previous valid plan พร้อมป้ายสถานะเพื่อให้ผู้ใช้มีข้อมูลอ้างอิง และป้องกันไม่ให้ผลที่ไม่สมบูรณ์เขียนทับแผนปัจจุบัน

4.3 องค์ประกอบการใช้งานที่พัฒนาขึ้น

OPERATIONAL CAPABILITIES

องค์ประกอบการใช้งานถูกกำหนดเป็น requirement ที่ตรวจสอบได้ โดยเชื่อมแต่ละฟังก์ชันกับ observable state, API contract และ evidence ที่เกิดขึ้นจริง ตารางต่อไปจึงตอบสามคำถามพร้อมกันว่า ผู้ใช้เห็นอะไร ระบบยืนยันอะไรได้จากข้อมูลที่บันทึก และประเด็นใดต้อง

ประเมินต่อในภาคสนาม วิธีนี้ทำให้คุณภาพของแอปถูกอธิบายจากพฤติกรรมและสัญญาของระบบ ไม่ใช่จากจำนวนไฟล์หรือจำนวนหน้าจอ

ตารางที่ 4-2 องค์ประกอบการใช้งาน หลักฐาน และแผนประเมิน

องค์ประกอบ	การออกแบบ	หลักฐานปัจจุบัน	การประเมินระยะถัดไป
การส่งต่อแผนสู่ผู้ขับ	Driver App, dispatch handoff, ใบงานรายรถและรายงาน road segment	driver manifest, route-version handoff และ report payload	ทดลองใช้กับผู้ปฏิบัติงานจริง
ความชัดเจนของหน้าจอ	UI แบบ map-first แยกโหมดใช้งาน/วิเคราะห์ และใช้สถานะตามภาษางาน	persisted route-state semantics และ responsive views	SUS, completion time และ error rate
เหตุการณ์ถนนและการจัดใหม่	incident inbox, approval, directed-edge removal, stale signaling และ rerun	approved edge event, graph revision, stale state และ transaction receipt	ข้อมูลจราจรสดและตำแหน่งรถ
การตรวจสอบย้อนกลับ	แกน C++ แยกจาก UI พร้อม data/evidence contract	source inspection, invocation manifest, independent validation และ route-version evidence	controlled benchmark และ field trials

4.4 หลักการ UX ที่ใช้

UX PRINCIPLES

ตารางที่ 4-3 หลักการออกแบบประสบการณ์ใช้งาน

หลักการ	การใช้ในระบบ
Progressive disclosure	ซ่อน tuning, logs และ diagnostics ไว้ในพื้นที่ขั้นสูง แต่ไม่ซ่อนหลักฐานจากผู้ตรวจ
State before action	แสดงว่าอะไรพร้อม อะไรล้าสมัย และ action ถัดไปก่อนให้ผู้ใช้กดรัน
Map as workspace	ใช้แผนที่สว่างเป็นพื้นที่เลือกและตรวจ ไม่ใช้พื้นหลังดำที่อาจถูกมองว่าโหลดล้มเหลว
One meaning per term	แยก “โครงข่ายพร้อม”, “แผนเบื้องต้น” และ “แผนรถสมบูรณ์” ออกจากกัน
Recoverable operations	การรันใหม่ไม่ทำลาย previous valid plan และการปิดถนนต้องผ่าน approval
Technical language where useful	คงคำเฉพาะ เช่น clustering, convergence และ ACVRPTW พร้อมคำอธิบายตามบริบท

4.5 การกำกับสิทธิ์และความรับผิดชอบ

HUMAN OVERSIGHT

ระบบแบ่งอำนาจและความรับผิดชอบอย่างชัดเจน: ผู้ขับมีหน้าที่รายงาน ผู้ดูแลเป็นผู้อนุมัติเหตุการณ์และสั่งคำนวณใหม่ ส่วนระบบเผยแพร่แผนเฉพาะ candidate ที่ validator รับรอง กลไกนี้ป้องกันไม่ให้ข้อมูลภาคสนามที่ยังไม่ผ่านการตรวจแก้ directed graph หรือเปลี่ยนแผนรถทั้งกองโดยทันที และยังคง audit trail ของการตัดสินใจไว้

ระดับความพร้อมของระบบ ผ่าน contract/state verification และ controlled benchmark ในขอบเขตระบบ พร้อมเข้าสู่หลาย-seed ablation และ controlled pilot; enterprise identity, live telemetry และ backup policy อยู่ในระยะถัดไป

5 การทดสอบและประเมินผล
VERIFIED RESULTS AND COMPARATIVE EVALUATION

คำเรียกที่ใช้ตลอดบท
• ตัวแก้เส้นทางรุ่นรอบภูมิภาค = backend ที่ใช้ในการนำเสนอรอบภูมิภาคของโครงการ
• ตัวแก้เส้นทางรุ่นปัจจุบัน = backend ที่อยู่ในระบบฉบับส่งรอบนี้และเป็นผล benchmark หลัก 3 รันต่อขนาด
• การทดลองวิเคราะห์ 100 จุด = การตั้งค่าพิเศษเพื่อหาสาเหตุของ regression; ไม่ใช่ผล benchmark หลัก
• Google OR-Tools = baseline ภายนอก 12 configurations; ตัดสินข้าม solver หลังประเมิน route ด้วย metric เดียวกัน

12 / 12 รันของรุ่นปัจจุบันผ่าน feasibility	**3 ชนะ · 1 เสมอ รุ่นรอบภูมิภาค vs Google OR-Tools**	**2 ชนะ · 1 เสมอ · 1 แพ้ รุ่นปัจจุบัน vs OR-Tools**	**รุ่นรอบภูมิภาค vs OR-Tools และ รุ่นปัจจุบัน ชนะ 5/5 ในการทดลองวิเคราะห์ 100 จุด**

ลำดับการอ่านผล
1) เริ่มจาก backend ที่ใช้ในรอบภูมิภาค: เสมอ OR-Tools ที่ 50 จุด และชนะที่ 100/300/450 ตามโปรโตคอลเดิม
2) ดูว่ารุ่นปัจจุบันเปลี่ยนจากจุดตั้งต้นนั้นอย่างไร: ดีขึ้นที่ 50/300/450 แต่ถดถอยที่ 100
3) ใช้ผล benchmark หลักของรุ่นปัจจุบันเทียบ OR-Tools ภายใต้ metric เดียวกัน
4) แยกการทดลอง 100 จุดออกเป็น diagnostic เพื่อหาสาเหตุ ไม่ใช้แทนคะแนนหลัก

5.1 คำถามการประเมินและหลักการอ่านผล

EVALUATION QUESTIONS AND INTERPRETATION

การทดลองถูกออกแบบให้ตอบความถูกต้อง ความสม่ำเสมอของการค้นหา ความสามารถในการขยายขนาด และความสามารถในการแข่งขัน โดยแยกคำถาม “คำตอบ feasible หรือไม่” ออกจาก “คำตอบสั้นเพียงใด” และ “ใช้เวลาเท่าใดหรือหยุดเพราะเหตุใด” งาน benchmark ของโครงการและ Google OR-Tools ใช้ Google Colab VM ภายใต้ protocol ฮาร์ดแวร์เดียวกัน โดยใช้ CPU Intel Xeon 2.4 GHz จึงสามารถอ่าน wall-clock time เป็นตัวชี้วัดภายในสภาพแวดล้อมการทดลองนี้ได้ อย่างไรก็ตาม เมตาฮิวริสติกอาจหยุดก่อน budget สูงสุดจาก no-improvement rule และ cloud scheduling ยังสร้างความผันผวนระดับเล็กได้ จึงไม่ขยายผลเวลาเป็น hardware-independent speedup โดยอัตโนมัติ

1. RQ1 — ความถูกต้องของคำตอบ: ผลลัพธ์ให้บริการลูกค้าครบหนึ่งครั้ง ไม่เกินความจุ ไม่ละเมิดกรอบเวลา และผ่าน independent feasibility gate หรือไม่
2. RQ2 — ความสม่ำเสมอและการขยายขนาด: เมื่อรันซ้ำบนอินพุตเดียวกัน คุณภาพคำตอบและเวลาที่สังเกตเปลี่ยนแปลงอย่างไรเมื่อเพิ่มจาก 50 เป็น 450 จุด
3. RQ3 — ความสามารถในการแข่งขัน: เมื่อถอดลำดับจุดของตัวแก้เส้นทางรุ่นปัจจุบันและ Google OR-Tools แล้วประเมินซ้ำด้วยนิยามระยะทางบนกราฟถนนแบบมีทิศทางเดียวกัน ผล best-of-12 ของ OR-Tools และผลสามรันของโครงการต่างกันเท่าใดในแต่ละขนาด
4. RQ4 — การตรวจแบบแม่นตรง: บนอินสแตนซ์ขนาดเล็กที่ CP-SAT พิสูจน์ OPTIMAL ได้ ช่องว่างของคำตอบโครงการจาก optimum เท่าใด

RQ5 — พฤติกรรมการค้นหาในชุด 100 จุด: trace ของตัวแก้เส้นทางรุ่นปัจจุบัน และการทดลองจับคู่ 5 seed กับตัวแก้เส้นทางรุ่นรอบภูมิภาคภายใต้งบ 100 วินาที บอกอะไรเกี่ยวกับการค้นพบ route partition ที่ดีและความไวของกระบวนการค้นหา

5.2 ชุดทดสอบและเงื่อนไขที่ควบคุมร่วมกัน

DATASETS AND CONTROLLED CONDITIONS

ชุดทดสอบถนนจริงทั้งสี่ขนาดสร้างจากกราฟแบบมีทิศทางและใช้ metric=road จุดยอดแทนตำแหน่งบนเครือข่าย ขอบถนนเก็บระยะทางและความเร็ว ลูกค้าทุกจุดมี demand, service time และกรอบเวลา 08:00–18:00 น. ชุดเหล่านี้จึงใช้ทดสอบ real-road ACVRPTW ภายใต้กรอบเวลาค่อนข้างกว้าง ไม่ใช้แทน benchmark มาตรฐานที่มี time windows รัดกุมกว่า การทดลองตัวแก้ของโครงการและ OR-Tools ดำเนินบน Google Colab VM ที่ควบคุมสภาพแวดล้อมให้ใช้ Intel Xeon 2.4 GHz เพื่อให้การเปรียบเทียบเวลาและคุณภาพคำตอบเกิดภายใต้ platform class เดียวกัน

ลูกค้า	จุดยอด N	เส้นเชื่อม M	รถสูงสุด K	อุปสงค์รวม	ขอบล่างรถ	SHA-256 ของ input
50	694	1,542	10	900	5	12b21609c7e3...
100	1,762	3,986	20	1,800	9	f1592eb53b44...
300	6,367	14,707	60	5,400	27	179fa7137ed0...
450	9,630	20,641	100	8,100	41	727e3a27b374...

ตารางที่ 5.1 ลักษณะของชุดทดสอบบนโครงข่ายถนนจริง

ที่มา: dataset_*.json, input_*.txt และค่า SHA-256 ที่บันทึกในชุดผลการทดลอง; repetition ของแต่ละขนาดใช้ instance เดียวกัน

$$K_{lb} = \lceil (\Sigma_i q_i) / Q \rceil$$

ขอบล่างเชิงความจุของจำนวนรถ โดย q_i คืออุปสงค์ของลูกค้า i และ Q = 200 หน่วย

การยืนยันอินพุต
สาม repetition ของแต่ละขนาดอ้าง input identity เดียวกัน และการเทียบกับ OR-Tools ใช้ instance/road-cost contract เดียวกันในขนาดที่มีหลักฐานครบ ค่า SHA-256 ในตารางทำหน้าที่ป้องกันการเปรียบเทียบผลจากอินพุตคนละชุด

5.3 ระเบียบวิธีทดลองของแกนคำนวณ

NATIVE SOLVER EXPERIMENTAL PROTOCOL

แต่ละชุดปัญหาถูกรันเป็นกระบวนการอิสระ 3 ครั้ง ภายใต้ seed ฐาน และสัญญาการตั้งค่าที่กำหนด ค่า instances=6 หมายถึงเส้นทางการค้นหาภายใน solve เดียวที่ทำงานขนานกัน ไม่ใช่จำนวน repetition. สำหรับรุ่นปัจจุบัน แอปจะเลือกชุดพารามิเตอร์ตามจำนวนลูกค้าและงบเวลาที่ผู้ใช้กำหนด แล้วบันทึกค่าที่เลือกจริงและคำสั่งที่ส่งให้ C++ ไว้เป็นหลักฐาน ทำให้ benchmark ใช้เงื่อนไขเดียวกับ backend ที่ระบบเรียกใช้งานจริง

ลูกค้า	Tier ที่เลือก	Runtime ที่กำหนด (s)	Stage time (s)	No-improvement (s)	Effective instances	Early-LNS workers
50	50-customer	60	6	20	6	1
100	100-customer	120	8	30	6	1
300	300-customer	170	10	45	6	1
450	450-customer heavy	350	10	75	6	6

ตารางที่ 5.2 ค่าที่แอป resolve สำหรับการทดลองตามจำนวนลูกค้าและ runtime ที่กำหนด

ที่มา: dynamic_policy_*.json ซึ่งบันทึกนโยบายที่ระบบเลือกจริงในแต่ละรัน; runtime เป็นงบที่กำหนดจากงานทดลอง ส่วน stage time, patience, จำนวน instances และจำนวน worker ในช่วงค้นหาเริ่มต้นถูกเลือกตามขนาดปัญหาและ runtime

รายการพารามิเตอร์เชิงลึกไม่ได้คัดลอกเป็นคำสั่ง benchmark แยกจากแอป แต่ถูกเลือกผ่าน backend แล้วบันทึก policy/argv ใน evidence ของแต่ละรัน แนวทางนี้ลดความเสี่ยงที่ผลทดลองจะมาจาก configuration คนละชุดกับที่ระบบใช้งานจริง

5.4 ความสมบูรณ์ของหลักฐานและความเป็นไปได้ของคำตอบ

INTEGRITY AND FEASIBILITY GATE

ผลการทดลองผ่าน evidence/feasibility gate ครบ 12 ครั้ง: native process คืนสถานะปกติ ไม่มี timeout เกินกรอบความปลอดภัย มี route ที่ parser อ่านได้ และ independent validator ยืนยัน served-once, capacity, time-window/fleet contract ตามการตั้งค่าของ instance นั้น นอกจากนี้ input identity ที่ผูกกับแต่ละ attempt ยังคงตรงกับ source contract ตลอดการรัน

ลูกค้า	ผ่าน evidence gate	feasible	input ไม่เปลี่ยน	เกินกรอบเวลา
50	3/3	3/3	3/3	0/3
100	3/3	3/3	3/3	0/3
300	3/3	3/3	3/3	0/3
450	3/3	3/3	3/3	0/3

ตารางที่ 5.3 ผลการตรวจ evidence และ feasibility gate

ที่มา: benchmark_manifest.json, run_summary.csv และหลักฐาน stdout/stderr/convergence ของชุดผลการทดลอง

5.5 ผลการทดลองซ้ำของแกนคำนวณ

REPEATED-RUN RESULTS

ลูกค้า	ครั้ง	ระยะทาง (km)	เวลาจริง (s)	รถที่ใช้	ความเป็นไปได้
50	rep01	32.738	26.485	5	ผ่าน
50	rep02	32.738	25.107	5	ผ่าน
50	rep03	32.738	25.088	5	ผ่าน
100	rep01	128.272	48.008	10	ผ่าน
100	rep02	128.051	46.380	10	ผ่าน
100	rep03	125.435	51.250	10	ผ่าน

ลูกค้า	ครั้ง	ระยะทาง (km)	เวลาจริง (s)	รถที่ใช้	ความเป็นไปได้
300	rep01	323.301	106.546	28	ผ่าน
300	rep02	325.943	90.460	29	ผ่าน
300	rep03	325.527	102.314	28	ผ่าน
450	rep01	757.301	165.915	42	ผ่าน
450	rep02	768.193	161.133	42	ผ่าน
450	rep03	762.676	163.731	42	ผ่าน

ตารางที่ 5.4 ผลการทดลองรายครั้ง จำนวน 12 ครั้ง

ที่มา: run_summary_*.csv; ระยะทางต่ำกว่าถือว่าดีกว่า

ลูกค้า	เฉลี่ย (km)	SD (km)	ดีที่สุด (km)	CV	เวลาเฉลี่ย (s)	รถที่ใช้
50	32.738	0.000	32.738	0.000%	25.56	5
100	127.253	1.578	125.435	1.240%	48.55	10
300	324.924	1.420	323.301	0.437%	99.77	28/29
450	762.723	5.446	757.301	0.714%	163.59	42

ตารางที่ 5.5 สถิติเชิงพรรณนาของผลการรันซ้ำ

หมายเหตุ: SD เป็นส่วนเบี่ยงเบนมาตรฐานตัวอย่าง (n = 3); ไม่ใช้การทดสอบนัยสำคัญเนื่องจากจำนวนซ้ำยังน้อย

ความแปรปรวนมีค่า 0 ใน 50 จุดและยังอยู่ต่ำกว่า 1% ของค่าเฉลี่ยในชุด 300 และ 450 จุด ส่วนชุด 100 จุดมี CV เท่ากับ 1.240% ซึ่งสูงที่สุดในสี่ชุด ทั้งนี้ n = 3 เพียงพอสำหรับแสดงพฤติกรรมเบื้องต้น แต่ยังไม่เพียงพอสำหรับสรุปการกระจายของอัลกอริทึมในเชิงสถิติอนุมาน

5.5.1 พัฒนาการของตัวแก้เส้นทาง: จากรอบภูมิภาคสู่รุ่นปัจจุบัน

BACKEND DEVELOPMENT · REGIONAL ROUND → CURRENT SUBMISSION

ก่อนพิจารณาผลของรุ่นปัจจุบัน จำเป็นต้องระบุจุดตั้งต้นให้ชัด: “ตัวแก้เส้นทางรุ่นรอบภูมิภาค” คือ backend ที่ใช้ในการนำเสนอรอบภูมิภาคของโครงการ ไม่ใช่ชื่อแอปและไม่ใช่ baseline ภายนอก. ภายใต้โปรโตคอล benchmark ที่ใช้ในช่วงนั้น รุ่นรอบภูมิภาคเสมอกับ Google OR-Tools ที่ 50 จุด และให้ระยะทางสั้นกว่าที่ 100, 300 และ 450 จุด. หลังรอบภูมิภาคจึงมีการพัฒนา backend ต่อโดยแก้นิยามต้นทุนถนนให้สอดคล้องกับ objective ระยะทาง ยกเลิกตัวคูณ peak-hour ที่ไม่ได้ถูก materialize ใน input เพิ่มนโยบายการค้นหาตามขนาด และเพิ่ม/ขยายกลไกค้นหาเชิงโครงสร้างพร้อม evidence gate ที่เข้มขึ้น

ลูกค้า	รุ่นรอบภูมิภาคเฉลี่ย (native km)	รุ่นปัจจุบันเฉลี่ย (native km)	การเปลี่ยนแปลงของรุ่นปัจจุบัน*	เวลาเฉลี่ยรอบภูมิภาค → ปัจจุบัน (s)
50	32.747	32.738	+0.028%	21.20 → 25.56
100	121.500	127.253	–4.734%	83.56 → 48.55
300	335.763	324.924	+3.228%	161.44 → 99.77
450	777.960	762.723	+1.958%	191.15 → 163.59

ตารางที่ 5.6 ผล native เพื่อดูพัฒนาการจากตัวแก้รุ่นรอบภูมิภาคสู่ตัวแก้รุ่นปัจจุบัน

หมายเหตุ: สำหรับชุด 300 จุดของรุ่นรอบภูมิภาค ใช้ final distance จาก solver_output ของ rep01/rep02/rep03 โดยตรง (336.080654, 334.050479 และ 337.158686 km) เพราะ run_summary ของ rep02 มี appended row มากกว่าหนึ่งแถว; ค่าเฉลี่ยที่ถูกต้องจึงเป็น 335.763273 km. เปอร์เซ็นต์ในตารางนี้เป็น version diagnostic จาก native output ของแต่ละ executable ไม่ใช่คะแนน head-to-head ข้ามรุ่น เพราะนิยาม terminal path ถูกแก้ไขระหว่างการพัฒนา

เมื่ออ่านในฐานะพัฒนาการของรุ่น ค่าเฉลี่ย native objective ของตัวแก้รุ่นปัจจุบันต่ำกว่ารุ่นรอบภูมิภาคที่ 50, 300 และ 450 จุด 0.028%, 3.228% และ 1.958% ตามลำดับ ซึ่งเป็นผลดี แต่สูงกว่าที่ 100 จุด 4.734% จึงไม่สรุปว่า “รุ่นใหม่ดีกว่าเสมอ”: หลักฐานที่ตรงกว่าคือการปรับ backend ให้ผลดีขึ้นชัดเจนในขนาด 300/450 และเล็กน้อยที่ 50 แต่เกิด regression เฉพาะชุด 100 จุด เพราะการเปลี่ยนรุ่นมีหลายปัจจัยพร้อมกัน ตัวเลขนี้บอกทิศทางการพัฒนาแต่ยังไม่พิสูจน์เชิงเหตุว่าองค์ประกอบใดเป็นสาเหตุเพียงตัวเดียว

จุดตั้งต้นจากกรอบภูมิภาค
ตัวแก้เส้นทางที่ใช้ในการนำเสนอรอบภูมิภาคเสมอกับ Google OR-Tools ที่ชุด 50 จุด และให้ระยะทางสั้นกว่าที่ 100, 300 และ 450 จุดภายใต้โปรโตคอลที่ใช้ในขณะนั้น. ผลนี้ทำหน้าที่เป็น baseline ทางประวัติศาสตร์ของการพัฒนา backend ไม่ใช่ผลของตัวแก้รุ่นปัจจุบัน และไม่ถูกนำมาปนกับคะแนน external benchmark รอบปัจจุบัน

5.6 การเปรียบเทียบกับ Google OR-Tools

CONTROLLED BEST-OF-12 GOOGLE OR-TOOLS BENCHMARK

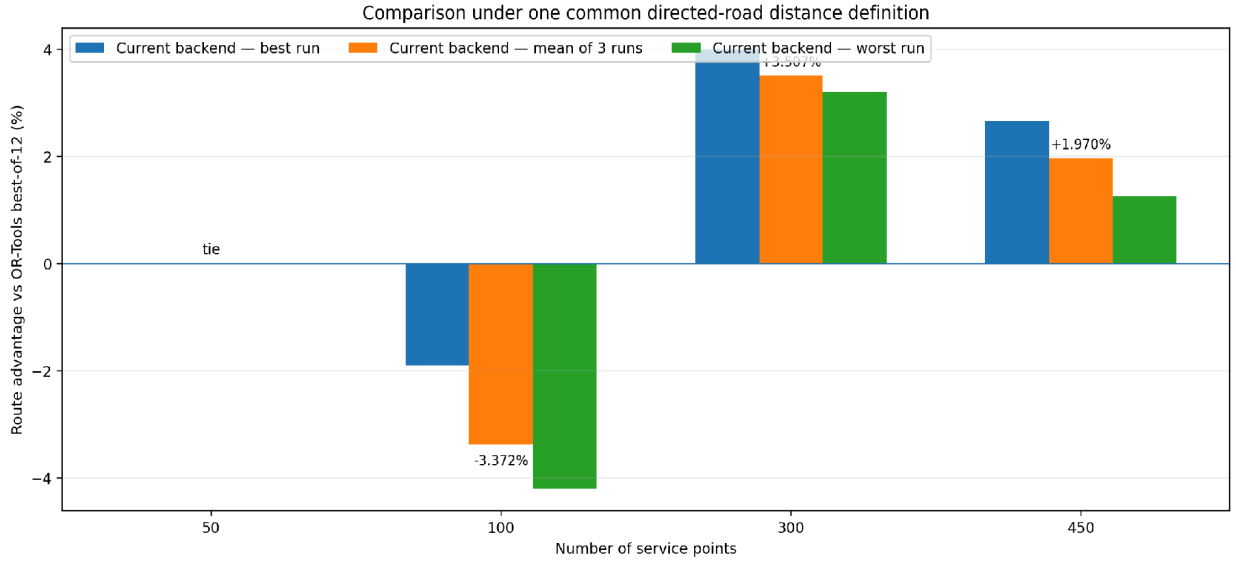
Google OR-Tools ถูกทดสอบ 12 configurations ต่อขนาดจาก objective 3 แบบ × initial strategy 2 แบบ × local-search strategy 2 แบบ และรับเฉพาะ route ที่ผ่าน served-once, time-window และ capacity checks. เพื่อไม่ให้ความต่างของนิยามต้นทุนระหว่าง executable รุ่นต่าง ๆ ทำให้คะแนนคลาดเคลื่อน รายงานจึงถอด route sequence ของ OR-Tools แล้วคำนวณระยะทางใหม่ด้วยนิยาม shortest directed-road distance เดียวกับตัวแก้รุ่นปัจจุบันก่อนเลือก best-of-12. ดังนั้นคำว่า “ชนะ/เสมอ/แพ้” ในส่วนนี้หมายถึงการตัดสินภายใต้ metric เดียวกันจริง ไม่ใช่การหักลบตัวเลข native ที่สร้างจากคนละนิยาม

เกณฑ์ตัดสินข้าม solver: ประเมิน route ด้วย metric เดียวกัน ตัวเลข native จาก executable คนละรุ่นไม่ถูกนำมาหักลบกันโดยตรง . ทุก OR-Tools route ต้องผ่าน feasibility แล้วจึงคำนวณระยะทางใหม่ด้วยนิยาม shortest directed-road distance เดียวกับรุ่นปัจจุบันก่อนเลือก best-of-12. ชุด 50 จุดเป็นตัวอย่างสำคัญ: native output เดิมดูเหมือนรุ่นปัจจุบันสั้นกว่าเล็กน้อย (32.737521 vs 32.746557 km) แต่เมื่อประเมิน route ของ OR-Tools ใหม่ด้วย metric เดียวกันได้ 32.737521 km เท่ากันพอดี จึงตัดสินเป็น “เสมอ”

ลูกค้า	OR best metric เดียวกัน (km)	รุ่นปัจจุบัน best	รุ่นปัจจุบัน mean	รุ่นปัจจุบัน worst	worst-run margin	คำตัดสิน
50	32.738	32.738	32.738	32.738	0.000%	เสมอ 3/3
100	123.102	125.435	127.253	128.272	−4.200%	OR ชนะ 3/3
300	336.733	323.301	324.924	325.943	+3.204%	รุ่นปัจจุบัน ชนะ 3/3
450	778.051	757.301	762.723	768.193	+1.267%	รุ่นปัจจุบัน ชนะ 3/3

ตารางที่ 5.7 ผลตัวแก้เส้นทางรุ่นปัจจุบันเทียบ Google OR-Tools best-of-12 หลังประเมิน route ด้วย metric เดียวกัน

ทุก route ในตารางผ่าน feasibility; หน่วยเป็นกิโลเมตร ค่า + หมายถึงตัวแก้รุ่นปัจจุบันสั้นกว่า OR-Tools best-of-12, 0 คือเสมอ และ − หมายถึง OR-Tools สั้นกว่า



รูปที่ 5.1 ส่วนต่างระยะทางของตัวแก้เส้นทางรุ่นปัจจุบันเทียบ OR-Tools best-of-12 ภายใต้ metric เดียวกัน

กราฟครอบคลุมชุดถนนจริง 50, 100, 300 และ 450 จุด โดยใช้ metric เดียวกัน ค่าบวกหมายถึงตัวแก้รุ่นปัจจุบันสั้นกว่า ค่าลบหมายถึง OR-Tools สั้นกว่า และศูนย์หมายถึงเสมอ

คำตัดสินหลักของตัวแก้เส้นทางรุ่นปัจจุบันเมื่อเทียบกับ Google OR-Tools best-of-12 คือ “ชนะ 2 ขนาด · เสมอ 1 ขนาด · แพ้ 1 ขนาด”: ชุด 50 จุดเสมอที่ 32.737521 km; ชุด 100 จุด OR-Tools ชนะด้วย 123.102066 km เทียบกับ best/mean ของรุ่นปัจจุบัน 125.435011/127.252510 km; ชุด 300 และ 450 จุด รุ่นปัจจุบันมี

mean distance ต่ำกว่า OR-Tools 3.507% และ 1.970%. ผลที่ 300/450 ไม่ได้เกิดจากเลือกเฉพาะรันที่ดีที่สุด เพราะแม้รันที่แย่ที่สุดของโครงการยังสั้นกว่า OR-Tools 3.204% และ 1.267% ตามลำดับ. ภาพจึงชัดว่า backend รุ่นปัจจุบันแข็งแรงขึ้นในสเกล 300/450 แต่ยังมีกรณี 100 จุดที่ OR-Tools ทำได้ดีกว่า

ผล benchmark หลักของตัวแก้รุ่นปัจจุบัน
เมื่อเทียบ Google OR-Tools best-of-12 ด้วย metric เดียวกัน รุ่นปัจจุบันชนะที่ 300 และ 450 จุด เสมอที่ 50 จุด และแพ้ที่ 100 จุด. เมื่อเทียบ repeated runs กับ baseline ประจำขนาด ได้ 6 wins, 3 ties และ 3 losses จาก 12 runs. ผลนี้เป็นคะแนนของ backend รุ่นปัจจุบันเท่านั้น ไม่รวมผลของรุ่นรอบภูมิภาคหรือการทดลองวิเคราะห์ 100 จุด

5.6.1 สภาพแวดล้อมที่ควบคุมและการตีความเวลา

CONTROLLED RUNTIME INTERPRETATION

ลูกค้า	รันของรุ่นปัจจุบันที่ให้ผลดีที่สุด (s)	OR-Tools 12 configs × งบ/รูปแบบ	เพดานงบรวมของ best-of-12 sweep	งบต่อรูปแบบ / เวลารุ่นปัจจุบัน
50	26.485	12 × 30 s	360 s (6 min)	1.13×
100	51.250	12 × 175 s	2,100 s (35 min)	3.41×
300	106.546	12 × 400 s	4,800 s (80 min)	3.75×
450	165.915	12 × 1,000 s	12,000 s (200 min)	6.03×

ตารางที่ 5.8 เวลาเชิงสังเกตของผลที่ให้ระยะทางต่ำสุด

การรันของตัวแก้เส้นทางรุ่นปัจจุบันและ OR-Tools อยู่บน Google Colab VM ภายใต้ hardware protocol เดียวกัน (Intel Xeon 2.4 GHz) ตารางที่ 5.8 แยกชัดระหว่าง runtime ของรันที่ให้ระยะทางดีที่สุด กับงบสูงสุดต่อ OR-Tools configuration และเพดานงบของการกวาด 12 configurations อัตราส่วนคอลัมน์สุดท้ายจึงเป็น budget-to-completion ratio ไม่ใช่ universal algorithmic speedup

ที่ 100 จุด OR-Tools ได้ 175 วินาทีต่อ configuration หรือเพดาน sweep 2,100 วินาที (35 นาที); การกวาดจริงครบ 12 รูปแบบใช้รวม 1,214.022 วินาที และ route ที่ดีที่สุดภายใต้ metric กลางมาจาก distance + Local Cheapest Insertion + Guided Local Search. รันที่

ดีที่สุดของตัวแก้รุ่นปัจจุบันจบใน 51.250 วินาทีแต่ยังยาวกว่า OR-Tools จึงเป็นตัวอย่างว่าความเร็วกับคุณภาพต้องอ่านคู่กัน. ที่ 300/450 จุด OR-Tools ได้งบ 400/1,000 วินาทีต่อรูปแบบ ขณะที่รันที่ดีที่สุดของตัวแก้รุ่นปัจจุบันใช้ 106.546/165.915 วินาทีและยังให้เส้นทางสั้นกว่า best-of-12; ที่ 50 จุดคุณภาพเสมอกัน

5.6.2 แบบจำลองอ้างอิง MILP และการตรวจสอบแบบแม่นตรงด้วย CP-SAT

MILP REFERENCE FORMULATION AND CP-SAT EXACT VALIDATION

เพื่อแยกคำว่า ‘แข่งขันได้’ ออกจากคำว่า ‘เหมาะที่สุด’ รายงานใช้แบบจำลองเชิงคณิตศาสตร์แบบจำนวนเต็มผสม (MILP) เป็นกรอบอ้างอิงของตัวแปรเส้นเชื่อม การเลือกใช้รถ ความจุ ลำดับบริการ และกรอบเวลา แล้วใช้ Google OR-Tools CP-SAT เป็นเครื่องมือตรวจสอบแบบแม่นตรงบนอินสแตนซ์สังเคราะห์ขนาดเล็ก CP-SAT เป็นตัวแก้เชิงข้อจำกัดจำนวนเต็ม ไม่ใช่ MILP solver แบบดั้งเดิม จึงระบุชื่อแบบจำลองและชื่อเครื่องมือแยกกันอย่างชัดเจน

$$\min z = \sum_k \sum_i \sum_j c_{ij}\, x_{ijk}$$

ลดระยะทางรวม โดย $x_{ijk} = 1$ เมื่อรถ k เดินทางจากจุด i ไป j; แบบจำลองยังบังคับให้ลูกค้าแต่ละรายถูกให้บริการหนึ่งครั้ง รักษาการไหลของแต่ละเส้นทาง ความจุรถ และลำดับเวลาบริการ

เกณฑ์รับผลแบบแม่นตรง

รับเฉพาะผลที่ CP-SAT รายงานสถานะ OPTIMAL และมีค่า objective เท่ากับ best bound ภายในความละเอียดที่รายงาน จากนั้นตรวจคำตอบของโครงการด้วยตัวตรวจอิสระว่าครบทุกจุด ไม่เกินความจุ และไม่ละเมิดกรอบเวลา

ลูกค้า	ค่าที่เหมาะที่สุด z* (m)	best bound (m)	โครงการ (m)	ช่องว่างจาก z*
10	18,689.640	18,689.640	18,763.142	+0.3933%
15	28,034.460	28,034.460	28,144.713	+0.3933%

ลูกค้า	ค่าที่เหมาะที่สุด z* (m)	best bound (m)	โครงการ (m)	ช่องว่างจาก z*
20	37,379.280	37,379.280	37,526.284	+0.3933%

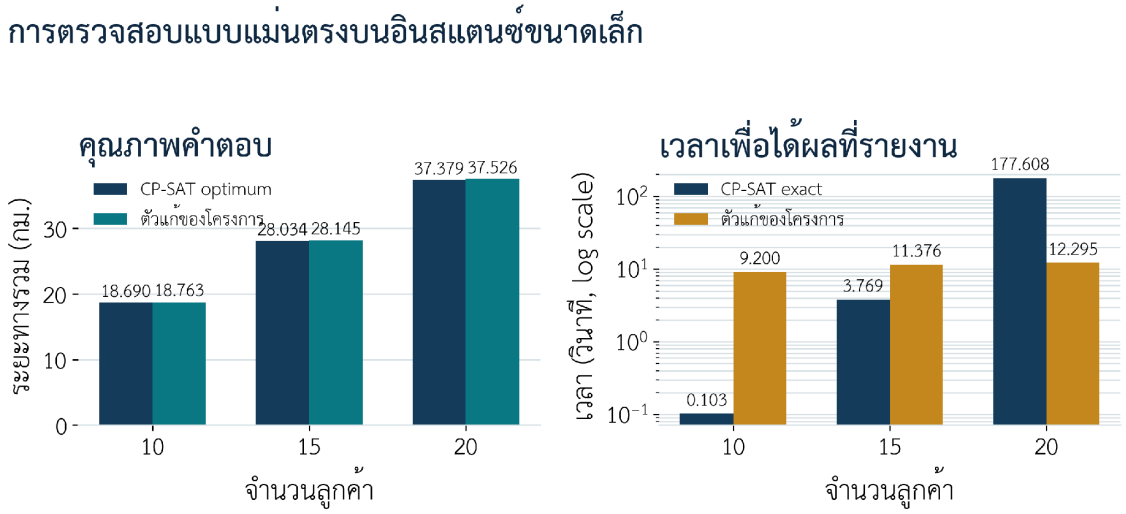
ตารางที่ 5.9 คุณภาพคำตอบเทียบค่าที่เหมาะที่สุดบนอินสแตนซ์ขนาดเล็ก

ที่มา: ชุดหลักฐาน exact validation; CP-SAT รายงาน OPTIMAL และ objective = best bound ครบทั้งสามขนาด

ลูกค้า	สถานะ exact	เวลา CP-SAT (s)	เวลาโครงการ (s)	ตรวจข้อจำกัด
10	OPTIMAL	0.103	9.200	ผ่าน
15	OPTIMAL	3.769	11.376	ผ่าน
20	OPTIMAL	177.608	12.295	ผ่าน

ตารางที่ 5.10 สถานะและเวลาของการตรวจแบบแม่นตรง

เวลา CP-SAT และเวลาโครงการเป็น wall-clock time จาก Google Colab VM / Intel Xeon 2.4 GHz ภายใต้สภาพแวดล้อมการทดลองเดียวกัน จึงรายงานเพื่อแสดง scaling behavior ได้ แต่ไม่ใช้เป็นข้อพิสูจน์ speedup เชิงอัลกอริทึม เพราะ CP-SAT และเมตาฮิวริสติกมี stopping condition และรูปแบบการค้นหาต่างกัน



รูปที่ 5.2 ผลตรวจแบบแม่นตรง: ช่องว่างคำตอบและเวลาเมื่อขนาดเพิ่มขึ้น

แกนเวลาของกราฟด้านขวาใช้สเกลลอการิทึม; ระยะทางของโครงการสูงกว่าค่าที่เหมาะที่สุด 0.3933% ครบทั้งสามขนาด

ผลดังกล่าวแสดงว่าตัวแก้ของโครงการสร้างคำตอบใกล้ optimum บนอินสแตนซ์เล็กที่พิสูจน์ได้และยังผ่านข้อจำกัดครบทุกขนาด อย่างไรก็ตาม exact certificates เหล่านี้ไม่ขยายความหมายไปยังชุดถนนจริง 50–450 จุดซึ่งไม่มี z* ที่พิสูจน์ได้ การประเมินชุดใหญ่จึงรายงาน repeated-run distribution, convergence และ external baseline แยกต่างหาก

5.7 วิเคราะห์กรณี 100 จุด: เหตุใดรุ่นปัจจุบันจึงถดถอย

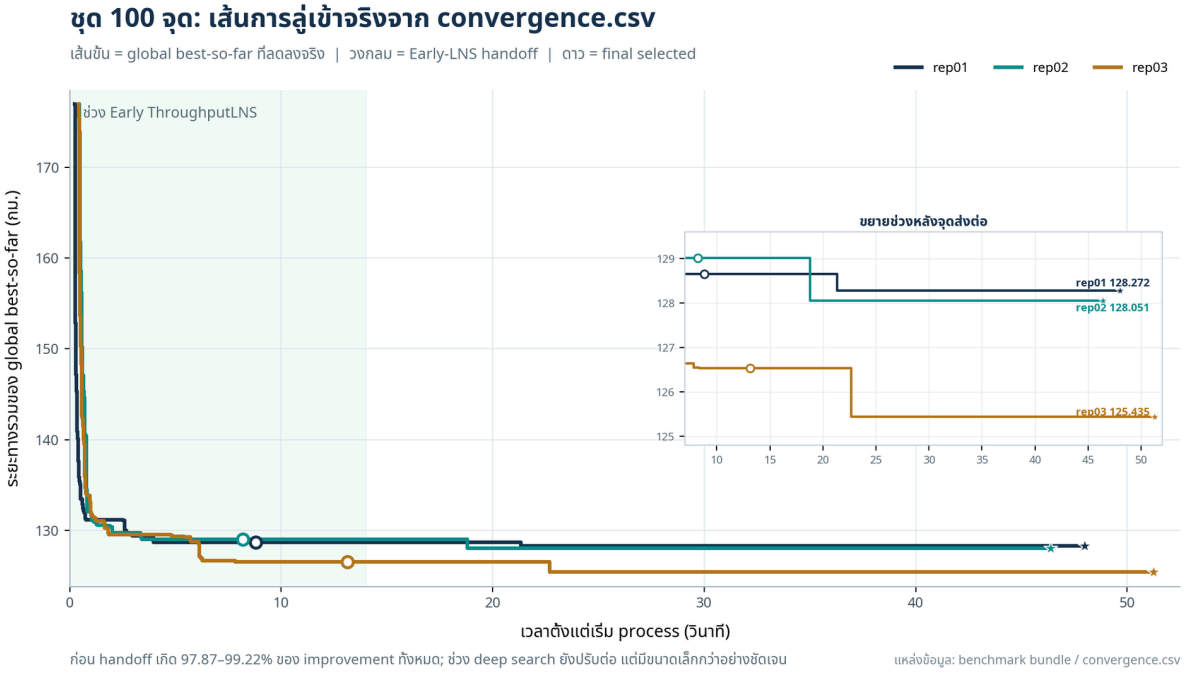
100-CUSTOMER CASE STUDY · SEARCH-DISCOVERY SENSITIVITY

กรณี 100 จุดต้องอ่านตามลำดับ ไม่ใช่ผสมเป็นคะแนนเดียว. ประการแรก ผลหลักของตัวแก้รุ่นปัจจุบันคือสามรันตามนโยบายที่ระบบใช้จริง: mean 127.252510 km และแพ้ OR-Tools best-of-12 ที่ 123.102066 km. ประการที่สอง เพื่อดูว่าปัญหาเกิดจากช่วงค้นหาเริ่มต้นเพียงอย่างเดียวหรือไม่ จึงทำการทดลองวิเคราะห์แยกต่างหาก โดยใช้ source รุ่นปัจจุบันแต่ปิดช่วงค้นหาเร็ว ปิด no-improvement cutoff และให้แกนค้นหาหลักทำงานเต็ม 100 วินาที แล้วจับคู่กับตัวแก้รุ่นรอบภูมิภาคบน 5 seeds ภายใต้ metric เดียวกัน. ประการที่สาม ค่าที่ดีที่สุดจากการทดลองวิเคราะห์ใช้บอกศักยภาพของพื้นที่ค้นหาเท่านั้น ไม่แทนผลหลักของรุ่นปัจจุบัน

ครั้งที่	ช่วงค้นหาเร็ว (s)	จุดส่งต่อ (km)	คำตอบสุดท้าย (km)	ดีขึ้นหลังส่งต่อ (km)	หยุดจริง (s)	Improvement ก่อนส่งต่อ
rep01	8.56	128.651	128.272	0.379	48.00	99.22%
rep02	7.76	129.012	128.051	0.961	46.37	98.03%
rep03	12.70	126.531	125.435	1.096	51.24	97.87%

ตารางที่ 5.11 การกระจาย improvement ก่อนและหลังจุดส่งต่อจากช่วงค้นหาเร็วสู่แกนค้นหาหลักบนชุด 100 จุด

ที่มา: convergence.csv, dynamic_policy.json และ run_summary.csv ของ benchmark bundle; ทั้งสามรันใช้ Savings warm start = false, ช่วงค้นหาเร็ว 1 worker, instances=6, hard budget 120 วินาที และ no-improvement patience 30 วินาที*



รูปที่ 5.3 เส้น global best-so-far ของสามรันบนชุดถนนจริง 100 จุด

กราฟใช้ process clock และค่า global_best_obj จาก convergence.csv โดยตรง เส้นขั้นเปลี่ยนเฉพาะเมื่อ incumbent ที่ดีที่สุดดีขึ้นจริง วงกลมระบุจุดส่งต่อจากช่วงค้นหาเร็วสู่แกนค้นหาหลัก และดาวระบุคำตอบที่ถูกเลือกเป็นผลสุดท้าย; กรอบขยายแสดงการปรับปรุงหลังจุดส่งต่อ จึงไม่ใช่การเชื่อมเพียงสามค่าจากแต่ละ stage

Seed	รุ่นปัจจุบัน การตั้งค่าวิเคราะห์ 100 s (km)	รุ่นรอบภูมิภาค metric เดียวกัน (km)	ผู้ให้ระยะทางสั้นกว่า
777	125.328	122.852	รุ่นรอบภูมิภาค สั้นกว่า 2.476 km
1777	124.751	121.406	รุ่นรอบภูมิภาค สั้นกว่า 3.345 km
2777	125.988	121.354	รุ่นรอบภูมิภาค สั้นกว่า 4.635 km
3777	124.283	122.837	รุ่นรอบภูมิภาค สั้นกว่า 1.446 km
4777	122.983	122.589	รุ่นรอบภูมิภาค สั้นกว่า 0.394 km

ตารางที่ 5.12 การทดลองจับคู่ 5 seed: รุ่นปัจจุบันภายใต้การตั้งค่าวิเคราะห์ 100 วินาที เทียบกับรุ่นรอบภูมิภาคภายใต้ metric เดียวกัน

5.7.1 การตีความร่วมจาก phase trace และการทดลองจับคู่ข้ามรุ่น

JOINT INTERPRETATION OF SEARCH TRACE AND CROSS-VERSION CONTROL

Phase trace ของตัวแก้รุ่นปัจจุบันแสดงว่าช่วงค้นหาเร็วส่ง incumbent ที่ดีให้แกนค้นหาหลัก: จาก baseline 176.951 km ลดสู่ 128.651, 129.012 และ 126.531 km ก่อนจุดส่งต่อ และแกนค้นหาหลักลดต่ออีก 0.379, 0.961 และ 1.096 km (99.22%, 98.03% และ 97.87% ของ improvement เกิดก่อนจุดส่งต่อ) อย่างไรก็ตาม การทดลองจับคู่แสดงว่าเมื่อปิดช่วงค้นหาเร็วของ source รุ่นปัจจุบันแล้วให้เวลา 100 s เต็ม, no-improvement = 0 และ instances = 6, ตัวแก้รุ่นรอบภูมิภาคยังให้ระยะทางสั้นกว่าครบ 5/5 seeds หลังประเมินด้วย metric เดียวกัน. จึงไม่สามารถอธิบาย regression ที่ 100 จุดด้วยช่วงค้นหาเร็วเพียงปัจจัยเดียว

ตัวแก้รุ่นรอบภูมิภาคมี mean common distance 122.207561 km เทียบกับการตั้งค่าวิเคราะห์ของรุ่นปัจจุบัน 124.666764 km หรือสั้นกว่า 1.973%; best/worst ของรุ่นรอบภูมิภาค = 121.353511/122.852426 km และของรุ่นปัจจุบันภายใต้การตั้งค่านี้ = 122.983357/125.988288 km ทั้ง 10 runs ผ่าน coverage, capacity, time windows และ input-identity checks. เมื่อเทียบ OR-Tools baseline 123.102066 km รุ่นรอบภูมิภาคต่ำกว่าครบ 5/5 ส่วนการตั้งค่าทดลองของรุ่นปัจจุบันต่ำกว่า 1/5 (seed 4777 = 122.983357 km). ข้อสรุปที่รองรับจึงคือ backend รุ่นรอบภูมิภาคยังค้นพบ route partition ที่ดีกว่าอย่างสม่ำเสมอบน instance นี้ แม้แกนรุ่นปัจจุบันจะสามารถแตะคำตอบที่ดีกว่า OR-Tools ได้ในบาง seed. เพราะ executable สองรุ่นต่างกันหลายองค์ประกอบ ผลนี้เป็น cross-version diagnostic และต้องทำ same-build one-factor ablation ก่อนสรุปเชิงเหตุ

อ่านผลชุด 100 จุดเป็น 3 ชั้น
ผลหลัก · รุ่นปัจจุบันเทียบ OR-Tools: OR-Tools ชนะ (123.102 km เทียบ best/mean ของรุ่นปัจจุบัน 125.435/127.253 km)
การทดลองวิเคราะห์ · รุ่นรอบภูมิภาคเทียบรุ่นปัจจุบันภายใต้การตั้งค่า 100 ร: รุ่นรอบภูมิภาคชนะ 5/5; mean 122.208 vs 124.667 km
ข้อสังเกต · รุ่นปัจจุบัน seed 4777 ได้ 122.983 km ต่ำกว่า OR-Tools เล็กน้อย แต่เป็น 1/5 diagnostic run จึงไม่เปลี่ยนผล benchmark หลัก

5.8 ความหมายเชิงประยุกต์

OPERATIONAL INTERPRETATION

หลังประเมินทุก route ด้วย metric เดียวกัน ภาพเชิงประยุกต์ชัดขึ้นและไม่พึ่ง metric artifact: ที่ 50 จุด ตัวแก้รุ่นปัจจุบันและ OR-Tools best-of-12 เสมอกัน; ที่ 100 จุด ตัวแก้ในรอบภูมิภาคชนะ OR-Tools และ รุ่นปัจจุบันยังยาวกว่า OR-Tools จึงไม่ควรใช้ขนาดนี้เป็นหลักฐานเชิง superiority. ที่ 300 และ 450 จุด รุ่นปัจจุบันมีระยะทางเฉลี่ยต่ำกว่า OR-Tools best-of-12 ประมาณ 11.809 และ 15.328 กิโลเมตรต่อแผนตามลำดับ และแม้รันที่แย่ที่สุดยังต่ำกว่าอยู่ประมาณ 10.790 และ 9.858 กิโลเมตร หรือ 3.204% และ 1.267%. ความต่างดังกล่าวเป็น computational evidence บน instance ที่รายงาน ไม่ใช่ระยะทางหรือเงินที่รับรองว่าจะประหยัดได้จริงในภาคสนาม

$$S_{annual} = (D_{ref} - D_{sys}) \times R \times Y \times C_{km}$$

D_{ref} ต้องเป็น baseline ที่นิยามชัดเจนในภาคสนาม; R คือจำนวนรอบต่อวัน, Y คือวันปฏิบัติงานต่อปี และ C_{km} คือต้นทุนต่อกิโลเมตร

อย่างไรก็ตาม รายงานไม่แปลงผลเป็นจำนวนเงินบาทโดยไม่มีข้อมูลต้นทุนเชื้อเพลิง ค่าแรง ค่าบำรุงรักษา จำนวนรอบ และจำนวนพื้นที่ปฏิบัติงานจริง สูตรข้างต้นทำหน้าที่เชื่อมผล benchmark กับการประเมินผลกระทบภาคสนามโดยไม่สร้างตัวเลขทางเศรษฐศาสตร์ที่ไม่มีหลักฐาน

โครงสร้างระบบรองรับงานเก็บขยะหลายคันโดยตรง และสามารถประยุกต์กับงานกระจายสินค้าเหลว น้ำดื่ม น้ำแข็ง หรือการส่งของจากโรงงานเดียวไปยังหลายจุดได้เมื่อกำหนดอุปสงค์ ความจุ เวลาบริการ และกรอบเวลาให้ตรงกับงาน การปิดถนนสามารถสะท้อนในกราฟแบบมีทิศทางด้วยการนำเส้นเชื่อมที่ได้รับผลกระทบออกก่อนสร้างเมทริกซ์เส้นทาง แล้วจึงแก้ปัญหาใหม่บนสถานะถนนล่าสุด

5.9 ข้อจำกัดและภัยคุกคามต่อความเที่ยงตรง

LIMITATIONS AND THREATS TO VALIDITY

ประเด็น	ผลต่อการตีความและแนวทางควบคุม
จำนวนการทดลองซ้ำ	benchmark หลักของรุ่นปัจจุบันมี 3 ครั้งต่อขนาดและใช้ seed ฐานตาม protocol จึงรายงานสถิติเชิงพรรณนา; ชุด 100 จุดมีการทดลองจับคู่เพิ่ม 5 seeds ระหว่างรุ่นรอบภูมิภาคกับการตั้งค่าวิเคราะห์ของรุ่นปัจจุบัน แต่เป็น cross-version diagnostic ไม่ใช่ benchmark หลัก
กรอบเวลา	ชุดถนนจริง ใช้กรอบเวลาแบบกว้าง 08:00–18:00 น. แม้ solver รองรับกรอบเวลารัดกุม แต่ผลนี้ไม่แทนการทดสอบ Solomon/Homberger
ฮาร์ดแวร์	ตัวแก้ของโครงการและ OR-Tools ใช้ Google Colab VM ภายใต้ protocol เดียวกันและ CPU Intel Xeon 2.4 GHz จึงเปรียบเทียบ wall-clock time ภายในงานนี้ได้ อย่างไรก็ตาม cloud virtualization อาจมี scheduling noise และ archive ไม่ได้เก็บ hardware fingerprint ราย attempt จึงไม่ขยายผลเป็น speedup ที่เป็นอิสระจาก platform
Metric normalization / OR-Tools 100 จุด	OR-Tools 100 จุดรันครบ 12/12 ที่ 175 s ต่อ configuration และมี route-level evidence. ทุกขนาดถูกประเมิน route ซ้ำด้วย metric เดียวกับรุ่นปัจจุบันก่อนตัดสิน winner; จึงไม่ใช้ native objective จาก executable ที่นิยาม terminal path ต่างกันมาเทียบตรง ๆ
การเลือกค่าพารามิเตอร์	รุ่นรอบภูมิภาคชนะการตั้งค่าวิเคราะห์ของรุ่นปัจจุบัน 5/5 ที่ชุด 100 จุด แต่ executable สองรุ่นต่างกันหลายองค์ประกอบ. ต้องทำ same-build one-factor ablation ก่อนสรุปว่าส่วนใดเป็นสาเหตุ

ประเด็น	ผลต่อการตีความและแนวทางควบคุม
ขอบเขตข้อมูล	แต่ละขนาดมีหนึ่งกราฟถนน การสรุปทั่วไปควรเพิ่มหลายเมือง หลายรูปแบบการกระจายลูกค้า และหลายระดับความหนาแน่นของกรอบเวลา
รุ่นรอบภูมิภาคกับรุ่นปัจจุบัน	historical native comparison ใช้ดูทิศทางการพัฒนาเท่านั้น: รุ่นปัจจุบันต่ำกว่าที่ 50/300/450 และสูงกว่าที่ 100. สำหรับชุด 300 จุดของรุ่นรอบภูมิภาคใช้ final distance จาก solver_output โดยตรงเพราะ run_summary หนึ่งไฟล์มี appended row

ตารางที่ 5.13 ข้อจำกัดของการประเมิน

ข้อจำกัดทั้งหมดถูกคงไว้ในข้อสรุป ไม่ใช้การคาดคะเนเพื่อเติมข้อมูลที่ขาด

5.10 สรุปผลการทดสอบ

EVALUATION CONCLUSION

ตัวแก้เส้นทางรุ่นปัจจุบันผ่าน evidence/feasibility gate ครบ 12/12 รันบนชุดถนนจริง 50, 100, 300 และ 450 จุด. เมื่อย้อนดูพัฒนาการจาก backend ที่ใช้ในรอบภูมิภาค ค่า native ของรุ่นปัจจุบันดีขึ้นที่ 50/300/450 แต่ถดถอยที่ 100; เนื่องจากนิยาม terminal path เปลี่ยนระหว่างรุ่น ผลนี้ใช้บอกทิศทางการพัฒนา ไม่ใช้ตัดสินผู้ชนะข้าม executable

สำหรับ external baseline ทุก OR-Tools route ถูกประเมินซ้ำด้วยนิยามระยะทางเดียวกับรุ่นปัจจุบัน จึงได้คำตัดสินรุ่นปัจจุบันชนะ 2 ขนาด เสมอ 1 ขนาด และแพ้ 1 ขนาด: เสมอที่ 50, แพ้ที่ 100 และชนะที่ 300/450. Mean margins ที่ 300/450 เท่ากับ 3.507%/1.970% และ worst-run margins ยังเป็น 3.204%/1.267%. ที่ 100 จุด OR-Tools ได้ 175 s ต่อ configuration และ sweep จริง 1,214.022 s; รุ่นปัจจุบันใช้เวลาน้อยกว่า (เฉลี่ย 48.546 s) แต่ยังยาวกว่า baseline จึงต้องแยกคุณภาพออกจากเวลา. การทดลองจับคู่ 5 seed แสดงว่ารุ่นรอบภูมิภาคให้ระยะทางสั้นกว่าการตั้งค่าวิเคราะห์ของรุ่นปัจจุบันครบ 5/5; ผลหนึ่ง seed ของรุ่นปัจจุบันที่ต่ำกว่า OR-Tools เป็นเพียง diagnostic

observation ไม่ใช่ผลหลัก. ภาพรวมจึงชี้ว่าจุดอ่อน 100 จุดอยู่ที่ search robustness/discovery และควรตรวจต่อด้วย same-build ablation

ข้อสรุปที่หลักฐานรองรับ
รุ่นปัจจุบัน feasible 12/12. จุดตั้งต้นจากรอบภูมิภาค: backend ที่ใช้ในรอบภูมิภาคเสมอ OR-Tools ที่ 50 และชนะที่ 100/300/450 ตามโปรโตคอลเดิม. พัฒนาการระหว่างรุ่น: รุ่นปัจจุบันมี native result ดีขึ้นที่ 50/300/450 แต่ regress ที่ 100. External baseline รอบปัจจุบันหลังประเมิน route ด้วย metric เดียวกัน: รุ่นปัจจุบันชนะ OR-Tools ที่ 300/450, เสมอ 50, แพ้ 100. การทดลองจับคู่ 100 จุดเป็นหลักฐานวิเคราะห์แยกต่างหากและให้รุ่นรอบภูมิภาคชนะ 5/5; จึงไม่ใช้ผล diagnostic ไปแทนคะแนนหลัก

5.11 ทะเบียนหลักฐานและการตรวจสอบย้อนกลับ

EVIDENCE REGISTER

ประเภท	หลักฐาน	SHA-256
ตัวแก้เส้นทางรุ่นปัจจุบัน	ACVRPTW_V31_DYNAMIC_3RUN_RESULTS(2).zip · manifest + run_summary + logs/convergence	1aa2585a3b84c49c1457e9327499a8e4c0d1e5bae69d769c2f0b624c7f2261f7
solver source	solver.cpp · source receipt	2508570e517419860a9b278e3692065316503a0f67efeb2d9ad16ded16de19de
dynamic policy	app/engine.py · source receipt	3a39a397b019492c3e23d09a53364bb0142badb98b5f9732434cc3f35ead28fb
ตัวแก้เส้นทางรุ่นรอบภูมิภาค	old(4).rar · native outputs/run summaries · ใช้เป็นหลักฐานพัฒนาการและ historical benchmark	53b6bb638e8974c418a22b272eea83fd66f122beffb9be1c857fed3922d408de
การทดลองวิเคราะห์ 100 จุด	5×100s paired ZIP · 5 seeds + common validation + aggregate summary	1a2cff6701ccf09d1520911df8d2902663c87e2337a85e42794c1a88a7da691d

ตารางที่ 5.14 หลักฐานหลักที่ใช้สร้างส่วนผลการทดสอบ
ค่าแฮชใช้ตรวจว่าเอกสารอ้างถึงไฟล์ฉบับเดียวกับที่ส่งมอบ

6 ปัญหาและอุปสรรค
PROBLEMS AND OBSTACLES

6.1 ความเสี่ยงทางเทคนิคและผลต่อการใช้งาน
ENGINEERING RISKS

ความเสี่ยงของระบบไม่ได้จำกัดอยู่ที่โปรแกรมล่มหรือคำตอบช้า แต่รวมถึงความหมายของข้อมูล การเปลี่ยนสถานะระหว่างงานที่ใช้เวลานาน การรักษาแผนที่ผ่านการตรวจแล้วเมื่อ attempt ใหม่ล้มเหลว และความสามารถในการอธิบายว่าแผนหนึ่งเกิดจาก input ใด ตารางนี้จึงบันทึกทั้งมาตรการที่นำมาใช้แล้วและ residual risk ที่ต้องกำกับต่อ ไม่ใช่เพียงรายการ bug ระหว่างพัฒนา

ตารางที่ 6-1 อุปสรรค การแก้ไข และ residual risk

อุปสรรค	ผลกระทบ	แนวทางที่ใช้	ความเสี่ยงที่เหลือ
ข้อมูลถนนมี one-way, parallel edge และ access tag ไม่สม่ำเสมอ	เส้นทางอาจย้อนทิศหรือปิดผิดเส้น	directed graph, reachability, edge identity และ geometry matching	snapshot อาจล้าสมัยหรือ tag ไม่ครบ
terminal matrix space complexity โตประมาณ n^2	RAM และเวลาเตรียมข้อมูลเพิ่มเร็ว	regional cache, shortest-path batching, tiered profiles	ขนาดใหญ่กว่านี้อาจต้อง sparse/on-demand costs
เมตาฮิวริสติกเป็น stochastic	ผลอาจต่างกันตาม seed และเวลา	ระบบรองรับ convergence trace, seed manifest และ stopping rule	ต้องวัดด้วย repeated benchmark ก่อนสรุป distribution
Colab มี session และทรัพยากรไม่คงที่	งานใหญ่หยุดกลางทางได้	two-cell launcher, checkpoint/evidence และ previous-plan preservation	persistent worker queue เป็นงานระยะถัดไป

อุปสรรค	ผลกระทบ	แนวทางที่ใช้	ความเสี่ยงที่เหลือ
ไฟล์ input/output หลายชุดอาจไม่ตรงกัน	รายงานผลผิดอินสแตนซ์	atomic materialization, SHA-256, invocation manifest และ contract revision	ต้องมี storage governance ใน production
UI ต้องรองรับทั้งงานปฏิบัติการและงานวิจัย	ผู้ใช้อาจเห็นรายละเอียดมากเกินงานที่ทำ	map-first flow, guided copy, driver manifest และ truthful states	เก็บ usability metrics ใน controlled pilot
การทดลองปัจจุบันใช้เพียงสามรันต่อขนาดและ Colab ต่าง session	ยังสรุปนัยสำคัญทางสถิติ speedup หรือการใช้งานทั่วไปทุกเมืองไม่ได้	เก็บผลรายรันและข้อจำกัดไว้ครบ พร้อมแยก exact, OR baseline และ field evidence	ต้องเพิ่มหลาย seed ตรึงฮาร์ดแวร์ และทำ controlled field pilot

6.2 ความท้าทายด้านความหมายของระบบ

SEMANTIC ALIGNMENT

ความผิดพลาดที่เป็นอันตรายต่อการตัดสินใจอาจเกิดขึ้นโดยโปรแกรมไม่ล่ม เช่น แสดงว่าเส้นทางเสร็จทั้งที่เพิ่งสร้างกราฟ แสดงรถว่างเพื่อให้ครบ K หรือแสดง objective ของแผนเก่าหลัง graph revision เปลี่ยน โครงการจึงกำหนด semantic contract ให้สถานะ backend, persisted state และข้อความบน UI สื่อความหมายเดียวกัน และไม่ยอมให้ stale evidence ถูกตีความเป็นแผนปัจจุบัน

ตารางที่ 6-2 ปัญหา semantic และเกณฑ์แก้ไข

กรณี	พฤติกรรมที่ป้องกัน	พฤติกรรมระบบ
Build graph	ประกาศ routing complete	แสดง network ready และแผนเบื้องต้นแบบเส้นประ
K vehicles	บังคับสร้าง K route cards	K เป็นเพดาน; ซ่อนรถที่ไม่ใช้

กรณี	พฤติกรรมที่ป้องกัน	พฤติกรรมระบบ
Failed rerun	ลบหรือเขียนทับแผนดีเดิม	เก็บ previous valid plan และหลักฐานรอบที่ล้มเหลว
Road closure	ลบทุกขอบที่ชื่อถนนเดียวกัน	ลบ exact directed edge และ reverse geometry pair เมื่อจำเป็น
Simulation	สื่อว่าเป็น GPS สด	ระบุ planned-route simulation และใช้เวลาจำลอง

6.3 สิ่งที่ทดลองแล้วแต่ไม่เลือกใช้

DESIGN DECISIONS

- ไม่ใช้เปอร์เซ็นต์ progress แบบประมาณสำหรับ C++/OSMnx/osmium เพราะไม่มี denominator ที่ตรวจสอบได้ จึงแสดง elapsed time และ live state แทน
- ไม่ปิดถนนตามชื่อ เนื่องจากชื่อเดียวกันอาจอยู่คนละทิศหรือคนละ geometry
- ไม่ commit candidate เพียงเพราะ process exit code เป็นศูนย์ ต้องผ่าน parser, independent validator และ fleet cross-check
- แยกผล SparseSpectral และ Savings ในระดับ preprocessing ออกจาก final routing เพราะ compactness เพียงค่าเดียวไม่อธิบายคุณภาพคำตอบปลายทาง
- ไม่ยกระดับ planned-route simulation เป็น live tracking เพื่อให้ถ้อยคำสอดคล้องกับ capability จริง

6.4 บทเรียนที่มีผลต่อการออกแบบ

LESSONS LEARNED

บทเรียนเชิงระบบ

การทำให้ตัวแก้เร็วขึ้นและการทำให้ผู้ใช้เชื่อผลลัพธ์เป็นคนละปัญหา ระบบที่นำไปประยุกต์ต้องตอบได้ว่าใช้ข้อมูลชุดใด รันอะไร ตรวจอะไร และจะเกิดอะไรเมื่อรอบใหม่ไม่สำเร็จ

บทเรียนสำคัญคือความซับซ้อนควรอยู่หลังขอบเขตที่ตรวจสอบได้ ไม่ใช่ผลักไปยังผู้ใช้ ระยะแรกของโครงการเน้นการพิสูจน์แกนคำนวณผ่าน notebook ส่วนระบบปัจจุบันทำให้ UI แสดงเฉพาะข้อมูลที่จำเป็นต่อการตัดสินใจ ขณะที่ command, hash, validation และ convergence ยังเปิดให้ผู้ตรวจสอบเข้าถึงได้ แนวทางนี้ช่วยรักษาทั้ง usability และความซื่อสัตย์ของหลักฐานโดยไม่บังคับให้ผู้ขับต้องเข้าใจรายละเอียดของ search pipeline

7 แนวทางการพัฒนาและประยุกต์ใช้ในขั้นต่อไป
FUTURE DEVELOPMENT AND APPLICATION

7.1 ลำดับความสำคัญของระยะถัดไป
PRIORITIZED ROADMAP

งานต่อยอดจากกรณี 100 จุดควรเริ่มด้วย incumbent-injection และ same-build factorized ablation ใช้ route จากตัวแก้รุ่นรอบภูมิภาคที่ผ่าน common validator เป็น starting incumbent ให้ source รุ่นปัจจุบัน แล้วตรึง initial state, time, thread count และ stopping rule ก่อนสลับทีละกลไก เป้าหมายคือแยกว่าแกนค้นหารุ่นปัจจุบันไม่สามารถรักษา/ปรับปรุง basin ที่ดี หรือเพียงค้นพบ basin นั้นได้ไม่สม่ำเสมอ ห้ามนำการตั้งค่าทดลอง 100 วินาทีไปแทน benchmark หลักของรุ่นปัจจุบันในการสรุปผล

ตารางที่ 7-1 Roadmap ที่วัดผลได้

ลำดับ	งานต่อยอด	การทดลอง/ผลส่งมอบ	เกณฑ์ผ่าน
P1	Ablation และ benchmark หลาย seed	ตรึงเครื่องและแยกผล Early ThroughputLNS, patience, incumbent inheritance	raw artifacts ครบ; validator ผ่าน; รายงาน distribution และ hardware
P2	Controlled field pilot	ผู้วางแผนและคนขับทำภารกิจจำลอง/จริงขนาดเล็ก	task success, error rate, SUS และ feedback
P3	Operational impact study	เปรียบเทียบแผนเดิมกับระบบหลายวัน	vehicle-km, duration, fleet use และข้อร้องเรียน
P4	Time-dependent road costs	รับ traffic/travel-time feed ที่มี timestamp	replay test และ no-regression feasibility

ลำดับ	งานต่อยอด	การทดลอง/ผลส่งมอบ	เกณฑ์ผ่าน
P5	Enterprise operations	identity, roles, audit log, backup และ durable worker queue	security/recovery tests และ SLA ที่กำหนด
P6	Richer VRP	heterogeneous fleet, multi-depot, pickup-delivery และ vehicle skills	formal contract และ small-case validation

7.2 การประเมินแกนคำนวณในขั้นถัดไป

NATIVE SOLVER BENCHMARK

การทดลองหลักใช้ตัวแก้เส้นทางรุ่นปัจจุบันภายใต้นโยบายที่ระบบเลือกจริงและเก็บสาม repetitions ต่อขนาดบน Google Colab VM / Intel Xeon 2.4 GHz. หลักฐานของรุ่นรอบภูมิภาคถูกเก็บแยกเพื่อดูพัฒนาการของ backend; การทดลองจับคู่ 100 จุดเป็น auxiliary cross-version experiment 5 seeds พร้อม balanced order และ common route evaluator. ขั้นถัดไปจึงเน้น same-build causal ablation, CPU/compiler fingerprint ราย attempt และจำนวนซ้ำที่มากพอสำหรับ inference

- จุดอ้างอิงแต่ละชนิดตอบคนละคำถาม: ผลของรุ่นปัจจุบันใช้ตอบสมรรถนะของระบบที่ส่งมอบ; รุ่นรอบภูมิภาคใช้ดูทิศทางการพัฒนา; Google OR-Tools best-of-12 ที่ประเมินด้วย metric เดียวกันใช้เป็น external empirical baseline; exact CP-SAT ใช้เฉพาะอินสแตนซ์เล็กที่พิสูจน์ optimum ได้; การทดลองจับคู่ 100 จุดใช้วิเคราะห์ search sensitivity. ผลคนละกลุ่มจะไม่ถูกนำมารวมเป็นค่าเฉลี่ยเดียว
- ทำ incumbent-injection experiment โดยเริ่ม source รุ่นปัจจุบันจาก route ของรุ่นรอบภูมิภาคที่ผ่าน common validator แล้วตรวจว่ารักษาหรือปรับปรุงคำตอบได้หรือไม่. จากนั้นทำ same-build ablation ทีละปัจจัยและเพิ่ม paired seeds เพื่อแยก “ค้นหา basin ไม่เจอ” ออกจาก “แกนค้นหาปรับปรุง basin นั้นไม่ได้”

- เก็บ final objective, feasible rate, used vehicles, wall-clock runtime, anytime convergence, CPU/compiler metadata และ peak memory พร้อมรายงานทั้งค่าเฉลี่ย ส่วนเบี่ยงเบน และผลรายรันโดยไม่เลือกเฉพาะคำตอบดีที่สุด
- ส่งมอบ raw routes, independent-validator output, command, dynamic policy, input checksum และตารางสรุปที่สร้างจากไฟล์ดิบชุดเดียวกัน เพื่อให้ผลทุกบรรทัดเชื่อมกลับไปยังหลักฐานต้นทางได้

7.3 การทดสอบภาคสนาม

FIELD VALIDATION

1. กำหนดงานตัวอย่างที่ไม่กระทบภารกิจสำคัญ พร้อมแผนสำรองที่ผู้ควบคุมอนุมัติ
2. ให้ผู้วางแผนสร้างกราฟ ตรวจจุดงาน รัน optimizer และส่งใบงานโดยไม่แก้โค้ด
3. ให้คนขับอ่านลำดับจุด เปิด navigation handoff และส่งรายงานถนนจำลอง
4. วัด task completion, เวลา, error, จำนวนครั้งที่ต้องขอความช่วยเหลือ และ SUS
5. สัมภาษณ์แบบกึ่งโครงสร้างเพื่อหาคำศัพท์หรือขั้นตอนที่ยังไม่ตรงกับภาษางานจริง
6. เปรียบเทียบ planned route กับ execution trace โดยแยกเหตุผลที่เบี่ยงเส้นทาง เช่น ความปลอดภัย ถนนเข้าไม่ได้ หรือคำสั่งภาคสนาม

หลักจริยธรรมในการนำร่อง

ระบบทำหน้าที่เป็นเครื่องมือช่วยวางแผน การตัดสินใจของผู้ควบคุมงานและความปลอดภัยบนถนนมีลำดับสูงกว่า objective ผู้ขับสามารถเบี่ยงจากแผนเมื่อสภาพจริงไม่ปลอดภัยหรือขัดกฎจราจร

7.4 ข้อมูลสดและการปรับเส้นทาง

DYNAMIC OPERATIONS

การขยายสู่ dynamic ACVRPTW ต้องเพิ่ม timestamped travel costs, current vehicle state, completed/locked stops และ horizon policy อย่างเป็นระบบ ไม่ควรเพียงเรียก API จราจรแล้วแทน matrix ระหว่างรอบ เพราะจะทำให้คำตอบเปลี่ยนโดยไม่ทราบ data revision งานต่อยอดจึงต้องบันทึก cost snapshot, event time, lock set และเหตุผลของการ replan ควบคู่กับ route version ใหม่

$$Plan_{t+1} = Optimize(G_{t+1}, \text{unfinished jobs, vehicle states, locked commitments}) \quad (10)$$

7.5 การขยายแบบจำลองสู่โดเมนอื่น

DOMAIN EXTENSIONS

สถาปัตยกรรมศูนย์เดียว-รถหลายคันสามารถใช้เป็นฐานสำหรับการกระจายน้ำดื่ม น้ำแข็ง ของเหลวชนิดเดียว และงานบริการภาคสนาม เมื่อ demand และ capacity สามารถแทนปริมาณจริงได้ อย่างไรก็ตาม การย้ายโดเมนต้องเริ่มจาก constraint audit เพื่อระบุข้อจำกัดเฉพาะ เช่น ความเข้ากันได้ของสินค้า ขนาดรถ เวลาบริการ หรือข้อจำกัดพื้นที่ ก่อนนำตัวแก้ไปใช้จริง

ตารางที่ 7-2 ขอบเขตการประยุกต์ซ้ำ

โดเมน	สิ่งที่ใช้ซ้ำได้	สิ่งที่ต้องตรวจ/เพิ่ม
เก็บขยะ	depot, capacity, service time, directed access	disposal trips, zone policy, vehicle-road access
ส่งน้ำ/น้ำแข็ง	single depot, multi-trip, demand	cold chain, delivery windows, container return
ของเหลวชนิดเดียว	volume capacity และ route plan	compartments, contamination, loading/unloading rules
บริการภาคสนาม	time windows และ route order	technician skills, parts, appointment priority

8 ข้อสรุปและข้อเสนอแนะ
CONCLUSIONS AND RECOMMENDATIONS

8.1 คำตอบต่อคำถามวิจัย
ANSWERS TO RESEARCH QUESTIONS

ตารางที่ 8-1 คำตอบต่อ RQ-1 ถึง RQ-5

คำถาม	คำตอบจากหลักฐาน
RQ-1 · แบบจำลองและข้อมูล	directed-road graph, terminal matrix, ACVRPTW data contract และ independent validator ใช้ความหมายร่วมกันตลอด pipeline
RQ-2 · การส่งต่อผล	native C++ handoff, parser, independent validator และ atomic commit ใช้สัญญาเดียวกัน; candidate ที่ไม่ผ่านหรือไม่ดีกว่าไม่เขียนทับแผนที่ยืนยันแล้ว
RQ-3 · ความจริงของสถานะ	UI แยก network ready, initial plan, optimizing, ready และ stale จาก persisted workflow state ไม่ใช้เปอร์เซ็นต์ปลอมกับงานที่ไม่มี denominator
RQ-4 · เหตุการณ์ถนน	incident approval เปลี่ยน graph revision ระบุแผนเดิมว่า stale และ failed rerun ไม่ทำลาย previous valid plan
RQ-5 · ขอบเขตหลักฐาน	อินสแตนซ์เล็กที่พิสูจน์ optimum ได้มี gap 0.3933% และตัวแก้เส้นทางรุ่นปัจจุบันผ่าน feasibility ครบ 12/12 รันบนชุดถนนจริงสี่ขนาด. เมื่อดูพัฒนาการจาก backend รอบภูมิภาค ค่า native ของรุ่นปัจจุบันต่ำกว่าที่ 50/300/450 แต่สูงกว่าที่ 100; ใช้ผลนี้เพื่อดูทิศทางการพัฒนา ไม่ใช่ตัดสินข้าม executable. สำหรับ external baseline ที่ประเมิน route ด้วย metric เดียวกัน รุ่นปัจจุบันชนะ OR-Tools best-of-12 ที่ 300/450 เสมอที่ 50 และแพ้ที่ 100; worst-run margins ที่ 300/450 = 3.204%/1.267%. การทดลองจับคู่ 100 จุดถูกแยกจากผลหลักและพบว่ารุ่นรอบภูมิภาคให้ระยะทางสั้นกว่าการตั้งค่าวิเคราะห์ของรุ่นปัจจุบันครบ 5/5 คู่

8.2 ข้อสรุปหลัก

MAIN CONCLUSION

โครงการเชื่อม directed-road model, ACVRPTW constraints, hybrid C++ search, independent validation และชั้นปฏิบัติการเข้าด้วยกันโดยรักษาความหมายของ capacity, time windows, multiple trips และ maximum-fleet semantics ตลอด pipeline จุดแข็งจึงไม่ได้อยู่ที่จำนวนกลไกภายใน แต่คือการทำให้ข้อมูลที่ผู้ใช้กำหนด candidate ที่ native solver สร้าง และแผนที่เผยแพร่เป็นวัตถุเดียวกันในเชิงสัญญา สามารถย้อนตรวจผ่าน graph revision, input hash, command, route schedule และ evidence ได้

ในระดับการใช้งาน ระบบจัดการทั้งความหมายและความรับผิดชอบ: ผู้วางแผนเห็นความพร้อมของข้อมูลและสถานะแผน คนขับได้รับเฉพาะรถที่ถูกใช้งานจริง เหตุการณ์ถนนต้องผ่านการอนุมัติก่อนเปลี่ยนกราฟ และ attempt ที่ล้มเหลวไม่ทำลาย previous valid plan ขณะเดียวกันผู้ตรวจสอบยังเข้าถึง command, hash, logs, convergence และ validation result ของแต่ละ route version ได้

ข้อสรุปหนึ่งประโยค

ระบบเปลี่ยนปัญหาเส้นทางที่ซับซ้อนให้เป็นกระบวนการตัดสินใจที่ตรวจสอบได้ ตั้งแต่ถนน จุดงาน และข้อจำกัด ไปจนถึงแผนรถ เหตุการณ์ภาคสนาม และหลักฐานของทุก route version

หลักฐานเชิงสมรรถนะทำให้ข้อสรุปมีน้ำหนักมากกว่าการสาธิตระบบเพียงอย่างเดียว. Exact cases 10–20 จุดให้ gap 0.3933% และตัวแก้รุ่นปัจจุบันผ่าน feasibility 12/12 รัน. เมื่อมองพัฒนาการจาก backend ที่ใช้ในรอบภูมิภาค รุ่นปัจจุบันมี native mean distance ต่ำกว่าที่ 50/300/450 แต่สูงกว่าที่ 100; ตัวเลขนี้ใช้ดู version direction ไม่ใช่ head-to-head score เพราะนิยาม terminal path ถูกแก้ระหว่างรุ่น. สำหรับ external baseline ที่ประเมิน route ด้วย metric เดียวกัน รุ่นปัจจุบันชนะ OR-Tools ที่ 300/450, เสมอที่ 50 และแพ้ที่ 100; แม้ worst runs ที่ 300/450 ยังสั้นกว่า 3.204%/1.267%. การทดลองจับคู่

100 จุดเป็นหลักฐานอีกชุดหนึ่งและพบรุ่นรอบภูมิภาคชนะการตั้งค่าวิเคราะห์ของรุ่นปัจจุบัน 5/5. ดังนั้นข้อสรุปที่ตรงกับข้อมูลคือ backend ปัจจุบันแข็งแรงขึ้นชัดในขนาด 300/450 ขณะที่กรณี 100 จุดยังเปิดเผยความไวของ search discovery ที่ต้องแยกสาเหตุด้วย same-build ablation ต่อไป

8.3 แนวทางนำระบบไปใช้

DEPLOYMENT GUIDELINES

- เริ่มจากพื้นที่และจำนวนรถที่ควบคุมได้ ตรวจข้อมูลถนนและจุดงานร่วมกับผู้ปฏิบัติงานก่อนรันจริง
- ใช้ระบบเป็น decision support และคงอำนาจ override ของผู้ควบคุมงาน โดยเฉพาะเมื่อมีประเด็นความปลอดภัยหรือข้อมูลถนนไม่ทันสมัย
- เก็บ baseline หลายวันและวัดทั้ง vehicle-km, duration, vehicles used, missed jobs และ feedback ไม่วัดเฉพาะ objective
- กำหนดผู้มีสิทธิ์อนุมัติ road incident และเก็บ audit trail ก่อนเชื่อมข้อมูลสด
- ทวนสอบ constraint ของแต่ละโดเมนก่อนใช้งานกับน้ำ น้ำแข็ง ของเหลว หรือธุรกิจอื่น

8.4 ระดับความพร้อมและขั้นต่อไป

READINESS AND NEXT STAGE

ระบบอยู่ในระดับต้นแบบวิจัยที่เชื่อม frontend, backend และแกน C++ จริงพร้อม controlled benchmark ตามขอบเขตที่รายงาน ขั้นต่อไปคือ multi-seed ablation บนเครื่องที่ตรึง การทดสอบชุดมาตรฐานที่มีกรอบเวลารัดกุม การทดลองหลายพื้นที่ และ controlled field pilot ที่วัดทั้ง vehicle-km, duration, missed jobs และ feedback ของผู้ใช้งาน

9 เอกสารอ้างอิง
REFERENCES

[1] Dantzig, G. B., and Ramser, J. H. (1959). The Truck Dispatching Problem. Management Science, 6(1), 80–91.

[2] Clarke, G., and Wright, J. W. (1964). Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. Operations Research, 12(4), 568–581.

[3] Laporte, G. (2009). Fifty Years of Vehicle Routing. Transportation Science, 43(4), 408–416.

[4] Toth, P., and Vigo, D. (Eds.). (2014). Vehicle Routing: Problems, Methods, and Applications (2nd ed.). Society for Industrial and Applied Mathematics.

[5] Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P. (1983). Optimization by Simulated Annealing. Science, 220(4598), 671–680.

[6] Glover, F. (1986). Future Paths for Integer Programming and Links to Artificial Intelligence. Computers & Operations Research, 13(5), 533–549.

[7] Ropke, S., and Pisinger, D. (2006). An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. Transportation Science, 40(4), 455–472.

[8] von Luxburg, U. (2007). A Tutorial on Spectral Clustering. Statistics and Computing, 17, 395–416.

[9] Ng, A. Y., Jordan, M. I., and Weiss, Y. (2002). On Spectral Clustering: Analysis and an Algorithm. Advances in Neural Information Processing Systems 14.

[10] Boeing, G. (2017). OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks. Computers, Environment and Urban Systems, 65, 126–139.

[11] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. Proceedings of the 7th Python in Science Conference, 11–15.

[12] Harris, C. R., et al. (2020). Array Programming with NumPy. Nature, 585, 357–362.

[13] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[14] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[15] Google. OR-Tools Documentation: Vehicle Routing and CP-SAT. https://developers.google.com/optimization (accessed 2026).

[16] OpenStreetMap contributors. OpenStreetMap data. https://www.openstreetmap.org/copyright (accessed 2026).

[17] PyVRP contributors. PyVRP Documentation. https://pyvrp.org/ (accessed 2026).

[18] CVRPLIB. Capacitated Vehicle Routing Problem Library. http://vrp.atd-lab.inf.puc-rio.br/ (accessed 2026).

[19] Vidal, T. (2022). Hybrid Genetic Search for the CVRP: Open-Source Implementation and SWAP* Neighborhood. Computers & Operations Research, 140, 105643.

[20] Solomon, M. M. (1987). Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. Operations Research, 35(2), 254–265.

[21] Gehring, H., and Homberger, J. (1999). A Parallel Hybrid Evolutionary Metaheuristic for the Vehicle Routing Problem with Time Windows. Proceedings of EUROGEN99, 57–64.

[22] FastAPI. Official Documentation. https://fastapi.tiangolo.com/ (accessed 2026).

[23] SQLite. Official Documentation. https://www.sqlite.org/docs.html (accessed 2026).

[24] Leaflet. Official Documentation. https://leafletjs.com/reference.html (accessed 2026).

แหล่งข้อมูลกับส่วนสมทบ

ข้อมูลและซอฟต์แวร์ภายนอกได้รับการอ้างอิงตามบทบาท และไม่ถือเป็นส่วนสมทบใหม่ของโครงการ

10 สถานที่ติดต่อผู้พัฒนาและอาจารย์ที่ปรึกษา
CONTACT INFORMATION

10.1 ข้อมูลโครงการ
PROJECT RECORD

ตารางที่ 10-1 ข้อมูลผู้พัฒนาและโครงการ

รายการ	ข้อมูล
ชื่อโครงการภาษาไทย	ระบบเพิ่มประสิทธิภาพเส้นทางรถเก็บขยะหลายคันบนโครงข่ายถนนจริงด้วยอัลกอริทึมไฮบริดเมตาฮิวริสติก
ชื่อภาษาอังกฤษ	A Hybrid Metaheuristic Optimization System for Multi-Truck Waste Collection Routing on Real Road Networks
ประเภท	โปรแกรมเพื่อการประยุกต์ใช้งาน · ระดับนักเรียน (หมวด 23)
ผู้พัฒนา	นายรัชชานนท์ ปรีชา
สถานศึกษา	โรงเรียนแสงทองวิทยา
ครูที่ปรึกษา	นางสาวปัณทิตา ก้องกิตไพศาล
หัวหน้าสถาบัน	ดร.ถนัด อนันต์
อีเมลหัวหน้าทีม	preecharatchanon@gmail.com
โทรศัพท์	
เอกสารฉบับนี้	รายงานฉบับสมบูรณ์ NSC 2026 · สิงหาคม 2569

10.2 รายการส่งมอบ
DELIVERABLES

- Google Colab notebook แบบ 2 เซลล์ที่ฝัง source package ของแอปพลิเคชัน
- FastAPI backend, SQLite state, HTML/CSS/JavaScript frontend และ Driver App

- C++17 solver และ clustering source พร้อม command/evidence pipeline
- ชุด regression ภายในที่ครอบคลุม data contract, native solver handoff, validation, transactions, route-state semantics และการใช้งานหลัก
- รายงานฉบับเต็ม DOCX/PDF พร้อมคู่มือติดตั้ง คู่มือใช้งาน และข้อจำกัด

ช่องทางสอบถาม
การนำระบบไปใช้กับข้อมูลจริงควรแนบคำอธิบายรูปแบบข้อมูล พื้นที่ ข้อจำกัดรถ และ objective ที่ต้องการ เพื่อให้ตรวจ domain compatibility ก่อนปรับใช้งาน

A	ภาคผนวก APPENDIX
ก	คู่มือการติดตั้งอย่างละเอียด DETAILED INSTALLATION GUIDE

ก.1 สิ่งที่ต้องมีเพื่อติดตั้ง

PREREQUISITES

- บัญชี Google ที่เปิด Google Colab ได้ และเบราว์เซอร์สมัยใหม่
- การเชื่อมต่ออินเทอร์เน็ตสำหรับติดตั้ง dependency และเปิด HTTPS tunnel ในครั้งแรก
- ไฟล์ Google Colab notebook ระบบของโครงการ
- หากใช้ offline Thailand road mode ให้เตรียมพื้นที่เก็บ PBF/cache ตามขนาดพื้นที่

โครงสร้าง notebook

ไฟล์ส่งมอบมี 2 เซลล์ที่ต้องรัน: เซลล์แรกฝัง source package ทั้งแอปและติดตั้ง/คอมไพล์ ส่วนเซลล์ที่สองเริ่มระบบและแสดงลิงก์เข้าใช้งาน ไม่จำเป็นต้อง upload ZIP เพิ่ม

ก.2 ขั้นตอนการติดตั้ง

INSTALL

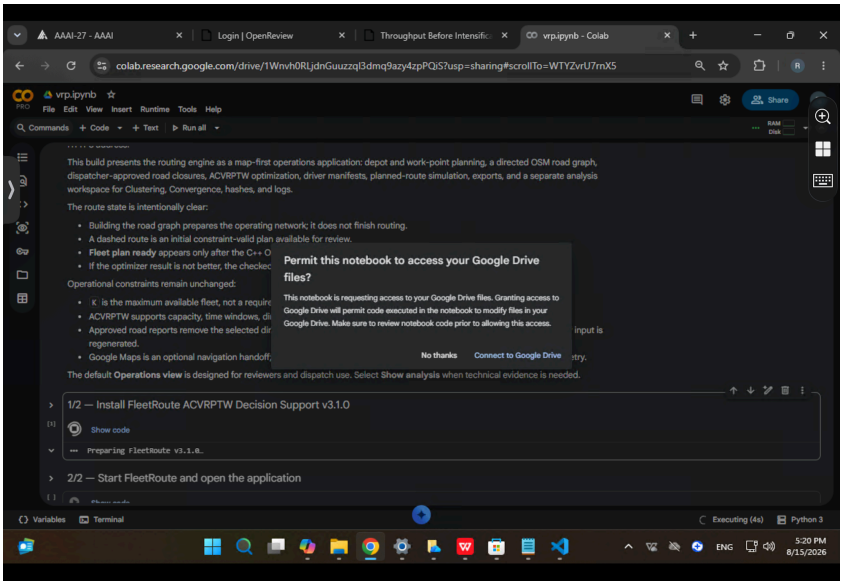
1. เปิด notebook ใน Google Colab และเลือก Runtime → Change runtime type หากต้องการตรวจประเภทเครื่อง
2. รันเซลล์ที่ 1 เพียงครั้งเดียวต่อ session รอจนการแตกแพ็กเกจ ติดตั้ง Python packages และคอมไพล์ solver.cpp/cluster.cpp เสร็จ
3. รันเซลล์ที่ 2 เพื่อเริ่ม FastAPI/Uvicorn บนพอร์ตว่าง ระบบจะแสดง local health status และลิงก์ HTTPS ชั่วคราว

4. เปิดลิงก์ที่แสดงและรอหน้า System overview โหลดครบ หาก Colab แสดงคำเตือน tunnel ให้ยืนยันเฉพาะลิงก์ที่สร้างจาก notebook ฉบับนี้

Two-cell launch contract
Cell 1 -> unpack embedded app -> install dependencies -> compile C++17 binaries
Cell 2 -> choose free port -> start FastAPI/Uvicorn -> publish temporary HTTPS URL

Colab Notebook link:
https://colab.research.google.com/drive/1Wnvh0RLjdnGuuzzql3dmq9azy4zpPQiS?usp=sharing

คลิก “No thanks” เมื่อรัน Cell 1



ก.3 การตรวจหลังเปิดระบบ

POST-LAUNCH CHECK

ตารางที่ ก-1 Checklist หลังติดตั้ง

จุดตรวจ	สิ่งที่ควรเห็น	หากไม่ตรง
System overview	พื้นหลังและแผนที่สว่าง พร้อมสถานะ backend	refresh หนึ่งครั้งและดู Cell 2 logs

จุดตรวจ	สิ่งที่ควรเห็น	หากไม่ตรง
Health/API	สถานะพร้อม ไม่มี error banner	ตรวจว่า process ยังรันและพอร์ตไม่ถูกใช้ซ้ำ
Optimizer	พบ solver binary และ profile	รัน Cell 1 ใหม่หาก Colab runtime session ถูก reset
Road runtime	พร้อมให้ build/download ตามโหมด	ตรวจพื้นที่ดิสก์และ network ของ Colab
Persistence	SQLite runtime สร้างได้	ใช้ session ใหม่หากไฟล์ runtime เดิมเสียหาย

ก.4 การแก้ปัญหาเบื้องต้น

TROUBLESHOOTING

ตารางที่ ก-2 ปัญหาที่อาจพบ

อาการ	สาเหตุที่เป็นไปได้	วิธีตรวจ/แก้
ลิงก์เปิดไม่ได้	tunnel หมดอายุหรือ process หยุด	ดู output เซลล์ 2 แล้วรันเซลล์ 2 ใหม่
solver binary missing	Cell 1 ไม่จบหรือ runtime reset	รัน Cell 1 และตรวจ compiler output
ดาวน์โหลดถนนช้า	network/PBF/cache ยังไม่พร้อม	ใช้พื้นที่เล็กก่อนและดู progress bytes/rate
งานบน App ค้าง	งาน C++/OSMnx ยังทำงานหรือทรัพยากรไม่พอ	ดู elapsed/logs; ยกเลิกผ่าน UI และคงแผนเดิม
หน้าเว็บแสดงข้อมูลเก่า	route/data revision เปลี่ยน	refresh แล้วอ่าน stale banner ก่อนสั่งรันใหม่

ข

คู่มือการใช้งานอย่างละเอียด
DETAILED USER GUIDE

ข.1 Quick start สำหรับผู้วางแผน
PLANNER QUICK START

1. เปิด Plan แล้วเลือกศูนย์ต้นทาง (depot) บนแผนที่
2. เพิ่มหรือนำเข้าจุดงาน ระบุ demand, service time และ time window เมื่อใช้
3. ไปที่ Roads & incidents แล้วเตรียมโครงข่าย ตรวจจุดที่เข้าไม่ถึงก่อนดำเนินการต่อ
4. ตรวจแผนเบื้องต้นแบบเส้นประ ใช้ขั้นนี้ตรวจข้อมูล ไม่ใช้เป็นหลักฐานว่า optimizer รันแล้ว
5. ไปที่ Optimizer กำหนด K เป็นจำนวนรถสูงสุด ระบุ Q, objective และงบเวลา
6. เริ่มรันและติดตาม elapsed time, logs และ convergence โดยไม่ปิด session
7. เมื่อสถานะเป็นแผนรถสมบูรณ์ ตรวจ validation summary, used trucks และ route version
8. เปิด Dispatch เพื่อส่งใบงาน จำลอง planned route หรือ export เอกสาร/หลักฐาน

ตารางที่ ข-1 คู่มืออ่านสถานะแผน

สถานะ	ความหมาย	สิ่งที่ควรทำ
ยังไม่มีโครงข่าย	ยังคำนวณต้นทุนถนนไม่ได้	กำหนดพื้นที่และ build/download road graph
โครงข่ายพร้อม	กราฟและ terminal costs พร้อม	ตรวจ reachability และข้อมูลจุดงาน
แผนเบื้องต้นพร้อม	มี feasible prepared plan แต่ C++ ยังไม่จบ	ตรวจข้อมูลหรือเริ่ม optimizer
กำลังจัดเส้นทาง	optimizer ทำงาน	ดู elapsed/log; หลีกเลี่ยงแก้ input กลางรอบ

สถานะ	ความหมาย	สิ่งที่ควรทำ
แผนรถสมบูรณ์	optimizer จบและ validator ผ่าน	ตรวจและส่งต่อ route version
แผนล้าสมัย	กราฟ/จุดงานเปลี่ยนหลัง route version	เก็บแผนเดิมไว้อ้างอิงและรันใหม่
รอบใหม่ไม่สำเร็จ	failed/cancelled/invalid candidate	ใช้ previous valid plan และตรวจ evidence

ข.2 คู่มือคนขับ

DRIVER APP

- เลือกเฉพาะรถที่ได้รับมอบหมาย รถที่ไม่ถูกใช้จะไม่ปรากฏเป็นใบงานว่าง
- ตรวจ route version, ลำดับจุด, demand/service และระยะ/เวลาประมาณก่อนออกงาน
- ใช้ปุ่ม navigation handoff เพื่อเปิดแผนที่ภายนอกตามจุดหมาย ไม่ตีความเส้นจำลองในแอปเป็นคำสั่งขับรถอัตโนมัติ
- ทำเครื่องหมายความคืบหน้าตามขั้นตอนที่ระบบบรองรับ โดยรักษาคำสั่งของผู้ควบคุมงานเป็นหลัก
- เมื่อพบถนนเข้าไม่ได้ ให้เลือก report road issue แล้วเลือก segment บนแผนที่ พร้อมระบุทิศทางและหมายเหตุ

ความปลอดภัยมาก่อนเส้นทาง

ผู้ขับต้องปฏิบัติตามกฎหมาย ป้ายจราจร สภาพหน้างาน และคำสั่งของผู้ควบคุม ไม่ควรโต้ตอบกับหน้าจอระหว่างขับรถ และไม่ควรเข้าสู่ถนนที่ไม่ปลอดภัยเพียงเพราะเส้นทางในแอปแนะนำ

ข.3 รายงานและอนุมัติเหตุถนนปิด

ROAD INCIDENTS

1. คนขับหรือผู้แจ้งเลือก segment ที่ตรงกับเหตุจริงและส่งรายงาน
2. Dispatcher เปิด incident inbox ตรวจตำแหน่ง geometry ทิศทาง และหมายเหตุ
3. เลือกอนุมัติหรือปฏิเสธ หากข้อมูลไม่พอให้ตรวจภาคสนามก่อน
4. เมื่ออนุมัติ ระบบอัปเดต directed graph ทำให้ input/route เดิมเป็น stale
5. ผู้วางแผนตรวจจุดเข้าถึงและรัน optimizer ใหม่
6. ยืนยัน route version ใหม่หลัง validator ผ่าน แล้วแจ้งคนขับถึงการเปลี่ยนแปลง

หากเหตุการณ์สิ้นสุด ให้เปิดถนนคืนผ่าน workflow ที่ระบบกำหนด ระบบจะกู้ข้อมูลจาก graph contract ที่ถูกต้อง ไม่สร้างขอบถนนจากชื่อหรือระยะเส้นตรงเอง หลังเปลี่ยนกราฟต้องรันแผนใหม่อีกครั้ง

ข.4 การส่งออกและตรวจหลักฐาน

EXPORT AND AUDIT

ตารางที่ ข-2 จุดตรวจหลักฐาน

ต้องการ	เปิดดู/ส่งออก	ตรวจอย่างน้อย
แผนปฏิบัติงาน	Dispatch / route manifest	version, vehicle, stop order, totals
ยืนยัน input	dataset summary / hash	N, M, K, Q, revision, SHA-256
ยืนยันการรัน	solver invocation	argv, cwd, stdin hash, exit state
ยืนยันคำตอบ	validator / routes JSON	served once, overload, lateness, objective
วิเคราะห์รอบ	convergence, stdout/stderr, job events	seed, global best, error/cancel/timeout

ค สัญญาข้อมูลและไฟล์หลักฐาน
DATA AND EVIDENCE CONTRACT

ค.1 ความหมายของตัวแปรหลัก
CORE SEMANTICS

ตารางที่ ค-1 Data dictionary ฉบับย่อ

ชื่อ	ชนิด/หน่วย	ความหมาย
N, M	จำนวนเต็ม	จำนวน node และ directed edge ใน solver graph
K	จำนวนเต็ม	จำนวนรถพร้อมใช้สูงสุด ไม่ใช่จำนวน route บังคับ
Q	หน่วย demand	ความจุต่อเที่ยว; reset เมื่อกลับ depot
q_i	หน่วย demand	ความต้องการของจุดงาน i
s_i	วินาที	เวลาบริการที่จุด i
$[a_i,b_i]$	วินาทีจากจุดเริ่มวัน	ช่วงเวลาเริ่มบริการเมื่อเปิด time-window enforcement
d_{ij}	เมตร	shortest directed-road distance ระหว่าง terminals
t_{ij}	วินาที	shortest directed-road travel time ตาม snapshot/profile
route version	identifier	แผนที่ commit แล้วซึ่งผูก data revision และ evidence

ค.2 เกณฑ์ตรวจคำตอบ
INDEPENDENT VALIDATION

```
Route acceptance gate
coverage_ok := every customer appears exactly once
capacity_ok := load(trip) <= Q for every depot-delimited trip
time_window_ok := service_start(i) within [earliest_i, latest_i]
fleet_ok := used_vehicle_count <= K and no empty route is published
objective_ok := parsed totals agree with recomputation within tolerance
provenance_ok := invocation input hash == active materialized input hash
commit_allowed := all checks above are true
```

ตัวตรวจไม่เชื่อ summary line ของ solver เพียงอย่างเดียว แต่คำนวณ coverage, trip loads, arrival/service times และ fleet totals จาก route sequence กับ dataset ที่ผูก hash ใหม่ หากการ parse ไม่ครบหรือค่าเกิน tolerance รอบนั้นจะไม่ถูกบันทึกเป็นแผนใช้งาน

ค.3 ไฟล์หลักฐานต่อรอบ

RUN ARTIFACTS

ตารางที่ ค-2 Evidence bundle

ไฟล์	สร้างเมื่อ	ใช้ตอบคำถาม
dataset_summary.json	materialize input	ข้อมูลชุดใด ขนาดเท่าใด และ contract ผ่านหรือไม่
solver_invocation.json	ก่อน spawn C++	รัน executable ใด ด้วย argument/input/hash อะไร
solver_output.txt	ระหว่าง/หลังรัน	ตัวแก้ส่ง route และ objective ใด
solver_stderr.txt	ระหว่าง/หลังรัน	มี heartbeat, warning หรือ error อะไร
convergence.csv	ระหว่าง search	คำตอบดีขึ้นเมื่อใดและมาจาก worker ใด
routes.json	หลัง parse/validate	ลำดับรถ จุดงาน และ totals ที่ยืนยันแล้ว

ไฟล์	สร้างเมื่อ	ใช้ตอบคำถาม
route version record	atomic commit	แผนใด active, stale หรือ previous-valid

ค.4 กฎเหตุการณ์ถนน

DIRECTED EDGE EVENT CONTRACT

- รายงานต้องอ้าง edge identity/geometry ที่ผู้ใช้เลือก ไม่อ้างเพียงชื่อถนน
- ปิดทิศเดียวลบเฉพาะ directed edge ที่เลือก; ปิดสองทิศอาจลบ reverse edge ที่ geometry matching ได้หนึ่งคู่
- ห้ามลบ parallel edges หรือถนนชื่อเดียวกันทั้งหมดโดยอัตโนมัติ
- เมื่อ active road revision เปลี่ยน input และ route ที่สร้างจาก revision เก่าต้องเป็น stale
- การเปิดถนนคืนใช้ source contract/snapshot และสร้าง revision ใหม่ ก่อนคำนวณ route version ใหม่

ง

รายการทดสอบและเกณฑ์ตรวจรับระบบ

TEST INVENTORY AND ACCEPTANCE CRITERIA

ง.2 เกณฑ์ตรวจรับองค์ประกอบสำคัญ

SYSTEM ACCEPTANCE MATRIX

ตารางที่ ง-1 เกณฑ์ตรวจรับระบบและสถานะการทดสอบ

องค์ประกอบ	Acceptance criteria	สถานะ
การส่งต่อแผนสู่ผู้ขับ	มีใบงานตามรถ navigation handoff และไม่แสดงรถว่าง	route-version และ manifest evidence พร้อม
ความชัดเจนของ UX	map-first, guided stages, light map, normal/advanced separation	persisted state พร้อม; field study เป็นขั้นถัดไป
ข้อมูลย้อนกลับจากภาคสนาม	ส่ง road-segment report พร้อม geometry และทิศทาง	report payload + geometry evidence พร้อม
ปรับเส้นทางเมื่อถนนเปลี่ยน	approval → graph revision → stale route → validated rerun	event-driven transaction พร้อม
ระบบต้องสื่อสถานะจริง	routing complete เฉพาะ native solver จบและ validator ผ่าน	native exit + validator gate
การทดสอบนำร่อง	controlled pilot พร้อม fallback และ metric	แผนงานระยะถัดไป

ง.3 Checklist ก่อนสาธิต

DEMONSTRATION CHECKLIST

- รัน notebook ใหม่ครบ 2 เซลล์โดยไม่อาศัย state จาก session เดิม
- เตรียมข้อมูลสาธิตขนาดกลางและหลักฐานสำรองกรณีเครือข่ายไม่เสถียร
- แสดงลำดับสถานะ network ready → initial plan → optimizer running → validated plan
- ตั้ง K สูงกว่าจำนวนรถที่ใช้จริง และตรวจว่า Driver App ซ่อนรถว่าง
- สาธิต road closure ตามลำดับ exact segment → approval → stale route → rerun
- เปิด evidence inspector เพื่อตรวจ route version, input hash, command, logs และ validation
- ปิดท้ายด้วยขอบเขตเหตุการณ์ถนนปัจจุบันและแผน GPS/traffic/field pilot

จ ข้อตกลงในการใช้ซอฟต์แวร์และการเปิดเผยการใช้ AI
SOFTWARE DISCLAIMER AND AI DISCLOSURE

จ.1 ข้อจำกัดการใช้งาน
OPERATIONAL DISCLAIMER

- ระบบเป็นซอฟต์แวร์วิจัยและเครื่องมือช่วยตัดสินใจ การนำทางและการควบคุมยานพาหนะยังเป็นหน้าที่ของผู้ปฏิบัติงานและเครื่องมือที่ได้รับการรับรอง
- ผู้ใช้งานต้องตรวจเส้นทางกับกฎหมาย ป้ายจราจร น้ำหนัก/ขนาดรถ สภาพอากาศ ความปลอดภัย และข้อกำหนดของหน่วยงาน
- ข้อมูล OpenStreetMap อาจไม่ครบหรือล้าสมัย การมีเส้นทางในกราฟไม่ยืนยันว่ารถจริงผ่านได้
- ผล benchmark หลักยังจำกัดที่สามรันต่อขนาดและชุดถนนจริงสี่ชุด แม้ตัวแก้ของโครงการและ OR-Tools จะถูกรันภายใต้ Google Colab VM / Intel Xeon 2.4 GHz protocol เดียวกัน. กรณี 100 จุดมีการทดลองจับคู่เพิ่ม 5 seeds ระหว่างรุ่นรอบภูมิภาคกับ source รุ่นปัจจุบันภายใต้การตั้งค่าวิเคราะห์ 100 วินาที แต่ยังเป็น cross-version diagnostic ไม่ใช่ same-build one-factor ablation. เนื่องจากนิยาม terminal path ของ executable รุ่นก่อนกับรุ่นปัจจุบันต่างกัน การตัดสินข้าม solver จึงใช้ route evaluator กลางส่วน native deltas ระหว่างรุ่นถูกจำกัดสถานะไว้เป็น version diagnostic
- การใช้กับของเหลวหรือโดเมนเฉพาะต้องเพิ่มและตรวจข้อจำกัดโดยผู้เชี่ยวชาญก่อนนำไปปฏิบัติ

จ.2 การใช้ข้อมูลและซอฟต์แวร์ภายนอก
THIRD-PARTY MATERIALS

ข้อมูลถนนมาจาก OpenStreetMap และต้องใช้งานตามข้อกำหนดการให้เครดิต/สัญญาอนุญาตของแหล่งข้อมูล ไลบรารีและเครื่องมือภายนอกมีลิขสิทธิ์และเงื่อนไขของผู้พัฒนาแต่ละราย รายงานนี้

อ้างอิงบทบาทของซอฟต์แวร์ดังกล่าวและไม่รวมเป็นผลงานใหม่ของโครงการ

จ.3 การเปิดเผยการใช้ปัญญาประดิษฐ์

GENERATIVE AI DISCLOSURE

ในกระบวนการพัฒนา ผู้พัฒนาใช้เครื่องมือปัญญาประดิษฐ์เชิงกำเนิดเป็นผู้ช่วยด้านการทบทวนโครงสร้างโค้ด การตรวจความสอดคล้องของถ้อยคำ การปรับ UI/UX การจัดระบบเอกสาร และการเสนอกรณีทดสอบ การใช้เครื่องมือดังกล่าวไม่ถูกนับเป็นผลทดลองและไม่ถูกใช้แทน source inspection, automated tests, controlled benchmark หรือการประเมินผู้ใช้

ผู้พัฒนาเป็นผู้รับผิดชอบการเลือกแบบจำลอง การตรวจ source และผลลัพธ์

> หลักความรับผิดชอบ
> AI assistance ไม่เปลี่ยนเกณฑ์หลักฐาน: ข้อสรุปเชิงระบบต้องย้อนถึง source inspection และ contract/state evidence ข้อสรุปเชิงสมรรถนะต้องอ้าง controlled benchmark และข้อสรุปด้านผู้ใช้ต้องอ้างการศึกษากับผู้ใช้จริง

จ.4 ขอบเขตความเป็นเจ้าของผลงาน

AUTHORSHIP BOUNDARY

ตารางที่ จ-1 ขอบเขตส่วนสมทบและเครื่องมือ

องค์ประกอบ	สถานะ
แนวคิดระบบและ workflow	พัฒนาในโครงการ
source ใน notebook และ test suite	พัฒนา/ปรับและตรวจโดยผู้พัฒนา
ข้อมูล OSM และไลบรารีภายนอก	องค์ประกอบภายนอก ใช้ตามสิทธิ์ผู้ให้บริการ
OR-Tools / CP-SAT	เครื่องมือภายนอกสำหรับ baseline และ exact validation
AI-generated suggestions	เครื่องมือช่วย; ผู้พัฒนาคัดเลือก ตรวจ และรับผิดชอบผลสุดท้าย